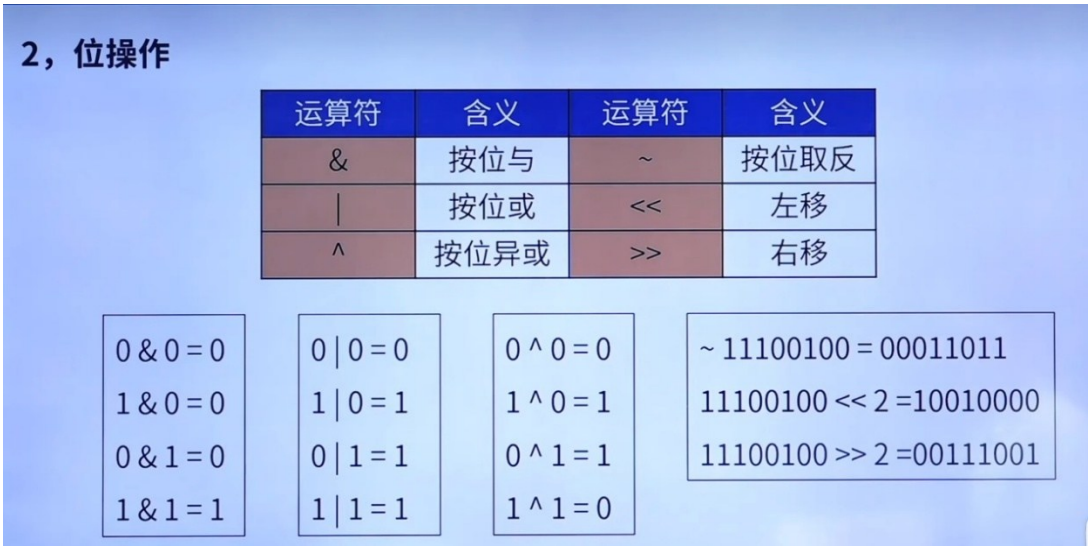
1 位操作



多位操作通用公式

1. 置 1（无条件设为 1）

```c
temp |= (1U << a) | (1U << b) | (1U << c);   // a, b, c 是位号，可连续可不连续
```

- 示例：位 6、位 9、位 12 置 1

```
temp |= (1U << 6) | (1U << 9) | (1U << 12);
```

2. 清 0（无条件设为 0）

```c
temp &= ~( (1U << a) | (1U << b) | (1U << c) );
```

- 示例：位 6、位 9、位 12 清 0

```
temp &= ~( (1U << 6) | (1U << 9) | (1U << 12) );
```

3. 翻转（取反切换）

```c
temp ^= (1U << a) | (1U << b) | (1U << c);
```

- 示例：同时翻转位 6 和位 9

```
temp ^= (1U << 6) | (1U << 9);
```

2 宏定义

带参数的宏定义

```
#define LED1(x)   do{ x ? \
                HAL_GPIO_WritePin(LED1_GPIO_PORT, LED1_GPIO_PIN, GPIO_PIN_SET) : \
                HAL_GPIO_WritePin(LED1_GPIO_PORT, LED1_GPIO_PIN, GPIO_PIN_RESET); \
            }while(0)
```

建议大家使用 do{ ... }while(0) 来 构造宏定义

这样不会受到大括号、分号、运算符优先级等的影响，总是会按你期望的方式调用运行!

注意：**以上为三目运算符**

3 条件编译

条件编译

让编译器只对满足条件的代码进行编译，不满足条件的不参与编译!

指令	作用
#if	编译预处理条件指令，类似if
#ifdef	判断某个宏是否已被定义
#ifndef	判断某个宏是否未被定义
#elif	若前面的条件不满足，则判定新的条件，类似else if
#else	若前面的条件不满足，则执行后面的语句，类似else
#endif	#if，#ifdef，#ifndef的结束标志

```
#ifndef  _LED_H
#define  _LED_H
#include "./SYSTEM/sys/sys.h"
code
#endif
```

头文件的条件编译

```
#if  SYS_SUPPORT_OS
    code
#endif
```

代码条件编译

4 Extern 声明

extern声明

放在函数/变量前，表示此函数/变量在其他文件定义，以便本文件引用

```
extern uint16_t g_usart_rx_sta;
extern void delay_us(uint32_t nus);
```

5 类型别名（typedef）

类型别名（typedef）

为现有数据类型创建一个新的名字，或称为类型别名，用来简化变量的定义

```
typedef  现有类型  新名字
```

```
typedef  unsigned char          uint8_t;
typedef  unsigned short int     uint16_t;
typedef  unsigned int           uint32_t;
```

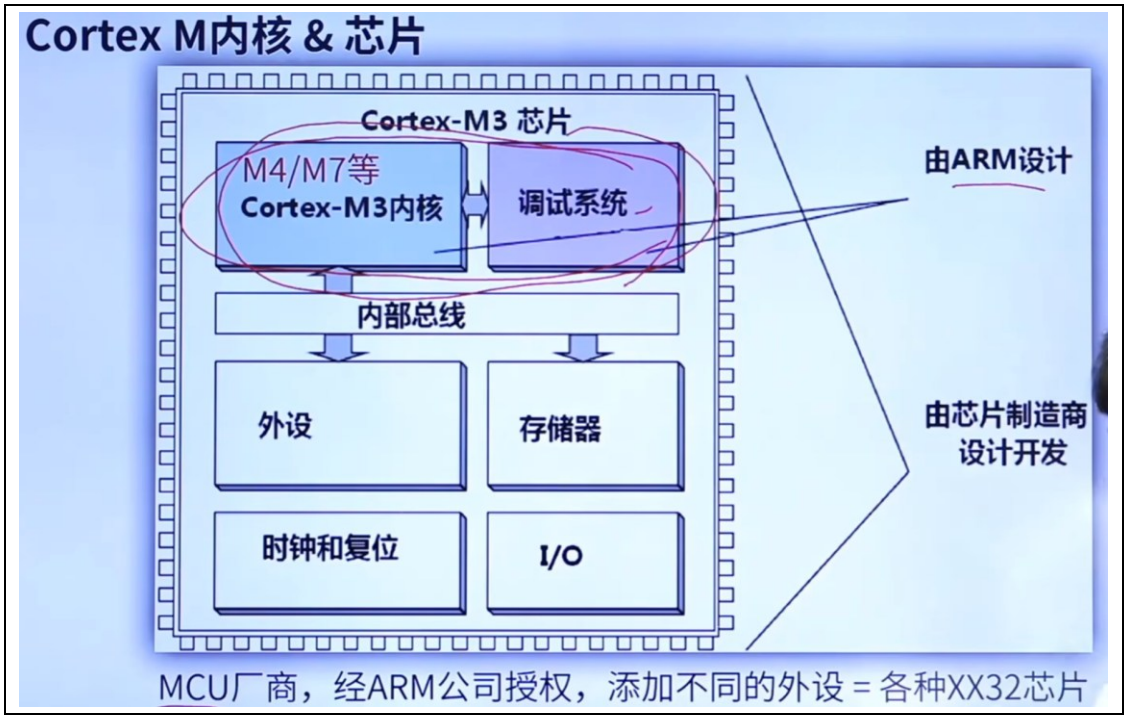
5.1 类型别名应用

```
Struct GPIO_TypeDef
{
   __IO uint32_t  CRL;
   __IO uint32_t  CRH;
   …
};
Struct   GPIO_TypeDef   gpiox
```

```
typedef struct
{
   __IO uint32_t  CRL;
   __IO uint32_t  CRH;
   …
} GPIO_TypeDef;
GPIO_TypeDef   gpiox
```

6 STM32 基础篇

6.1 Cortex M 内核&芯片



6.1.1 F1 系列

6.1.1.1 F1 系统架构

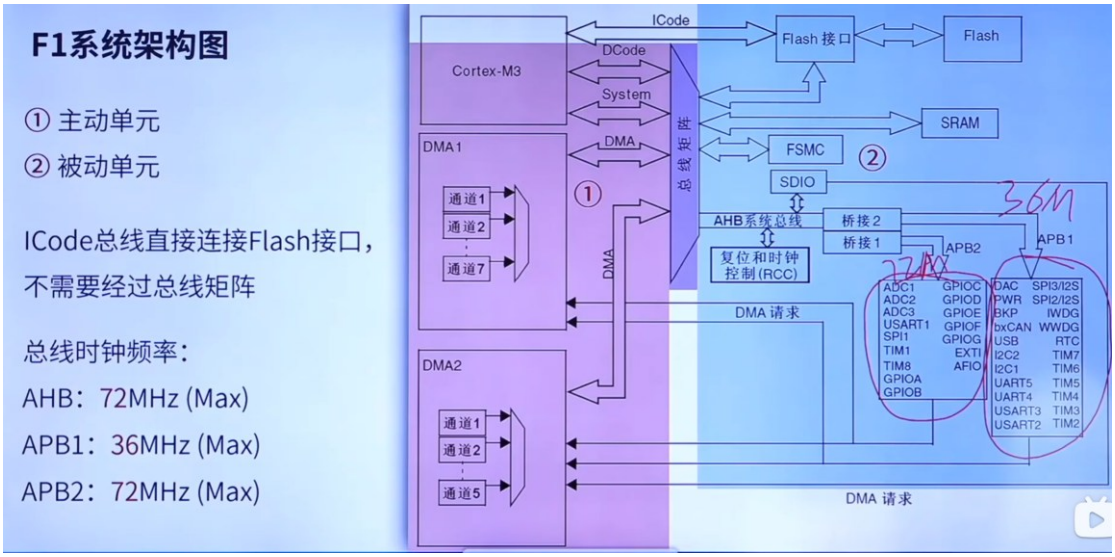
4个主动单元 + 4个被动单元

主动单元	被动单元
Cortex M3内核 DCode总线（D-Bus）	内部FLASH
Cortex M3内核 系统总线（S-Bus）	内部SRAM
通用DMA1	FSMC
通用DMA2	AHB到APB的桥，它连接的所有APB外设

AHB：高级高性能总线
APB：高级外围总线

参考：STM32F10xxx参考手册_V10（中文版）.pdf 2.1小节

6.1.1.2 F1 系统架构图



6.1.2 F4 系列

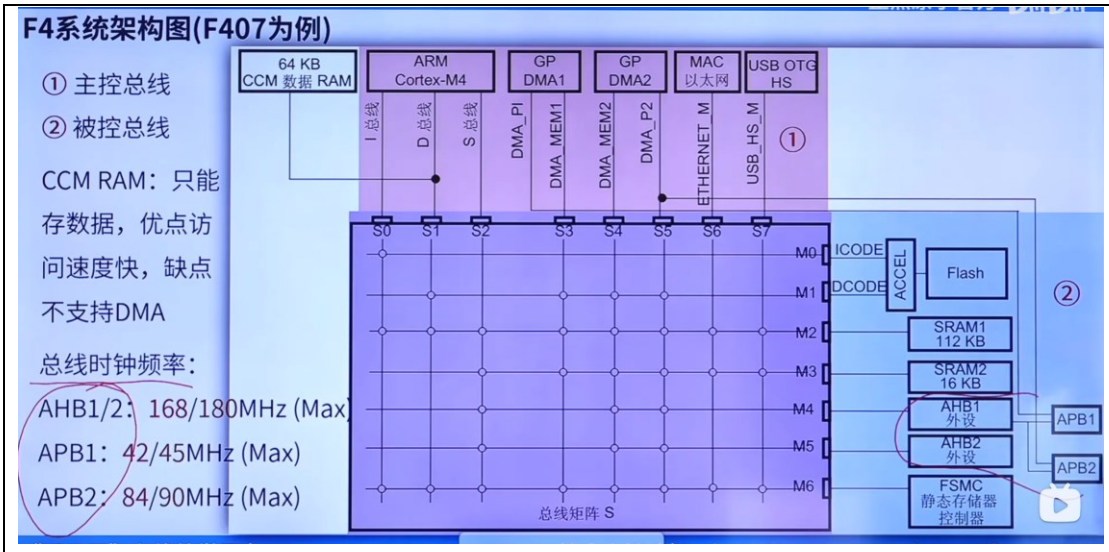
6.1.2.1 F4 系统架构

8个主控总线 + 7个被控总线

主控总线	被控总线
Cortex M4内核 I总线	内部FLASH Icode 总线
Cortex M4内核 D总线	内部FLASH Dcode 总线
Cortex M4内核 S总线	主要内部SRAM1（112KB）
DMA1存储器总线	辅助内部SRAM2（16KB）
DMA2存储器总线	辅助内部SRAM3（64KB）（适用于F42xxx和F43xxx）
DMA2外设总线	AHB1外设（包括AHB-APB总线桥和APB外设）
以太网DMA总线	AHB2外设
USB OTG HS DMA总线	FSMC

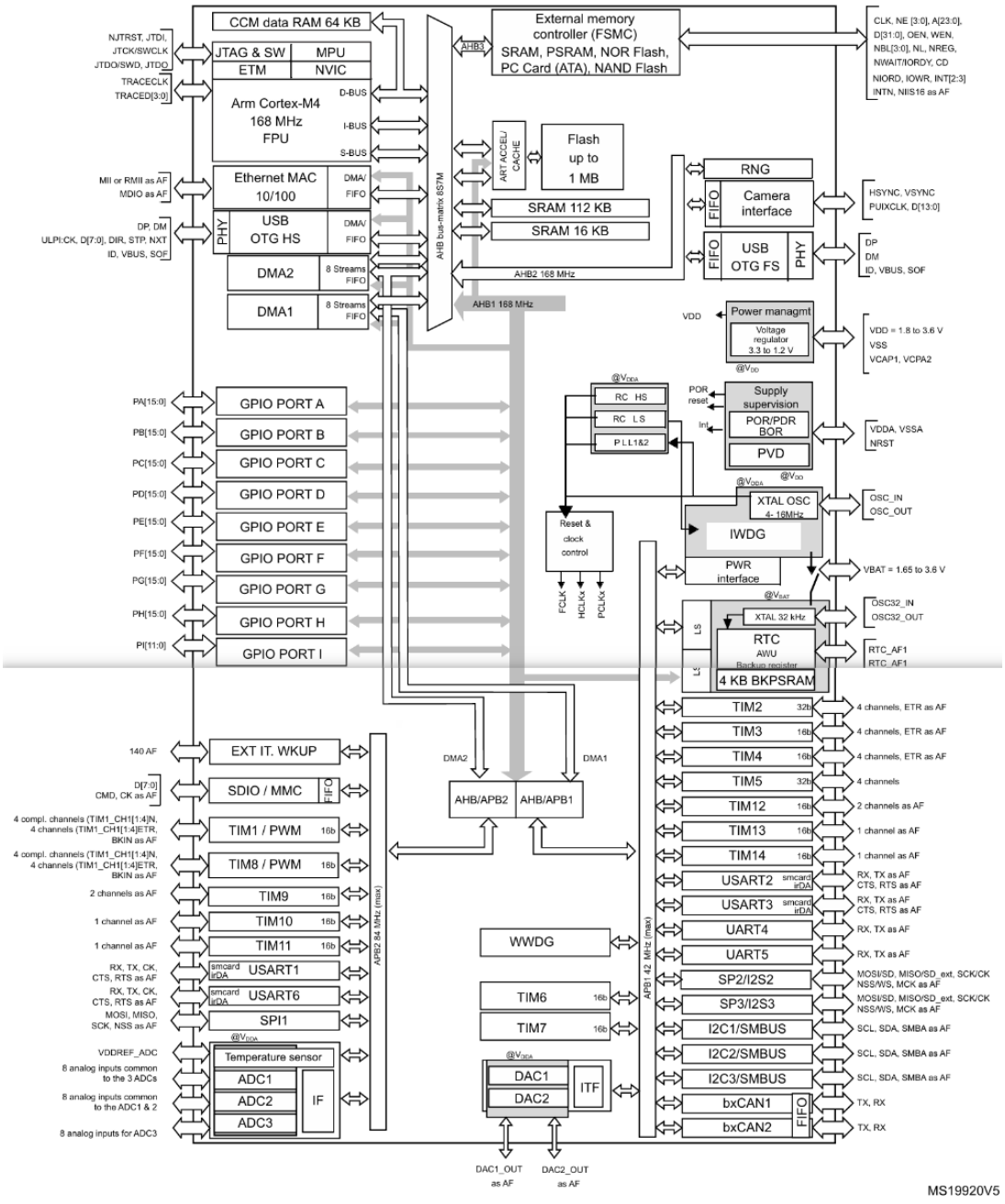
摘自：STM32F4xx参考手册_V4（中文版）.pdf 2.1小节

6.1.2.2 F4 系统架构图（F407 为例）



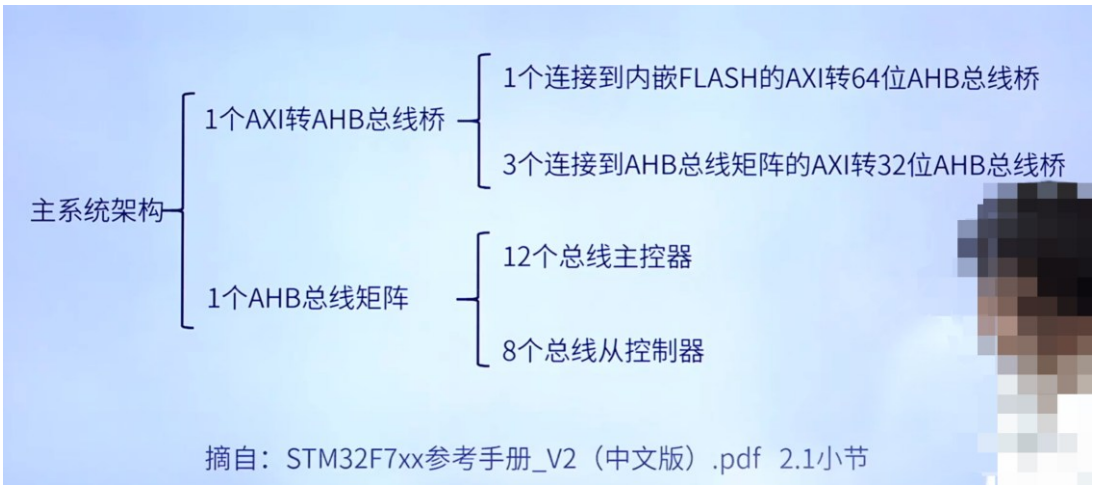
完整架构图见 stm32f407ZGT6-英文版第 20 页：

Figure 5. STM32F40xxx block diagram



1. The camera interface and ethernet are available only on STM32F407xx devices.

6.1.3 F7 系统架构



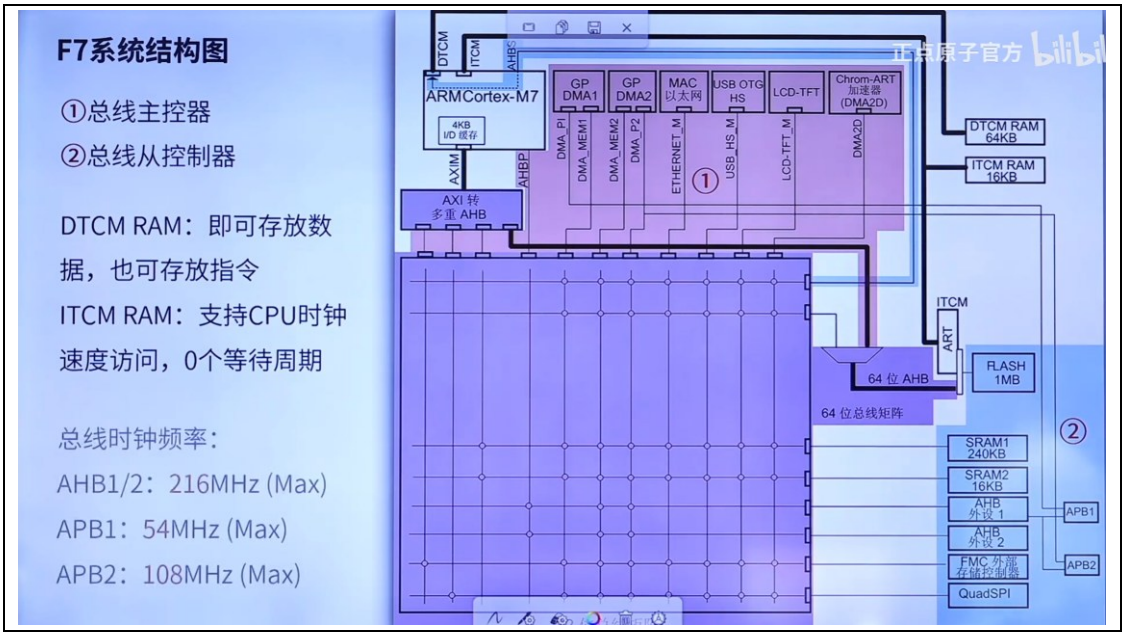
6.1.3.1 多重 AHB 总线矩阵

多重AHB总线矩阵

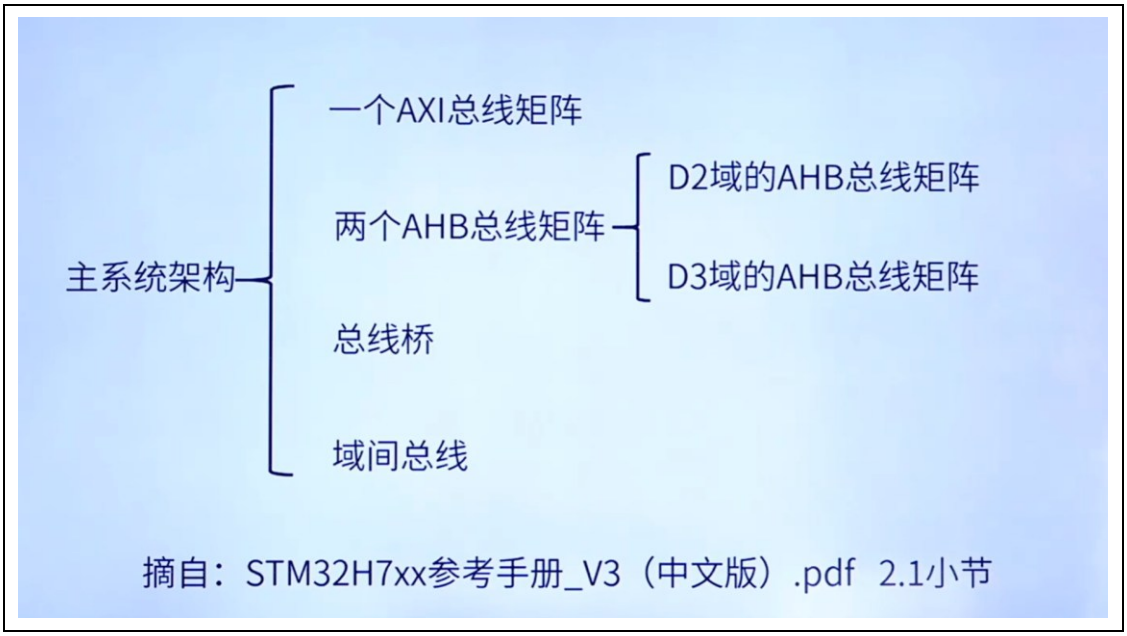
12个总线主控器 + 8个总线从控制器

总线主控器	总线从控制器
3x32位AHB总线	AHB总线上的内嵌Flash
连接到内嵌Flash的64位AHB总线	Cortex M7 AHBS从接口(仅用于DTCM RAM的DMA数据传输)
AHBP总线	主SRAM1（240KB）
DMA1存储器总线	辅助SRAM2（16KB）
DMA2存储器总线	AHB1外设（包括AHB-APB总线桥和APB外设）
DMA2外设总线	AHB2外设（包括AHB-APB总线桥和APB外设）
以太网DMA总线	FMC
USB OTG HS DMA总线	Quad SPI
LCD控制器DMA总线	
DMA2D存储器总线	

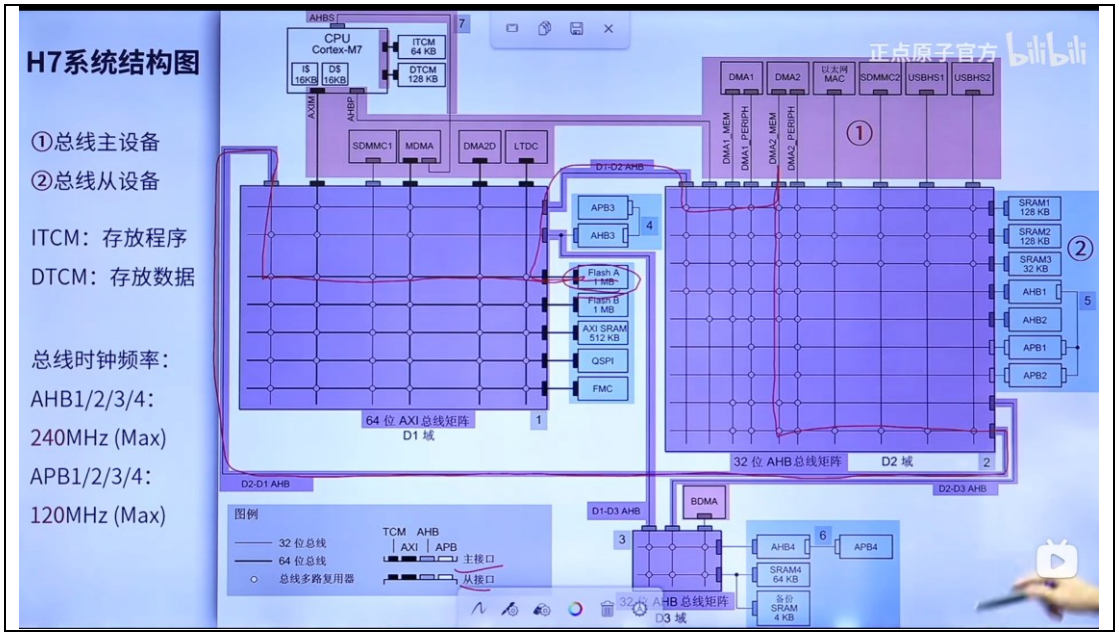
6.1.3.2 F7 系统结构图



6.1.4 H7 系统架构



6.1.4.1 H7 系统结构图



6.2 存储器映射

6.2.1 STM32 的寻址范围

STM32的寻址范围?

想要解答这个问题，我们需要清楚以下两点：

1，32位的单片机可以有32根地址线（每根地址线有两种状态：导通或不导通）

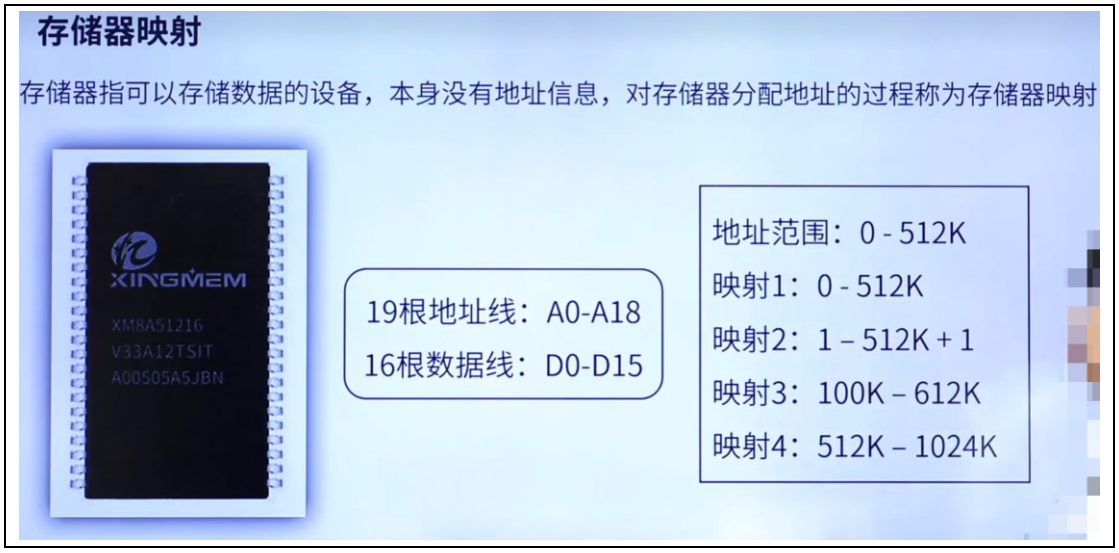
2，单片机内存地址访问的存储单元是按字节编址的（而不是bit）

地址线根数	地址编号(二进制)	地址编号数(即内存大小)
1	0、1	2
2	00、01、10、11	4
3	000、001、010、011、100、101、110、111	8
n	……	2^n

STM32寻址大小：2^{32} = 4G（字节）

STM32寻址范围：0x0000 0000 ~ 0xFFFF FFFF

6.2.2 存储器映射



6.2.3 存储器功能划分（F1 为例）

6.2.3.1 Block0（FLASH）功能划分

6.2.3.2 Block1（SRAM）功能划分

Block1（SRAM）功能划分

存储块	功能	地址范围
Block 1	SRAM	0X2000 0000 ~ 0x2000 FFFF（64KB）
	保留	0X2001 0000 ~ 0x3FFF FFFF

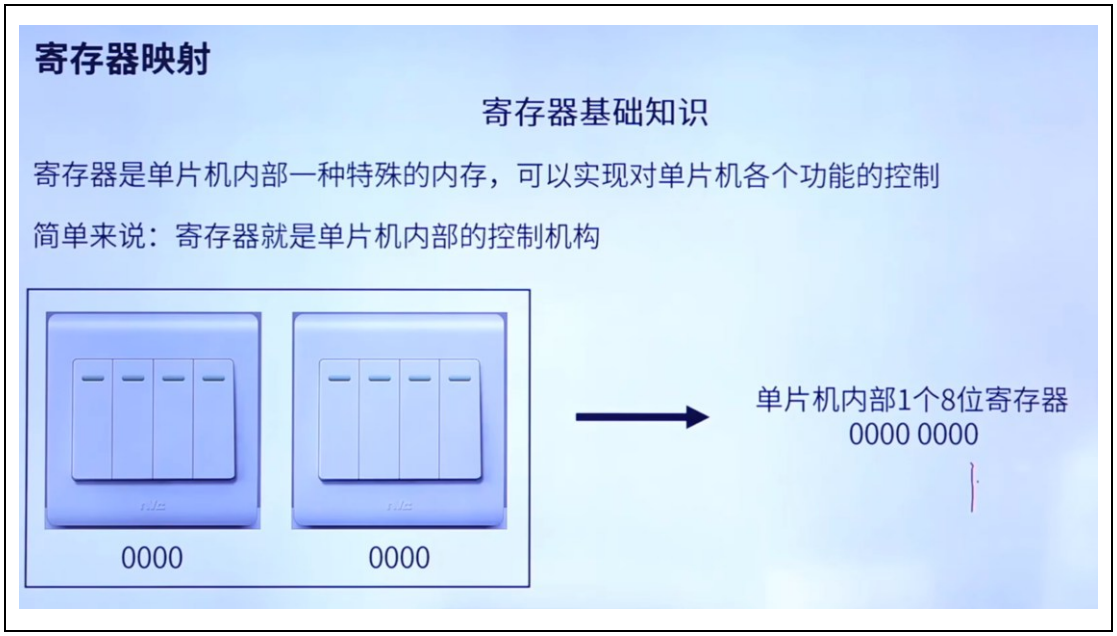
6.2.3.3 Block2（外设）功能划分

Block2（外设）功能划分

存储块	功能	地址范围
Block2	APB1总线外设	0X4000 0000 ~ 0x4000 77FF
	保留	0X4000 7800 ~ 0x4000 FFFF
	APB2总线外设	0X4001 0000 ~ 0x4000 3FFF
	保留	0X4001 4000 ~ 0x4001 7FFF
	AHB总线外设	0X4001 8000 ~ 0x4002 33FF
	保留	0X4002 3400 ~ 0x5FFF FFFF

6.3 寄存器映射

6.3.1 寄存器基础知识

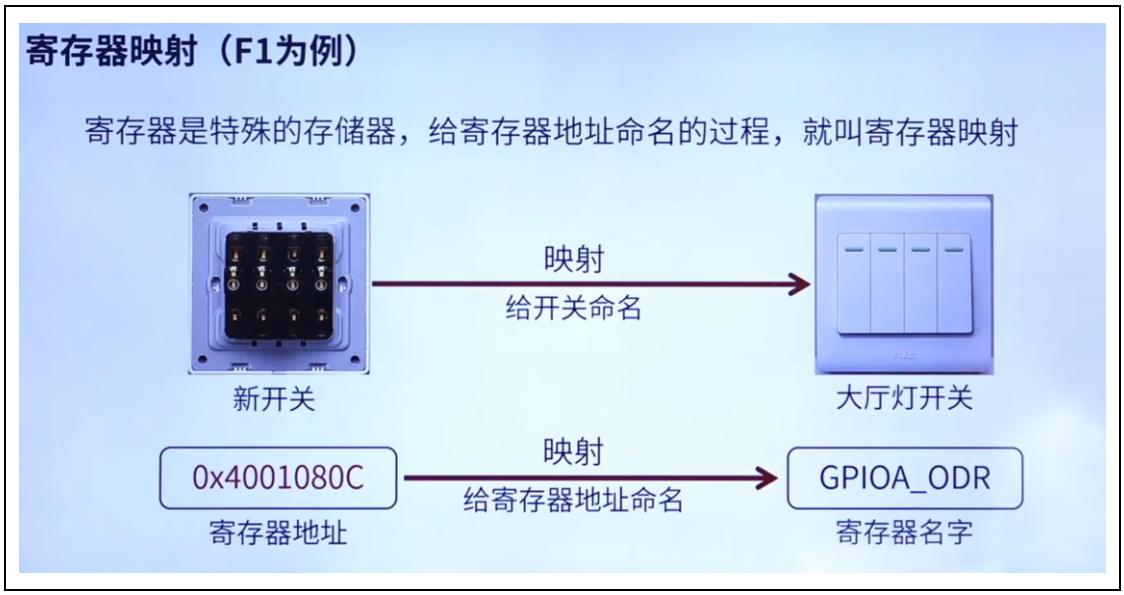


6.3.2 STM32 寄存器分类

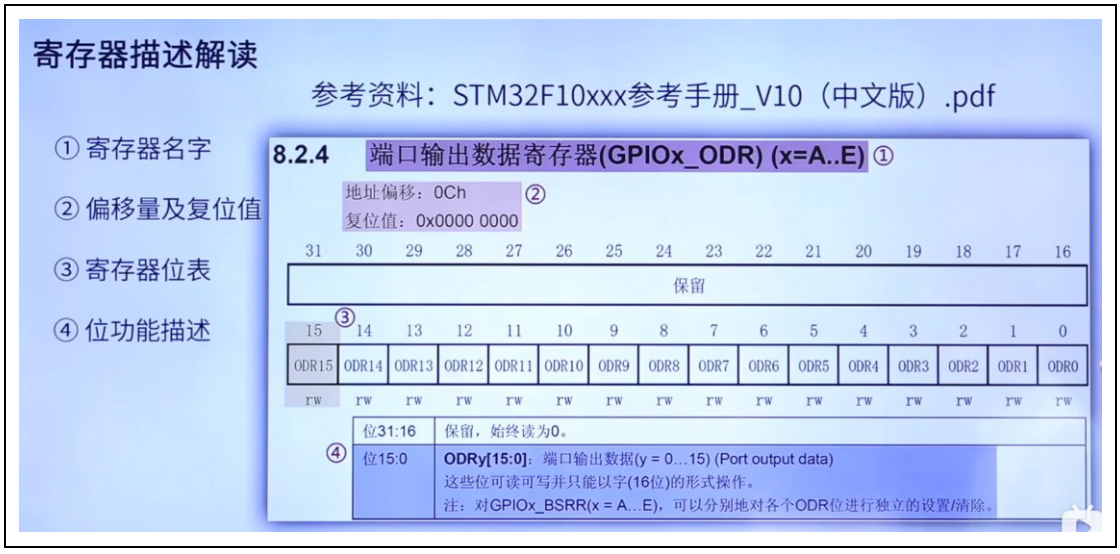
STM32寄存器分类

大类	小类	说明
内核寄存器	内核相关寄存器	包含R0~R15、xPSR、特殊功能寄存器等
	中断控制寄存器	包含NVIC和SCB相关寄存器，NVIC有：ISER、ICER、ISPR、IP等；SCB有：VTOR、AIRCR、SCR等
	SysTick寄存器	包含CTRL、LOAD、VAL和CALIB四个寄存器
	内存保护寄存器	可选功能，STM32F103没有
	调试系统寄存器	ETM、ITM、DWT、IPIU等相关寄存器
外设寄存器		包含GPIO、UART、IIC、SPI、TIM、DMA、ADC、DAC、RTC、I/WWDG、PWR、CAN、USB等各种外设寄存器

6.3.3 寄存器映射（F1 为例）



6.3.4 寄存器描述解读



6.3.5 寄存器映射举例

直接操作寄存器地址：

```
*(unsigned int *)(0x4001 080C) = 0XFFFF;
```

定义一个名字后再操作：

```
#define   GPIOA_ODR       *(unsigned int *)(0x4001 080C)
GPIOA_ODR = 0XFFFF;
```

6.3.6 寄存器地址计算

寄存器地址计算

为了方便编写代码及使用，我们将寄存器地址分为三个部分：

1、总线基地址（BUS_BASE_ADDR）

2，外设基于总线基地址的偏移量（PERIPH_OFFSET）

3，寄存器相对外设基地址的偏移量（REG_OFFSET）

寄存器地址 = BUS_BASE_ADDR + PERIPH_OFFSET + REG_OFFSET

6.3.6.1 总线基地址

总线基地址

总线	基地址	偏移量
APB1	0X4000 0000	0
APB2	0X4001 0000	0X1 0000
AHB	0X4001 8000	0X1 8000

APB1总线的基地址，也叫外设基地址（PERIPH_BASE）

此表的偏移量：是相对外设基地址（PERIPH_BASE）来说的

参考资料：STM32F10xxx参考手册_V10（中文版）.pdf 2.3小节

6.3.6.2 GPIO 外设基地址及偏移量

GPIO外设基地址及偏移量

所属总线	外设	基地址	偏移量
APB2 0X4001 0000	GPIOA	0X4001 0800	0X800
	GPIOB	0X4001 0C00	0XC00
	GPIOC	0X4001 1000	0X1000
	GPIOD	0X4001 1400	0X1400
	GPIOE	0X4001 1800	0X1800
	GPIOF	0X4001 1C00	0X1C00
	GPIOG	0X4001 2000	0X2000

此表的偏移量：是相对APB2外设基地址（APB2PERIPH_BASE）来说的

6.3.6.3 GPIOA 外设基地址及偏移量

GPIOA外设基地址及偏移量

所属总线	所属外设	寄存器	地址	偏移量
APB2 0X4001 0000	GPIOA 0X4001 0800	GPIOA_CRL	0X4001 0800	0X00
		GPIOA_CRH	0X4001 0804	0X04
		GPIOA_IDR	0X4001 0808	0X08
		GPIOA_ODR	0X4001 080C	0X0C
		GPIOA_BSRR	0X4001 0810	0X10
		GPIOA_BRR	0X4001 0814	0X14
		GPIOA_LCKR	0X4001 0818	0X18

此表的偏移量：是相对GPIOA外设基地址（GPIOA_BASE）来说的

6.3.6.4 GPIOA_ODR 寄存地址计算过程

GPIOA_ODR寄存器地址计算过程：

1，获取外设挂在哪个总线上面？查：系统结构图

2，获取总线基地址，APB2总线基地址：0X4001 0000

3，获取外设地址偏移量，GPIOA相对APB2总线偏移量是：0X800

4，获取寄存器地址偏移量，ODR相对GPIOA外设基地址的偏移量是：0X0C

寄存器地址 = BUS_BASE_ADDR + PERIPH_OFFSET + REG_OFFSET

GPIOA_ODR = 0X4001 0000 + 0X800 + 0X0C = 0X4001 080C

6.3.6.5 结构体映射寄存器

使用结构体，可以很方便的完成对寄存器的映射：

```
typedef struct
{
  __IO uint32_t CRL;
  __IO uint32_t CRH;
  __IO uint32_t IDR;
  __IO uint32_t ODR;
  __IO uint32_t BSRR;
  __IO uint32_t BRR;
  __IO uint32_t LCKR;
} GPIO_TypeDef;

GPIOA_BASE：  0X4001 0800
```

```
#define  GPIOA   ((GPIO_TypeDef *)GPIOA_BASE)
&GPIOA->CRL：  0X4001 0800
&GPIOA->CRH：  0X4001 0804
&GPIOA->IDR：  0X4001 0808
&GPIOA->ODR：  0X4001 080C
实际应用：
GPIOA-> ODR = 0XFFFF;
```

详见：stm32f103xe.h，725行开始

stm32f103xe.h主要组成部分

文件	主要组成部分	说明
stm32 f103xe.h	中断编号定义	定义IRQn_Type枚举类型，包含STM32F103内部所有中断编号（中断号），方便后续编写代码
	外设寄存器结构体类型定义	以外设为单位，使用结构体类型定义每个外设所有寄存器，方便寄存器映射
	寄存器映射	1，定义总线地址和外设基地址 2，使用外设结构体类型定义将外设基地址强制转换成结构体指针，完成寄存器映射
	寄存器位定义	定义外设寄存器每个功能位的位置及掩码
	外设判定	判断某个外设是否合法（即是否存在该外设）

6.4 新建寄存器版本 MDK 工程

6.4.1 新建工程前的准备工作

1，下载相关STM32Cube 官方固件包（F1/F4/F7/H7）

方式一：ST官网搜索STM32Cube

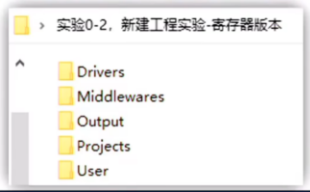
方式二：开发板A盘资料：A 盘→8，STM32 参考资料→1，STM32CubeXX固件包

2，搭建开发环境：请参考搭建开发环境相关视频

6.4.2 建寄存器版本 MDK 工程步骤

6.4.2.1 新建工程文件夹

在桌面新建一个工程根目录文件夹，并在该文件夹里新建D/M/O/P/U文件夹



文件夹名称	作用
Drivers	存放与硬件相关的驱动层文件
Middlewares	存放正点原子提供的中间层组件文件和第三方中间层文件
Output	存放工程编译输出文件
Projects	存放 MDK 工程文件
User	存放HAL库用户配置文件、main.c、中断处理文件，以及分散加载文件

6.4.2.1.1 拷贝/新建工程相关文件

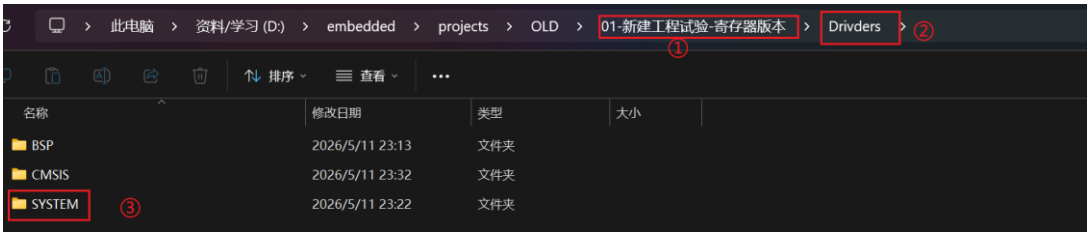
Drivers文件夹

文件夹名称	作用
BSP	存放开发板板级支持包驱动代码，如各种外设驱动
CMSIS	存放 CMSIS 底层代码，如启动文件（.s 文件）等
SYSTEM	存放正点原子系统级核心驱动代码，如 sys.c、 delay.c 和 usart.c 等

BSP：新建

CMSIS，SYSTEM：拷贝

SYSTEM 去已有的寄存器项目里拷贝即可：



CMSIS 需要去 Cube 固件文件夹里面复制,文件路径:STM32Cube_FW_F4_V1.24.1\STM32Cube_FW_F4_V1.24.1\Drivers\CMSIS

1， Include：

mpu_armv7.h ， cmsis_armcc.h ， cmsis_armclang.h ， cmsis_compiler.h ， cmsis_version.h ， core_cm3.h

2， Device→ST→STM32F4xx→Source→Templates→arm：

startup_stm32f407xx.s

Middlewares文件夹

该文件夹用于存放正点原子和其他第三方提供的中间层代码（组件/Lib 等），

如： USMART、MALLOC、 TEXT、 FATFS、 USB、 LWIP、各种 OS、各种 GUI 等

Output文件夹

该文件夹用于存放编译器编译工程输出的中间文件，比如： .hex、 .bin、 .o 文件等

Projects文件夹

该文件夹用于存放编译器（MDK、 IAR 等）工程文件，我们主要用 MDK，为了方便区分，我们在该文件夹下新建： MDK-ARM 文件夹，用于存放 MDK 的工程文件

User文件夹

该文件夹用于存放用户编写的代码，如： main.c 等

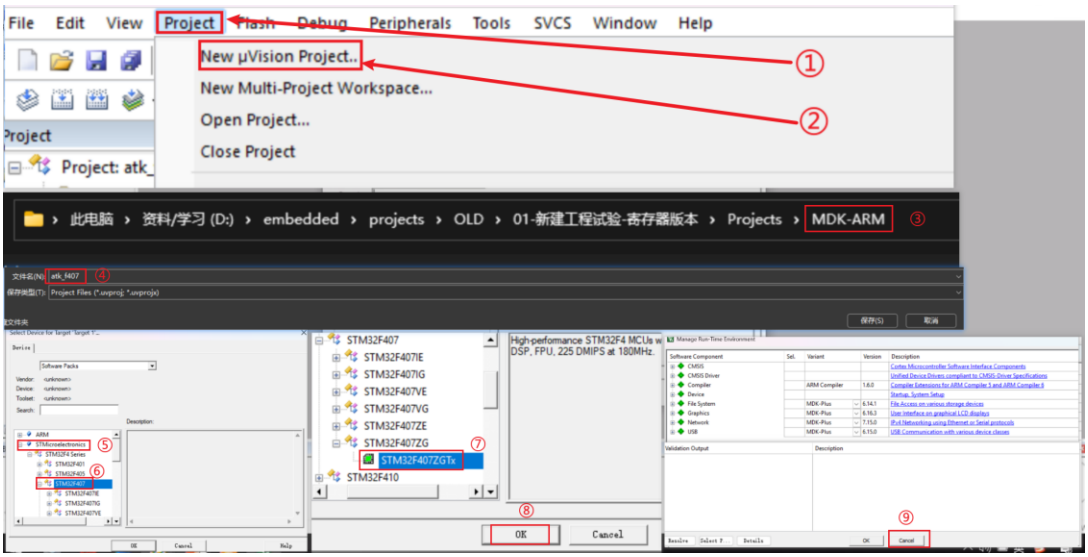
6.4.2.2 新建一个工程框架

新建一个工程框架步骤：

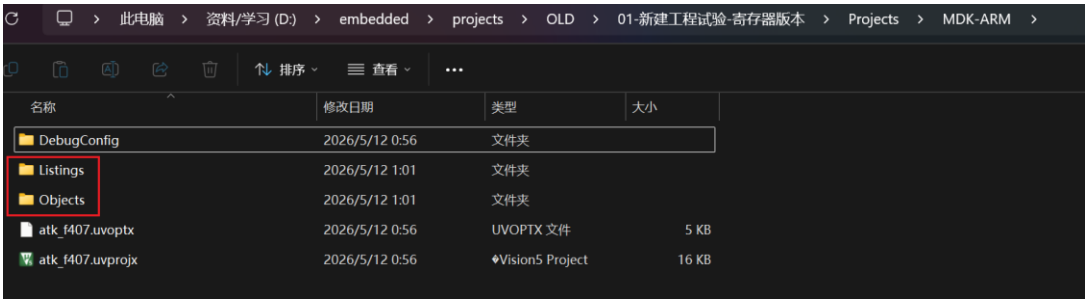
1，新建工程
2，保存工程
3，选择主控型号
4，删除文件夹

文件夹名称	作用
DebugConfig	用于存放调试设置信息文件（.dbgconf），不可删除!
Listings	用于存放编译过程产生的链接列表等文件
Objects	用于存放编译过程产生的调试信息、 .hex、预览、 .lib 文件等

具体步骤：



Listings，Objects 这两个文件夹需要删除：



6.4.2.3 添加文件

添加文件步骤：

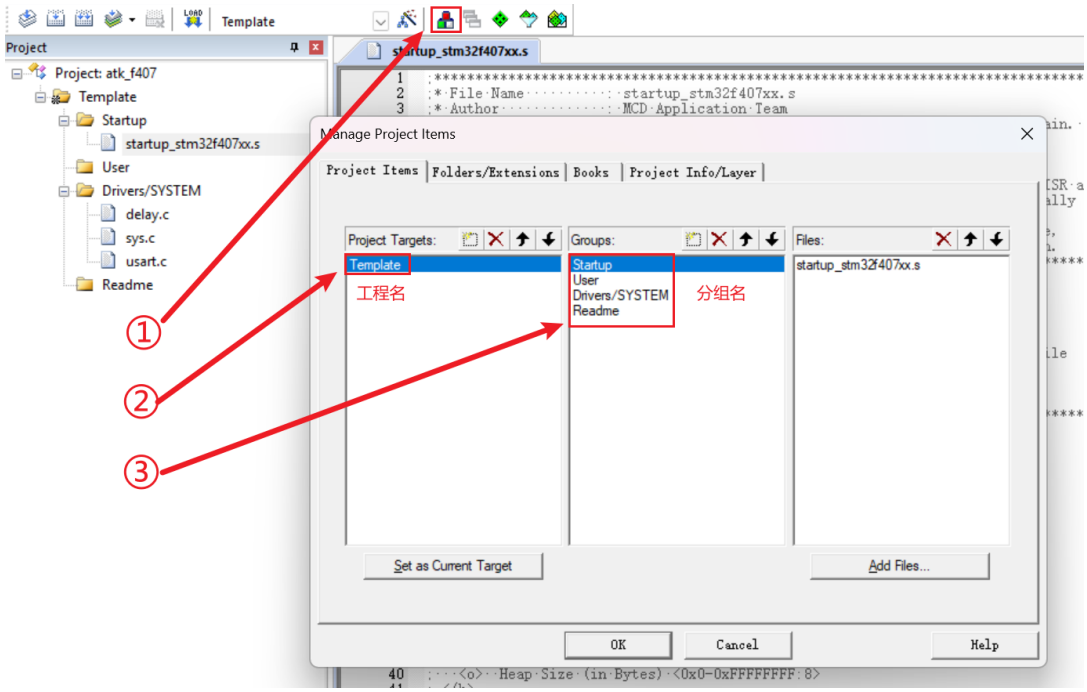
1，设置工程名和分组名

2，添加启动文件

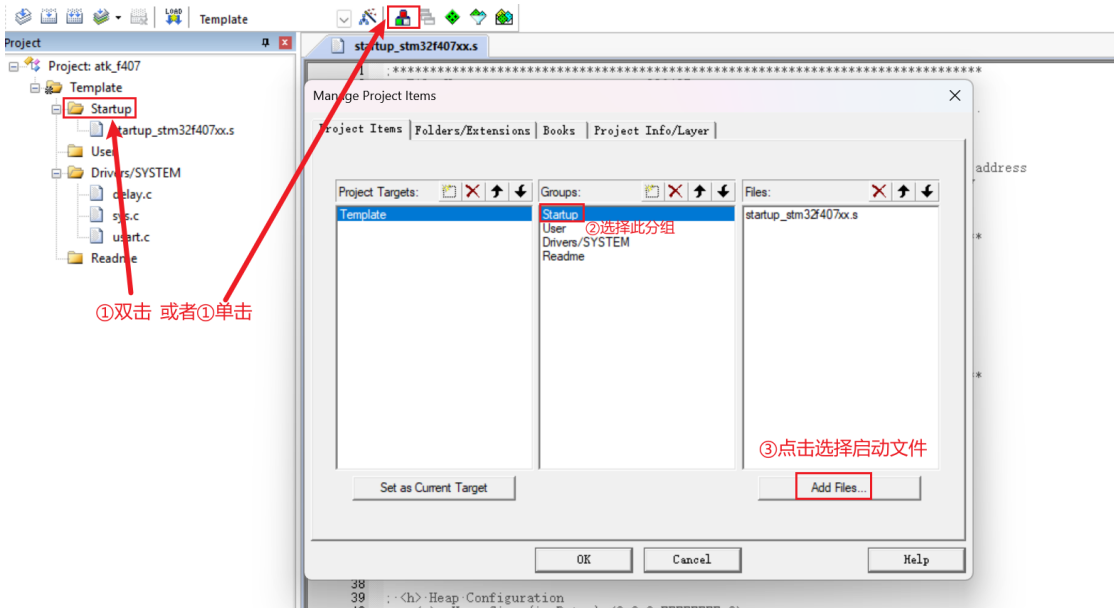
3，添加SYSTEM源码

工程名：Template
分组名：Startup，User，Drivers/SYSTEM，Readme

1，工程名以及分组名操作步骤：

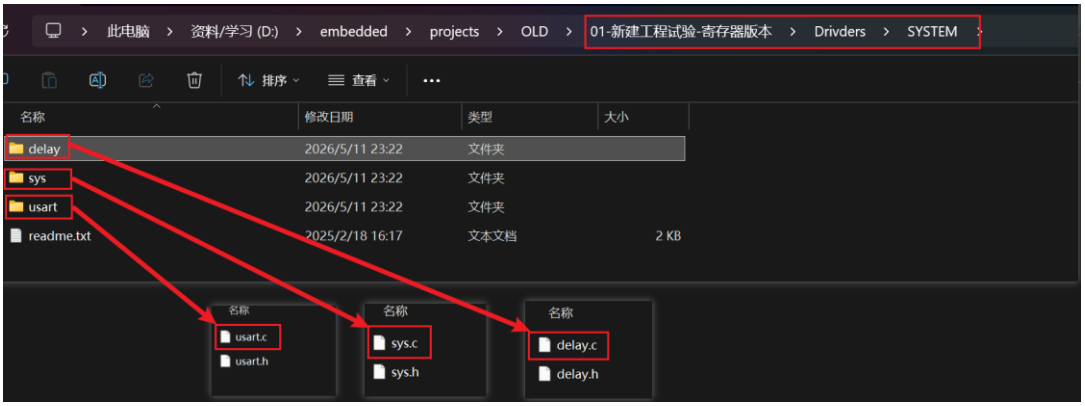


2，添加启动文件：
具体操作：



启动文件在以下文件中：

3，添加 SYSTEM 源码：



源码有两个点需要修改：

1，我们用不到编译器的内存管理函数，为节省内存，将 Heap_Size 改成 0，源码如下：

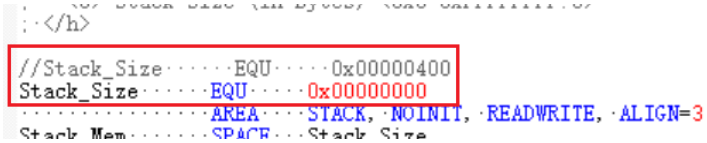
```
未用到编译器自带的内存管理(malloc,free 等)，设置 Heap_Szie 为 0
Heap_Size       EQU     0x00000000
```

2，寄存器代码不需要调用 SystemInit 函数，因此修改 Reset_Handler 函数，去掉 SystemInit 调用，源码如下：

```
Reset_Handler   PROC
                EXPORT  Reset_Handler           [WEAK]
                IMPORT  __main

                ;寄存器版本代码，因为没有用到 SystemInit 函数，所以注释掉以下代码为防止报错！
                ;HAL 库版本代码，建议加上这里（提供 SystemInit 函数），以初始化 stm32 时钟等。
                ;IMPORT  SystemInit
                ;LDR     R0, =SystemInit
                ;BLX     R0

                LDR     R0, =__main
                BX      R0
                ENDP
```

如图：

1，修改内存管理



2，修改 Reset_Handler 函数，去掉 SystemInit：

```
; Reset handler

Reset_Handler    PROC
                 EXPORT  Reset_Handler             [WEAK]
                 IMPORT  SystemInit
                 IMPORT  __main

                 LDR     R0, =SystemInit
                 BLX     R0
                 LDR     R0, =__main
                 BX      R0
                 ENDP
```

修改前

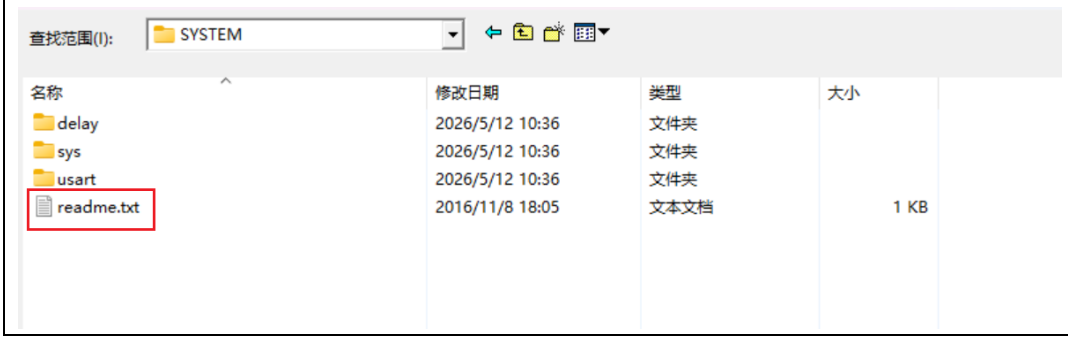
```
;2，寄存器代码不需要调用 SystemInit 函数，因此修改 Reset_Handler 函数，去掉 SystemInit
Reset_Handler    PROC
                 EXPORT  Reset_Handler             [WEAK]
                 IMPORT  __main
                 ;寄存器版本代码，因为没有用到SystemInit函数，所以注释掉以下代码为防止报错!
                 ;HAL库版本代码，建议加上这里(提供SystemInit函数)，以初始化stm32时钟等。
                 ;LDR     R0, =SystemInit
                 ;BLX     R0
                 LDR     R0, =__main
                 BX      R0
                 ENDP

; Dummy Exception Handlers (infinite loops which can be modified)
```

修改后

修改的完整代码:

```
;2，寄存器代码不需要调用 SystemInit 函数，因此修改 Reset_Handler 函数，去掉 SystemInit
Reset_Handler    PROC
                 EXPORT  Reset_Handler             [WEAK]
                 IMPORT  __main
                 ;寄存器版本代码，因为没有用到 SystemInit 函数，所以注释掉以下代码为防止报错!
                 ;HAL 库版本代码，建议加上这里(提供 SystemInit 函数)，以初始化 stm32 时钟等。
                 ;LDR     R0, =SystemInit
                 ;BLX     R0
                 LDR     R0, =__main
                 BX      R0
                 ENDP
```

4， 添加 Readme 文件



6.4.2.4 魔术棒设置

1，Target 选项卡	设置使用的编译器版本、外部晶振频率
2，Output 选项卡	设置Objects输出文件夹、生成.hex文件、输出浏览信息
3，Listing 选项卡	设置Listing输出文件夹
4，C/C++ 选项卡	设置全局宏定义、优化等级、C99 Mode、设置头文件包含路径
5，Debug 选项卡	设置使用的仿真器类型、选择的下载接口等
6，Utilities 选项卡	设置下载功能、下载算法、下载算法起始地址和大小等
7，Linker 选项卡(可选)	添加分散加载文件（北极星/MiniPro H750）

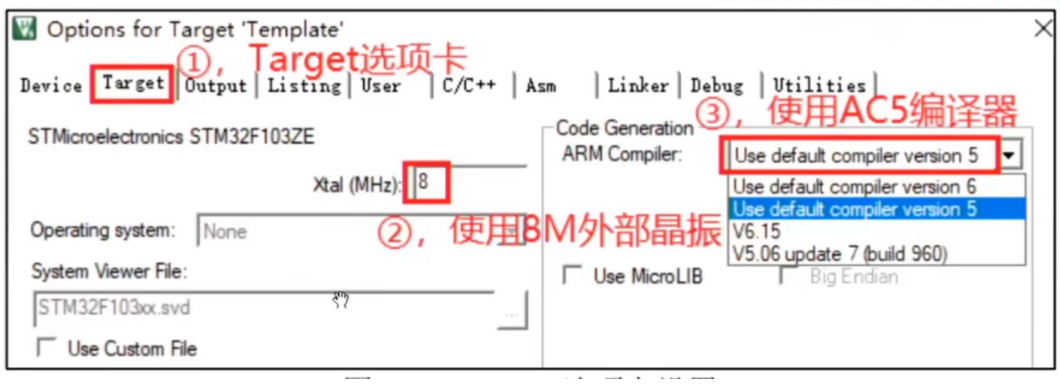
AC5和AC6编译器对比

对比项	AC5	AC6	说明
中文支持	较好	较差	AC6 对中文支持极差， goto definition 无法使用，误报等
代码兼容性	较好	较差	AC6 对某些代码优化可能导致运行异常，需慢慢调试
编译速度	较慢	较快	AC6 编译速度比 AC5 快
语法检查	一般	严格	AC6 语法检查非常严格，代码严谨性较好

选择编译器的方法：

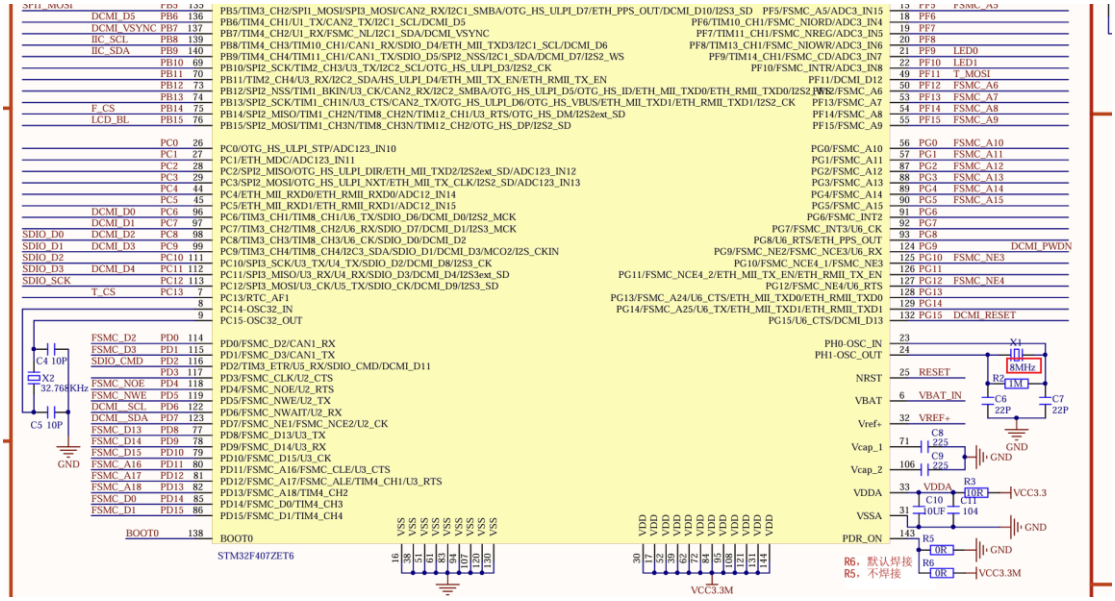
选项卡	AC5	AC6	说明
Target	选择AC5	选择AC6	选择相应的编译器
C/C++	无需设置	Misc Controls 设置： -Wno-invalid-source-encoding	AC6 需设置编译选项以关闭对汉字的错误警告， AC5 则不要

1， Targetf 选项卡：



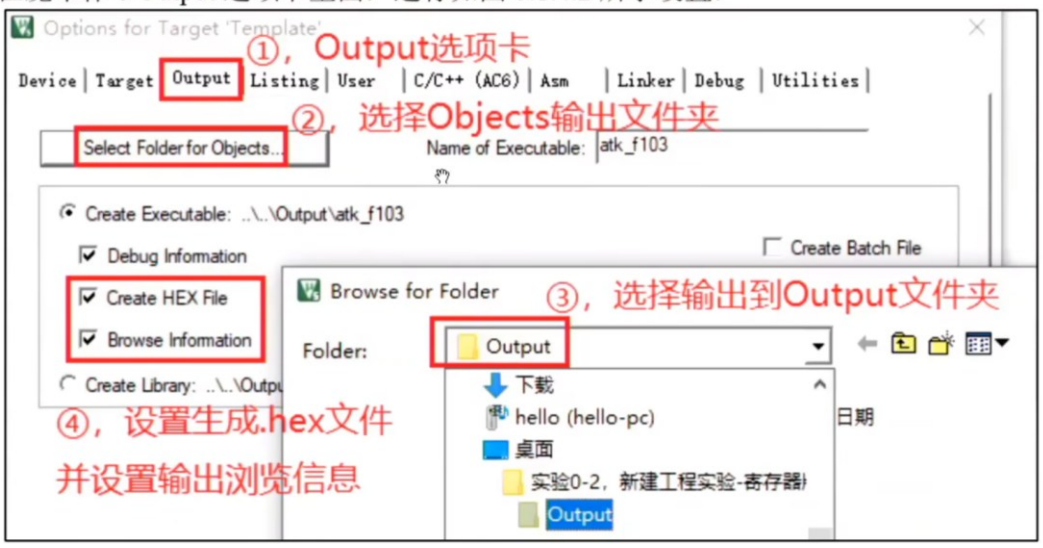
Xtal(MHz)：需要查看原理图得知填入的值：

文件：STM32F407_CORE_BOARD_V1.2_SCH.pdf



2，Output 选项卡：

在魔术棒→Output 选项卡里面，进行如图 6.1.4.2 所示设置：



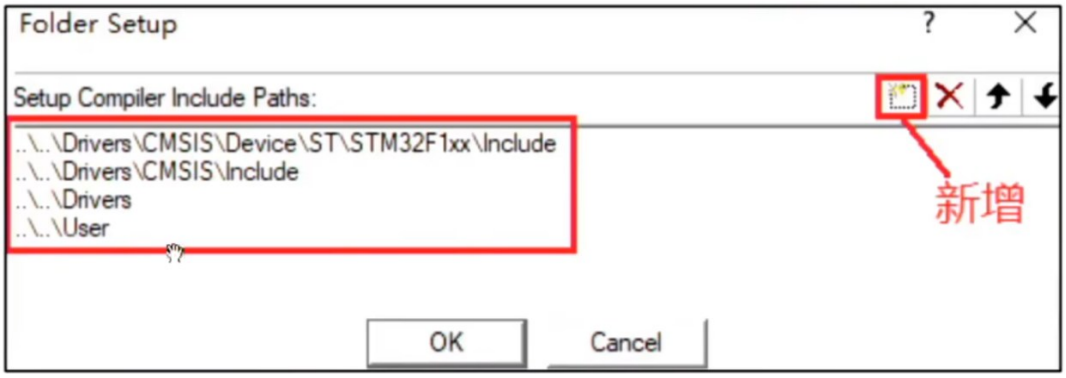
3，Listing 选项卡：

只需改一点，与上一步中的改目录一样操作，输出目录改为 Output 即可

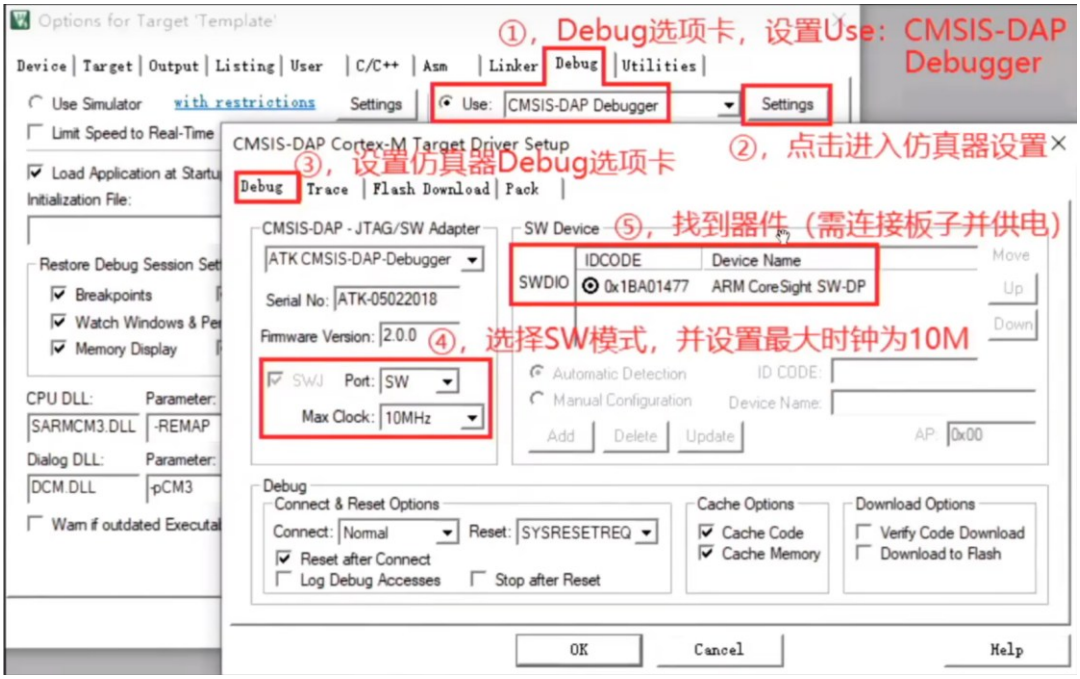
4，C/C++选项卡：

在魔术棒→C/C++选项卡里面，进行如图 6.1.4.4 所示设置：

续第⑤步：



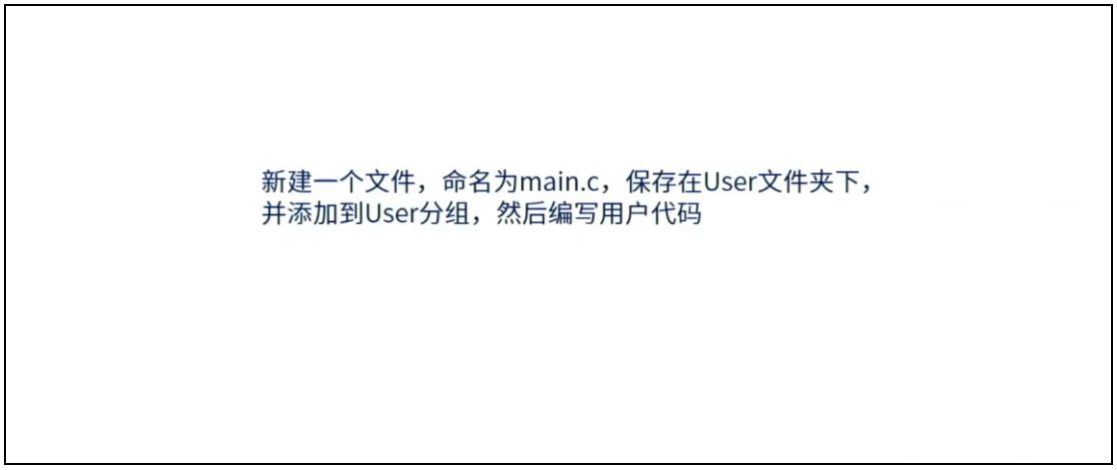
5， Debuge 选项卡：



6， Utilities 选项卡：

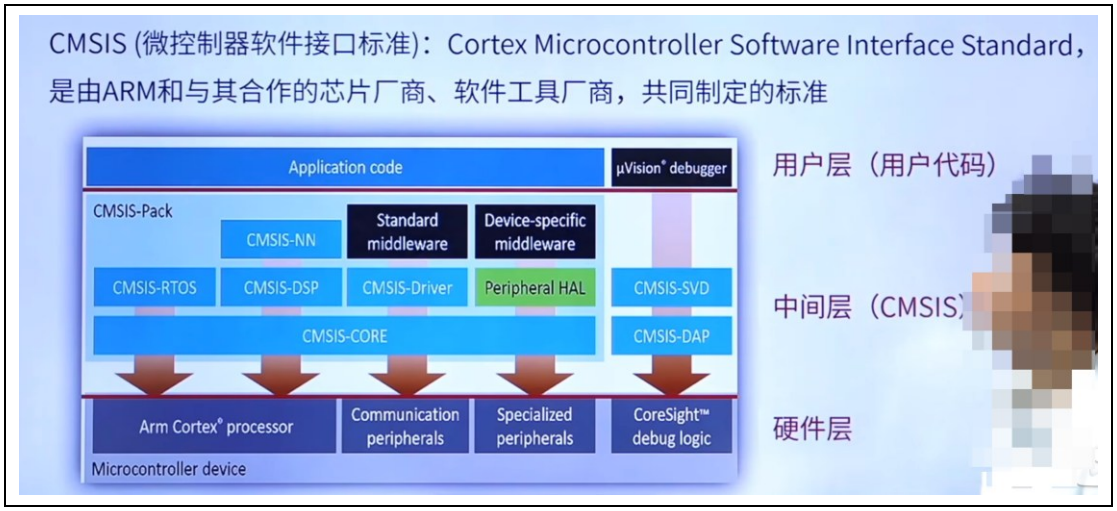
7，Linker 选项卡（可选）：

6.4.2.5 添加 main.c，并编写代码



6.5 初识 HAL 库

6.5.1 CMSIS 简介



6.5.2 HAL 简介

6.5.3 STM32Cube 固件包的浅析

6.5.3.1 如何获取 STM32Cube 固件包浅析

1，下载相关STM32Cube 官方固件包（F1/F4/F7/H7）

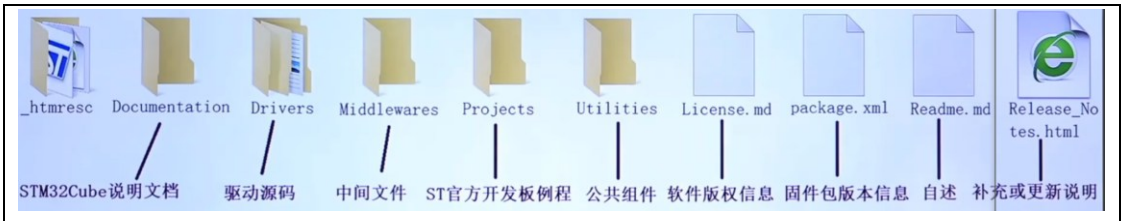
方式一：ST官网搜索STM32Cube

方式二：开发板A盘资料：A 盘→8，STM32 参考资料→1，STM32CubeXX固件包

2，搭建开发环境：请参考搭建开发环境相关视频

https://www.st.com.cn/zh 搜索自己开发板的型号：比如：STM32CubeF4
STM32CubeF4 | 产品 - 意法半导体

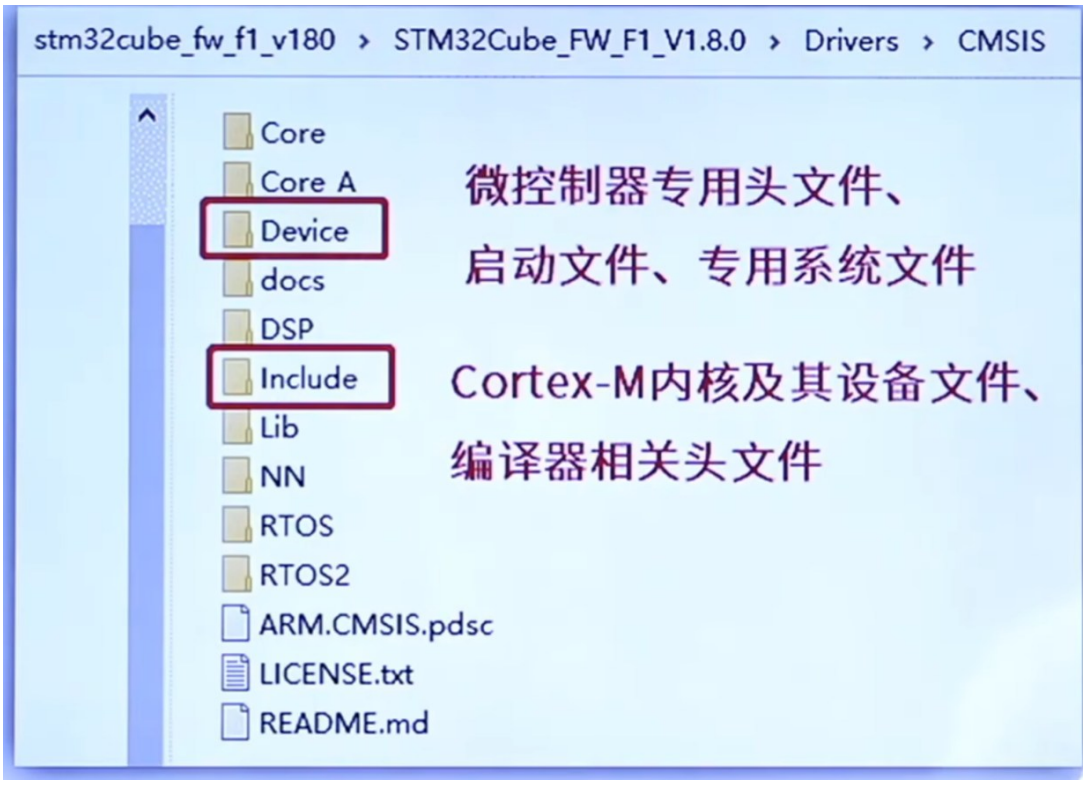
6.5.3.2 STM32Cube 固件包文件夹简介



1， Drivers

文件夹名称		作用	
Drivers	BSP	板级支持包，用于适配ST官方的开发板（可参考）	
	CMSIS	符合CMSIS的组件，包括：DSP库、Cortex-M内核及其设备文件、微控制器专用头文件、启动文件、专用系统文件等	
	STM32F1xx_HAL_Driver	HAL库外设驱动源码，包括F1系列HAL库源文件和头文件	
Middlewares	ST	STemWin	STemWin工具包
		STM32_USB_Device_Library	USB从机设备支持包
		STM32_USB_Host_Library	USB主机设备支持包
	Third_Party	FatFs	FAT文件系统支持包
		FreeRTOS	FreeRTOS实时系统支持包
		LwIP	LwIP网络通信协议支持包

6.5.3.3 CMSIS 文件夹关键文件



CMSIS 标准规定软件包目录：

文件/目录		描述
LICENSE.txt		Apache 2.0 授权的许可文件
Device		基于 Arm Cortex-M 处理器设备的 CMSIS 参考实现
ARM.CMSIS.pdsc		描述该 CMSIS 包的文件
CMSIS 组件	Documentation	这个数据包的描述文档
	Core	CMSIS-Core（Cortex-M）相关文件的用户代码模板,在 ARM.CMSIS.pdsc 中引用
	Core_A	CMSIS-Core（Cortex-A）相关文件的用户代码模板，在 ARM.CMSIS.pdsc 中引用
	DAP	CMSIS-DAP 调试访问端口源代码和参考实现
	Driver	CMSIS 驱动程序外设接口 API 的头文件
	DSP_Lib	CMSIS-DSP 软件库源代码
	NN	CMSIS-NN 软件库源代码
	Include	CMSIS-Core(Cortex-M)和 CMSIS-DSP 需要包括的头文件等。
	Lib	包括 CMSIS 核心（Cortex-M）和 CMSIS-DSP 的文件
	Pack	CMSIS-Pack 示例，包含包含设备支持、板支持和软件组件的软件包示例。
	RTOS	CMSIS-RTOS 版本 1 以及 RTX4 参考实现
	RTOS2	CMSIS-RTOS 版本 2 以及 RTX5 参考实现
	SVD	CMSIS-SVD 样例，规定开发者、制造商、工具制造商的分工和职能
	Utilities	PACK.xsd(CMSIS-Pack 架构文件),PackChk.exe(检查软件包的工具),CMSIS-SVD.xsd(MSIS-SVD 架构文件),SVDConv.exe(SVD 文件的转换工具)

Device 和 Include 文件夹的关键文件介绍:

以下图片中 1 为 Device, 2 为 Include

1

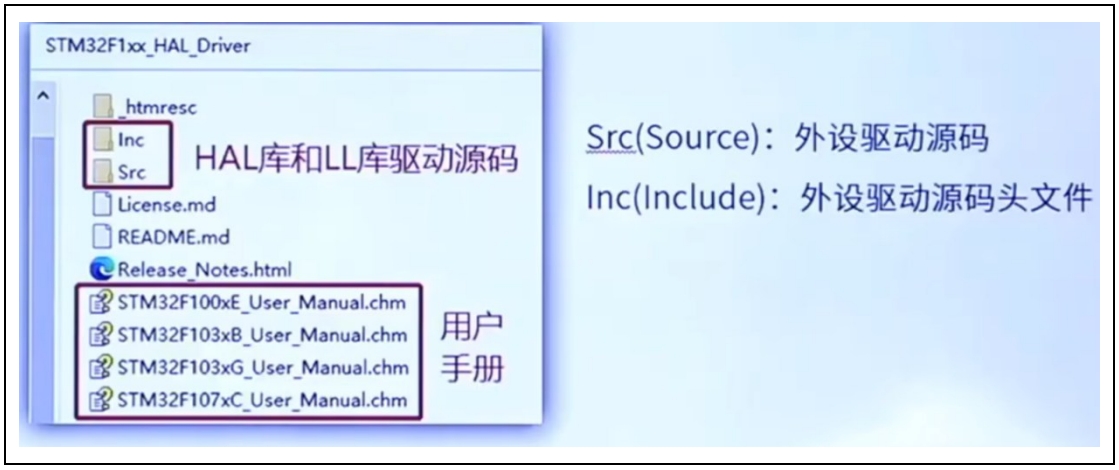
文件名称	描述
stm32f1xx.h	是所有F1系列的顶层头文件，通过条件编译来包含某个芯片的头文件，定义通用的枚举类型，定义通用的宏定义
stm32f103xe.h	包含：中断编号定义、外设寄存器结构体类型定义、寄存器映射、寄存器位定义、外设判定
system_stm32f1xx.c system_stm32f1xx.h	定义了系统初始化函数 SystemInit和系统时钟更新函数 SystemCoreClockUpdate
startup_stm32f103xe.s	大容量F103系列芯片的启动文件

2

cmsis_armcc.h、cmsis_armclang.h、cmsis_compiler.h、cmsis_version.h、core_cm3.h 、 mpu_armv7.h

6.6 HAL 框架结构

6.6.1 HAL 文件夹结构



6.6.2 HAL 库文件介绍

文件	描述
sm32f1xx_hal.c stm32f1xx_hal.h	HAL库初始化、系统滴答、HAL库延时等相关函数
stm32f1xx_hal_conf.h	HAL库的用户配置文件，用于裁剪HAL库、配置晶振参数等
stm32hxx_hal_def.h	包含HAL库通用的枚举类型数据和宏定义
stm32f1xx_hal_cortex.h stm32f1xx_hal_cortex.c	内核通用函数定义和声明，如NVIC、MPU、系统软复位、Systick等，其实主要是对core_cm3.h 文件的相关函数再次封装。
stm32f1xx_hal_ppp.c stm32f1xx_hal_ppp.h	某任意外设驱动源码，PPP表示任意外设
stm32f1xx_hal_ppp_ex.c stm32f1xx_hal_ppp_ex.h	主要是存放外设的扩展（特殊）功能的驱动源码，PPP表示任意外设
stm32f1xx_ll_ppp.c stm32f1xx_ll_ppp.h	LL库驱动源码，在部分STM32F1xx_hal_ppp.c 或stm32f1xx_hal_ppp_ex.c 中会被调用

6.6.3 HAL 库 API 函数和变量命名规则

文件名称	stm32f1xx_hal_ppp .(c/h)	stm32f1xx_hal_ppp_ex .(c/h)
函数名	HAL_PPP_Function	HAL_PPPEx_Function
外设句柄	PPP_HandleTypeDef	无
外设工作参数初始化结构体	PPP_InitTypeDef	PPP_InitTypeDef
配置参数结构体	PPP_ YyyyConfTypeDef	PPP_ YyyyConfTypeDef
初始化/反初始化函数:HAL_PPP_Init(), HAL_PPP_DeInit()		
外设读写函数:HAL_PPP_Read(),HAL_PPP_Write(),HAL_PPP_Transmit(), HAL_PPP_Receive()		
控制函数:HAL_PPP_Set (),HAL_PPP_Get ()		
状态和错误:HAL_PPP_GetState (), HAL_PPP_GetError ()		

6.6.4 HAL 库对寄存器位操作的相关宏定义

宏定义结构	用途
__HAL_PPP_ENABLE_IT(__HANDLE__, __INTERRUPT__)	使能外设中断
__HAL_PPP_DISABLE_IT(__HANDLE__,__INTERRUPT__)	禁用外设中断
__HAL_PPP_GET_IT (__HANDLE__, __ INTERRUPT __)	获取外设某一中断源
__HAL_PPP_CLEAR_IT (__HANDLE__, __ INTERRUPT __)	清除外设中断
__HAL_PPP_GET_FLAG (__HANDLE__, __FLAG__)	获取外设的状态标记
__HAL_PPP_CLEAR_FLAG (__HANDLE__, __FLAG__)	清除外设的状态标记
__HAL_PPP_ENABLE(__HANDLE__)	使能某一外设
__HAL_PPP_DISABLE(__HANDLE__)	禁用某一外设
__HAL_PPP_XXXX (__HANDLE__, __PARAM__)	针对外设的特殊操作
__HAL_PPP_GET_ IT_SOURCE (__HANDLE__, __INTERRUPT __)	检查外设的中断源

6.6.5 HAL 库回调函数

回调函数	举例
HAL_PPP_MspInit() / _MspDeInit ()	举例: HAL_USART_MspInit() 会被HAL_PPP_Init() 函数调用，该函数主要用于存放外设使用到的GPIO、CLOCK、NVIC、DMA等初始化代码
HAL_PPP_ProcessCpltCallback	举例: HAL_USART_TxCpltCallback 由外设中断或DMA中断调用，HAL库中断公共处理函数已经实现对中断标记位读取、判断和清除操作，用户只需要专注于中断逻辑功能的实现即可
HAL_PPP_ErrorCallback	举例: HAL_USART_ErrorCallback 外设或DMA中断中发生的错误，用于发生错误处理

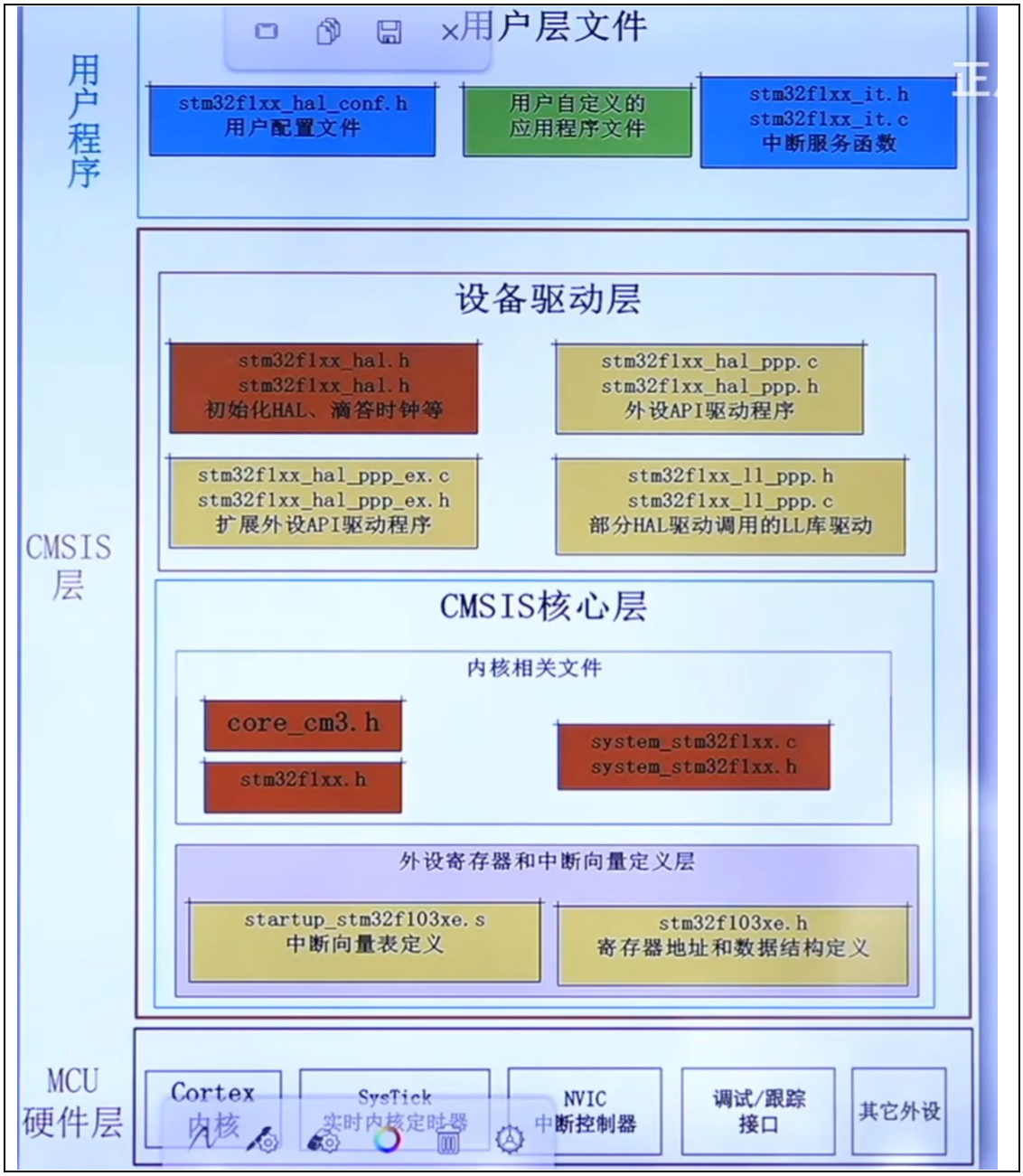
此类函数通常被_weak修饰（弱函数），允许用户重新定义该函数

6.7 使用 HAL 库

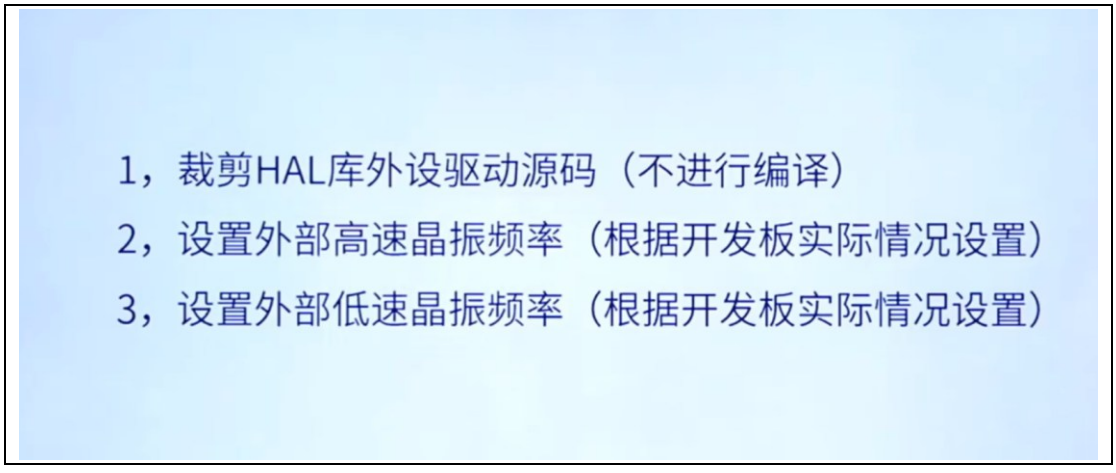
6.7.1 基于 CMSIS 应用程序文件描述

类别	文件名	描述	是否必须
用户程序文件	main.c	存放 main 函数，不一定要在这个文件	否
	main.h	包含头文件、声明等作用，已删除	否
	stm32f1xx_it.c	用户中断服务函数存放文件，不一定放到这个文件，可删除	否
	stm32f1xx_it.h		否
	stm32f1xx_hal_conf.h	用户配置文件	是
	stm32f1xx_hal_msp.c	回调函数存放文件，已删除	否
设备驱动层	sm32f1xx_hal.c	HAL 库的初始化、系统滴答，HAL 库延时函数等功能	是
	stm32f1xx_hal.h		是
	stm32hxx_hal_def.h	通用 HAL 库资源定义	是
	stm32f1xx_hal_ppp.c	外设的操作 API 函数文件	是
	stm32f1xx_hal_ppp.h		是
	stm32f1xx_hal_ppp_ex.c	拓展外设特性的 API 函数文件	是
	stm32f1xx_hal_ppp_ex.h		是
	stm32f1xx_II_ppp.c	LL 库文件，在一些复杂外设中实现底层功能	是
	stm32f1xx_II_ppp..h		是
CMSIS 核心层	stm32f1xx.h	STM32F1 系列的顶层头文件	是
	stm32f103xe.h	STM32F103系列片上外设头文件	是
	system_stm32f1xx.c	主要存放系统初始化函数 SystemInit	是
	system_stm32f1xx.h		是
	startup_stm32f1xx.s	启动文件，运行到 main 函数前的准备	是
	core_cm3.h	内核寄存器定义，如 Systick、SCB 等	是
	cmsis_armcc.h	编译器相关头文件，一般都不需要去了解，只需加入工程即可	是
	cmsis_armclang.h		
	cmsis_compiler.h		
	cmsis_version.h		
	mpu_armv7.h		

STM32 开发文件结构分布：



6.7.2 HAL 库的用户配置文件



6.7.2.1 裁剪 HAL 库外设驱动源码(不进行编译)

配置文件有两种方式：

① 官方提供的模版源码（stm32f1xx_hal_conf.h）：

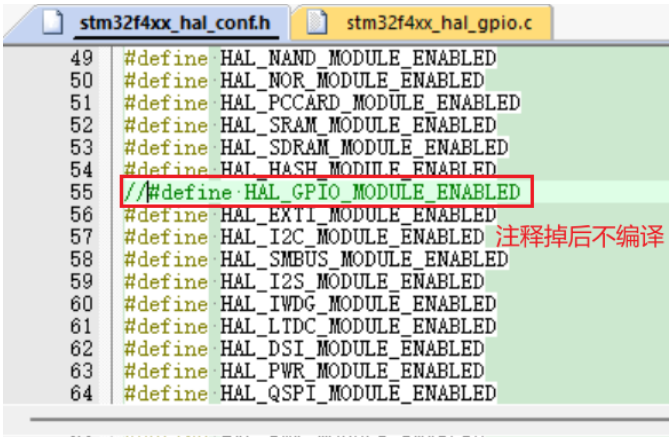
路径：STM32CubeF1-1.8.0\STM32CubeF1-1.8.0\Projects\STM3210E_EVAL\Templates\Inc

② 官方固件中提供的" stm32f1xx_hal_conf_template.h"文件:

路径: STM32CubeF1-1.8.0\STM32CubeF1-1.8.0\Drivers\STM32F1xx_HAL_Driver\Inc

将"stm32f1xx_hal_conf_template.h"重命名为"stm32f1xx_hal_conf.h"即可

裁剪，即把程序用不到的定义，函数等注释掉:



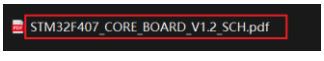
还有就是需要什么就导入什么，不要导入不必要的文件，导致编译缓慢

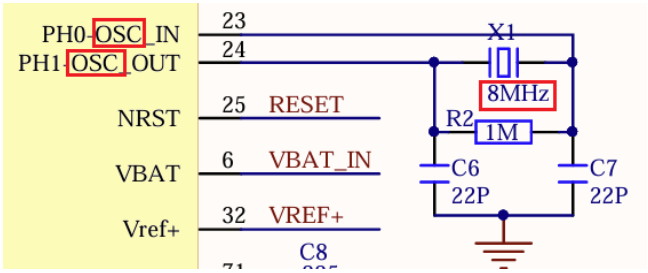
6.7.2.2 设置外部高速晶振频率(根据开发板实际情况设置)

HSE_VALUE: 外部高速晶振
LSE_VALUE: 外部低速晶振
HSI_VALUE: 内部高速晶振
LSI_VALUE: 内部低速晶振

1，外部高速晶振值的获取方式:

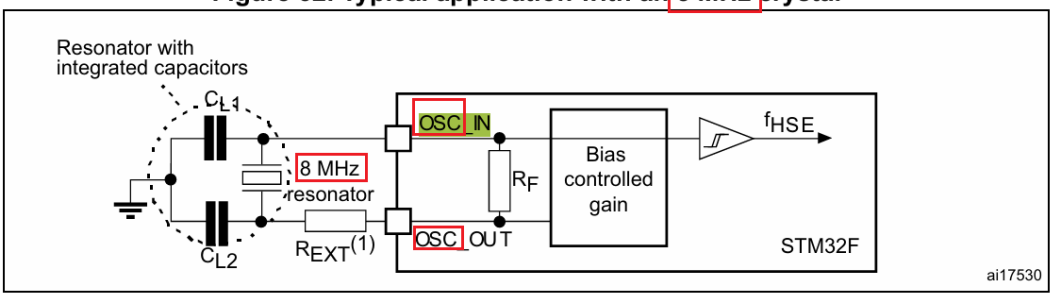
① :从原理图中获取:





② ：从参考手册中获取：stm32f407zg.pdf，部分开发版能找到

Figure 32. Typical application with an 8 MHz crystal



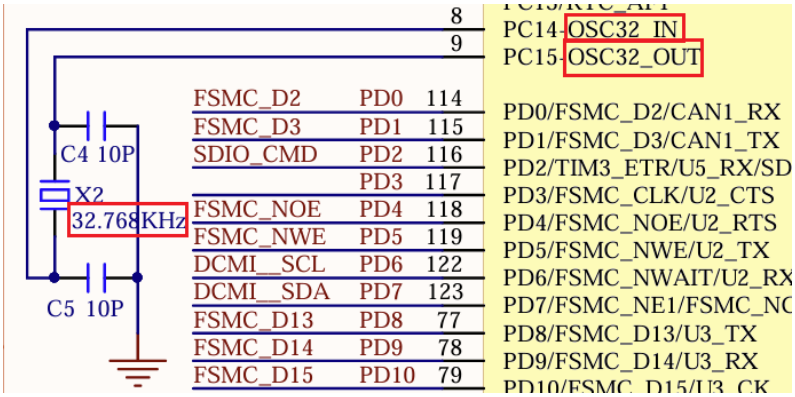
外部高速晶振值修改：

stm32f4xx_hal_conf.h　stm32f4xx_hal_gpio.c

```
79  #define HAL_HCD_MODULE_ENABLED
80  #define HAL_FMPI2C_MODULE_ENABLED
81  #define HAL_FMPSMBUS_MODULE_ENABLED
82  #define HAL_SPDIFRX_MODULE_ENABLED
83  #define HAL_DFSDM_MODULE_ENABLED
84  #define HAL_LPTIM_MODULE_ENABLED
85  #define HAL_MMC_MODULE_ENABLED
86
87  /* ############################ HSE/HSI Values adaptatio
88  /**
89    * @brief Adjust the value of External High Speed osc
90    *        This value is used by the RCC HAL module to
91    *        (when HSE is used as system clock source, d
92    */
93  #if !defined (HSE_VALUE)
94    #define HSE_VALUE ...... 8000000U /*!< Value of
95  #endif /* HSE_VALUE */                 8MHz
96
97  #if !defined (HSE_STARTUP_TIMEOUT)
98    #define HSE_STARTUP_TIMEOUT ... 100U ..... /*!< Time ou
99  #endif /* HSE_STARTUP_TIMEOUT */
100
101 /**
102   * @brief Internal High Speed oscillator (HSI) value.
103   *        This value is used by the RCC HAL module to
104   *        (when HSI is used as system clock source, d
105   */
106 #if !defined (HSI_VALUE)
```

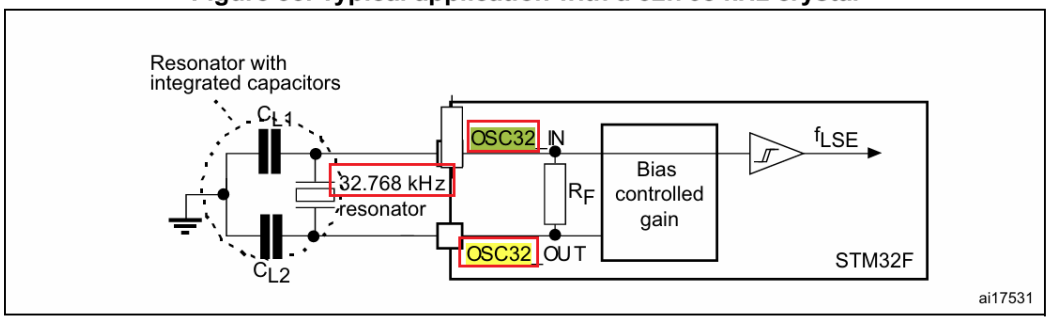
2，外部低速晶振值获取

③ 从原理图获取：STM32F407_CORE_BOARD_V1.2_SCH.pdf



④ 从参考手册中获取：stm32f407zg.pdf，部分开发版能找到

Figure 33. Typical application with a 32.768 kHz crystal



外部高速晶振值修改：

stm32f4xx_hal_conf.h | stm32f4xx_hal_gpio.c

```
112   */
113  #if !defined  (LSI_VALUE)
114    #define LSI_VALUE              40000U    /*!< LSI Typical Value in Hz */
115  #endif /* LSI_VALUE */                     /*!< Value of the Internal Low
116                                               The real value may vary de
117                                               in voltage and temperature
118  /**
119    * @brief External Low Speed oscillator (LSE) value.
120    */
121  #if !defined  (LSE_VALUE)
122    #define LSE_VALUE              32768U    /*!< Value of the External Low
123  #endif /* LSE_VALUE */                      32.768K
124
125  #if !defined  (LSE_STARTUP_TIMEOUT)
126    #define LSE_STARTUP_TIMEOUT    5000U     /*!< Time out for LSE start up,
127  #endif /* LSE_STARTUP_TIMEOUT */
128
129  /**
```

3，内部高速晶振值获取：芯片参考手册stm32f407zg.pdf，第一页

- Clock, reset and supply management
  - 2.0 to 3.6 V application supply and I/Os
  - POR, PDR, and programmable voltage detector (PVD)
  - 4-to-16 MHz crystal oscillator
  - Internal 8 MHz factory-trimmed RC
  - Internal 40 kHz RC with calibration
  - 32 kHz oscillator for RTC with calibration

内部高速晶振值修改：

stm32f4xx_hal_conf.h* | stm32f4xx_hal_gpio.c

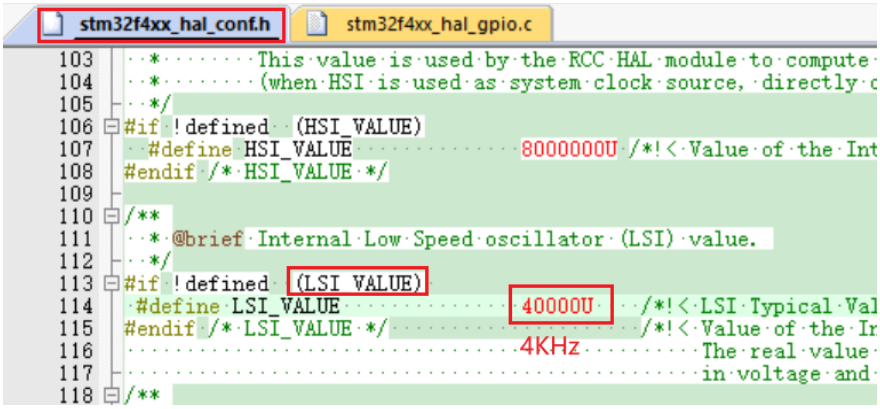
```
91     *          (when HSE is used as system clock source, directly
92     */
93   #if !defined  (HSE_VALUE)
94     #define HSE_VALUE              8000000U /*!< Value of the E
95   #endif /* HSE_VALUE */
96
97   #if !defined  (HSE_STARTUP_TIMEOUT)
98     #define HSE_STARTUP_TIMEOUT    100U     /*!< Time out for
99   #endif /* HSE_STARTUP_TIMEOUT */
100
101  /**
102    * @brief Internal High Speed oscillator (HSI) value.
103    *         This value is used by the RCC HAL module to comput
104    *         (when HSI is used as system clock source, directly
105    */
106  #if !defined  (HSI_VALUE)
107    #define HSI_VALUE              8000000U /*!< Value of the I
108  #endif /* HSI_VALUE */                      8MHz
109
110  /**
```

5， 内部低速晶振值获取：

芯片参考手册stm32f407zg.pdf，第一页

- Clock, reset and supply management
  - 2.0 to 3.6 V application supply and I/Os
  - POR, PDR, and programmable voltage detector (PVD)
  - 4-to-16 MHz crystal oscillator
  - Internal 8 MHz factory-trimmed RC
  - Internal 40 kHz RC with calibration
  - 32 kHz oscillator for RTC with calibration

内部低速晶振值修改：



6.7.3 stm32f1xx_hal.c 文件

```
HAL_StatusTypeDef HAL_Init(void)
{
    __HAL_FLASH_PREFETCH_BUFFER_ENABLE();          /* 使能FLASH预取缓冲 */

    HAL_NVIC_SetPriorityGrouping(NVIC_PRIORITYGROUP_2); /* 配置中断优先级分组 */

    /* 使用滴答定时器作为时钟基准，配置 1ms 滴答（重置后默认的时钟源为 HSI） */
    HAL_InitTick(TICK_INT_PRIORITY);

    HAL_MspInit();      /* 初始化其它底层硬件（如果必要） */

    return HAL_OK;      /* 返回函数状态 */
}
```

6.7.4 HAL 库使用的注意事项

1，使用HAL库出现问题，还是得通过参考手册检查是否硬件操作是否有问题

2，尽量不通过修改库源码实现功能，这样不方便库更新

3，HAL库可能会存在错误，要有质疑精神

4，有些HAL库API函数执行效率偏低，我们可能会直接通过操作寄存器的方式代替

6.8 新建 HAL 版本 MDK 工程

6.8.1 新建工程前的准备工作

1，下载相关STM32Cube 官方固件包（F1/F4/F7/H7）

方式一：ST官网搜索STM32Cube

方式二：开发板A盘资料：A 盘→8，STM32 参考资料→1，STM32CubeXX固件包

2，搭建开发环境：请参考搭建开发环境相关视频

6.8.2 新建 HAL 库版本 MDK 工程步骤

新建HAL库版本MDK工程简要步骤：

新建工程文件夹	D/M/O/P/U文件夹，5个文件夹
新建工程框架	工程命名，选芯片
添加文件	建分组，添加.s、.c
魔术棒设置	T/O/L/C/D/U，6个选项卡
添加main.c	编写自己的代码

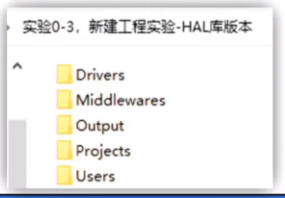
新建工程过程可参考： A 盘→ 4，程序源码→ 2，标准例程-HAL库版本

→实验0 基础入门实验→实验0-3，新建工程实验-HAL库版本

6.8.2.1 新建工程文件夹

2.1，新建工程文件夹

在桌面新建一个工程根目录文件夹，并在该文件夹里新建D/M/O/P/U文件夹



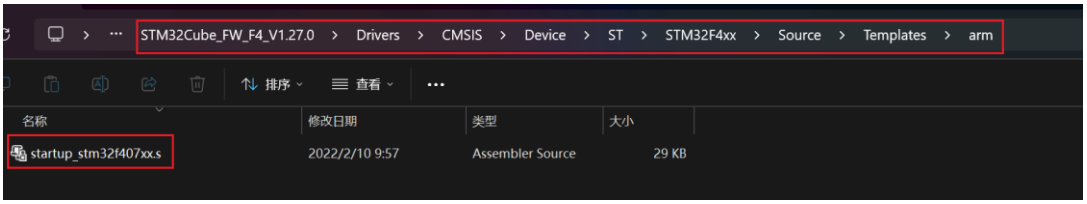
文件夹名称	作用
Drivers	存放与硬件相关的驱动层文件
Middlewares	存放正点原子提供的中间层组件文件和第三方中间层文件
Output	存放工程编译输出文件
Projects	存放 MDK 工程文件
User	存放HAL库用户配置文件、main.c、中断处理文件，以及分散加载文件

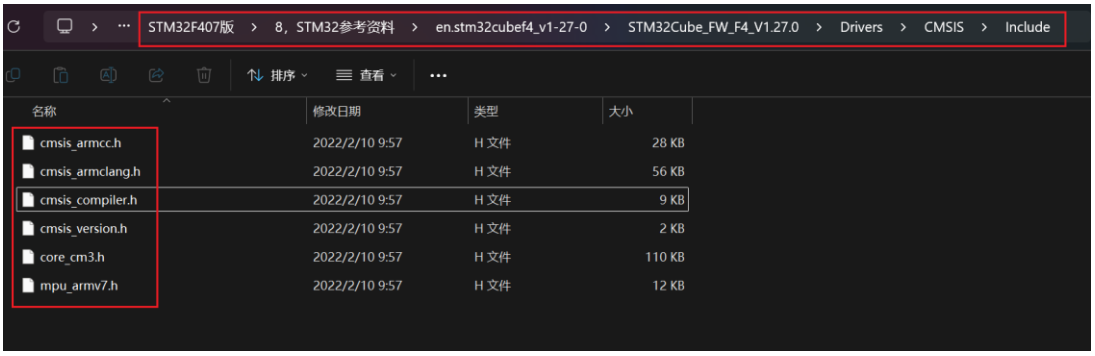
拷贝/新建工程相关文件

Drivers文件夹

文件夹名称	作用
BSP	存放开发板板级支持包驱动代码，如各种外设驱动
CMSIS	存放 CMSIS 底层代码，如启动文件（.s 文件）等
SYSTEM	存放正点原子系统级核心驱动代码，如 sys.c、 delay.c 和 usart.c 等
STM32F1xx_HAL_Driver	存放 ST 提供的 F1系列HAL库驱动源码

BSP：手动新建，其他需要拷贝
CMSIS 文件拷贝来源：





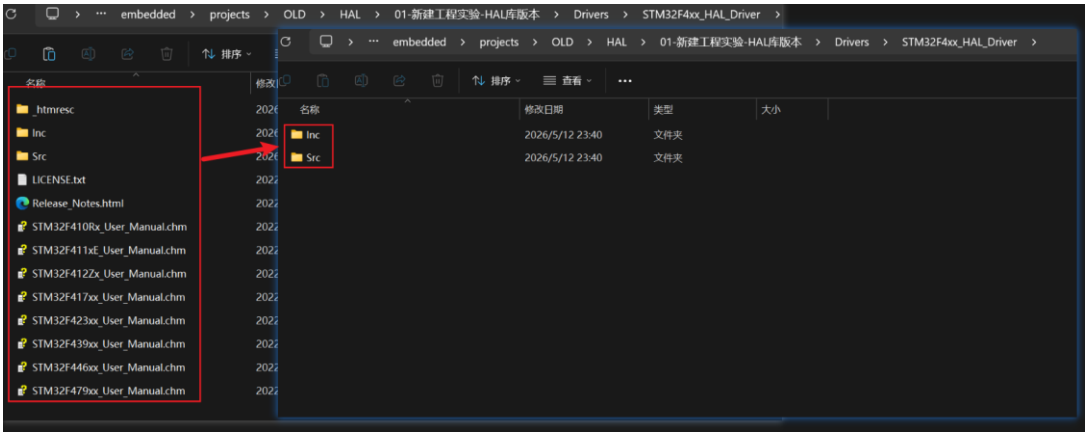
此处“core_cm3.h”文件应该改为“core_cm4.h”文件,根据具体开发板而定

SYSTEM 文件来源：
已有且正确的 HAL 项目中直接拷贝即可：

STM32F4xx_HAL_Driver 文件来源：

官方固件文件：

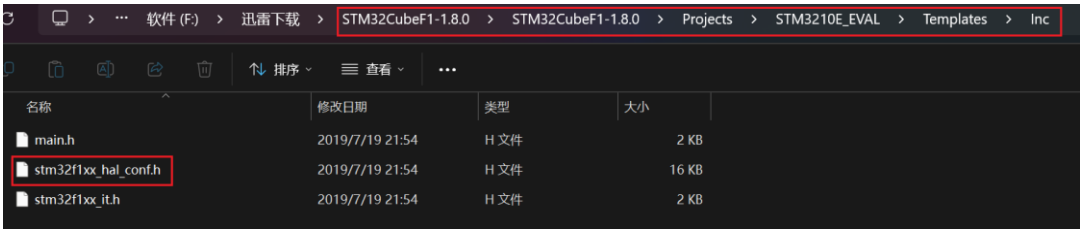
然后删掉多余的文件：



配置文件有两种方式（坑:注意两种方式的文件内容是否一致，以第二种方式好点）：

① 官方提供的模版源码（stm32f1xx_hal_conf.h）：

路径：STM32CubeF1-1.8.0\STM32CubeF1-1.8.0\Projects\STM3210E_EVAL\Templates\Inc



② 官方固件中提供的" stm32f1xx_hal_conf_template.h"文件：

路径： STM32CubeF1-1.8.0\STM32CubeF1-1.8.0\Drivers\STM32F1xx_HAL_Driver\Inc

将"stm32f1xx_hal_conf_template.h"重命名为"stm32f1xx_hal_conf.h"即可

USER 中固件包文件来源：

找一个官方案例或者已有项目中去拷贝：

6.8.2.2 新建一个工程框架

新建一个工程框架步骤：

1，新建工程

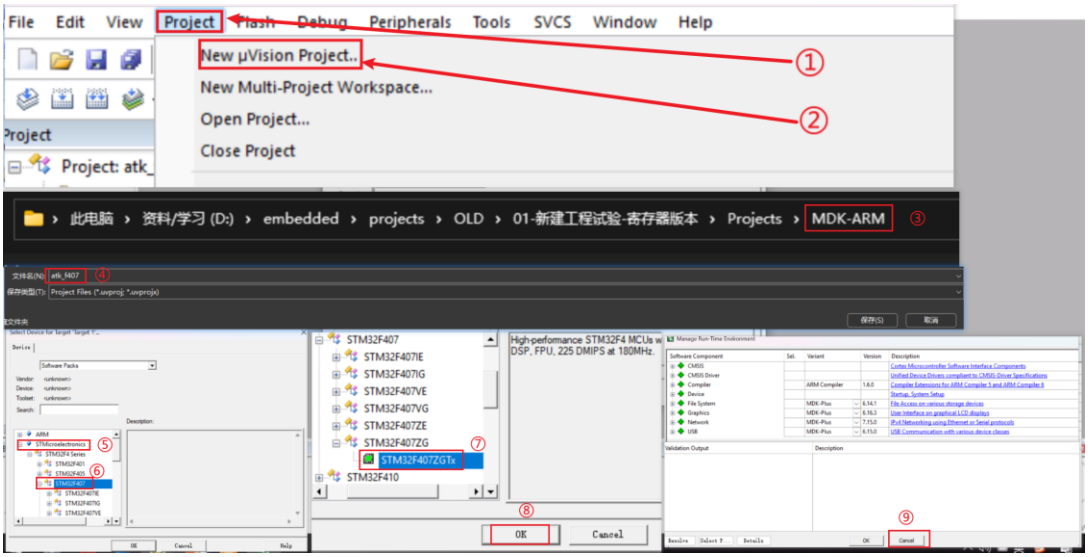
2，保存工程

3，选择主控型号

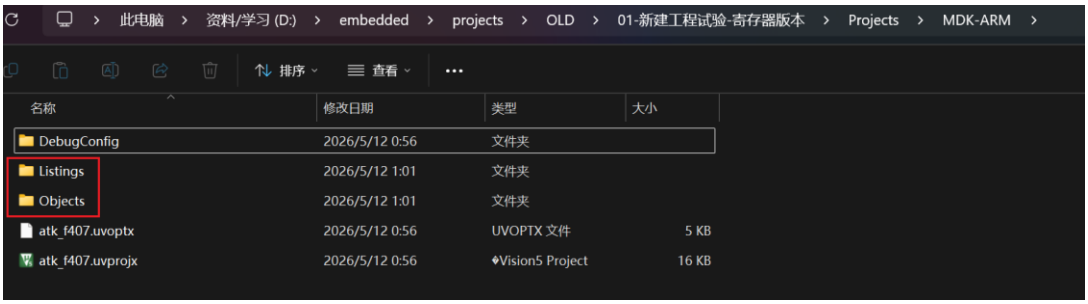
4，删除文件夹

文件夹名称	作用
DebugConfig	用于存放调试设置信息文件（.dbgconf），不可删除!
Listings	用于存放编译过程产生的链接列表等文件
Objects	用于存放编译过程产生的调试信息、.hex、预览、.lib 文件等

具体步骤：



Listings，Objects 这两个文件夹需要删除：



6.8.2.3 添加文件

添加文件步骤：

1，设置工程名和分组名

2，添加启动文件

3，添加User源码

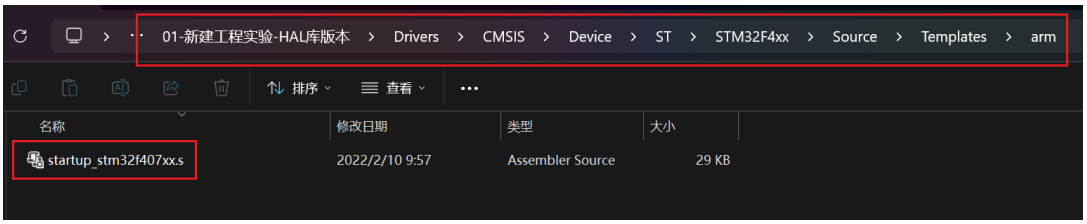
4，添加SYSTEM源码

5，添加STM32F1xx_HAL_Driver 源码

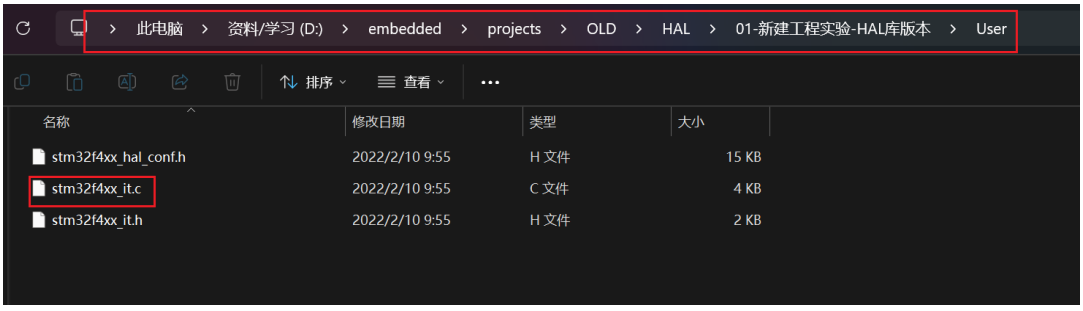
工程名：Template

分组名：Startup，User，Drivers/SYSTEM， Drivers/STM32F1xx_HAL_Driver，Readme

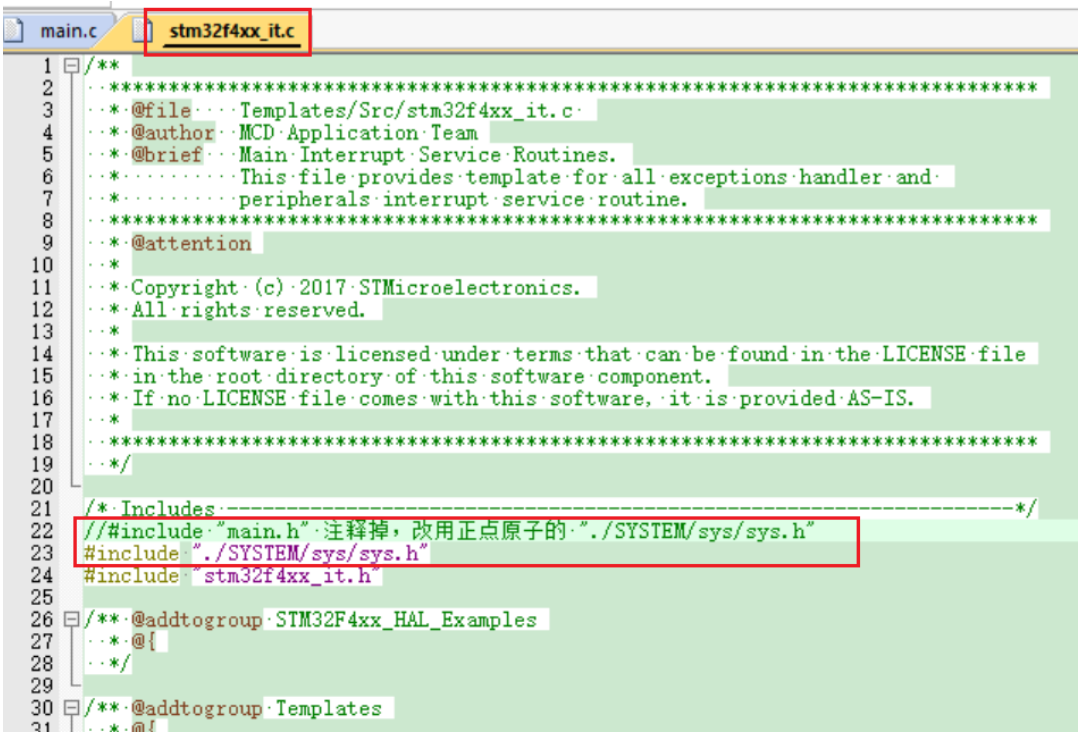
添加启动文件:
文件路径:

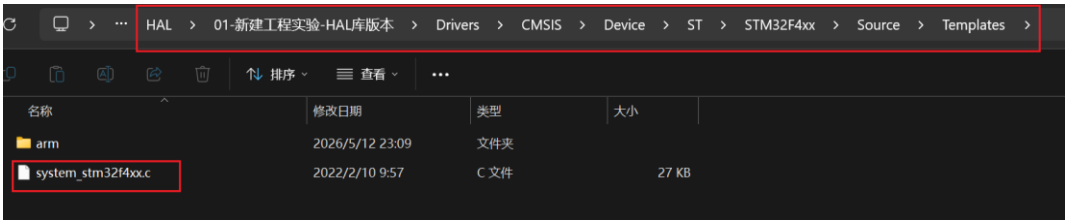


添加 User 源码：
文件路径:

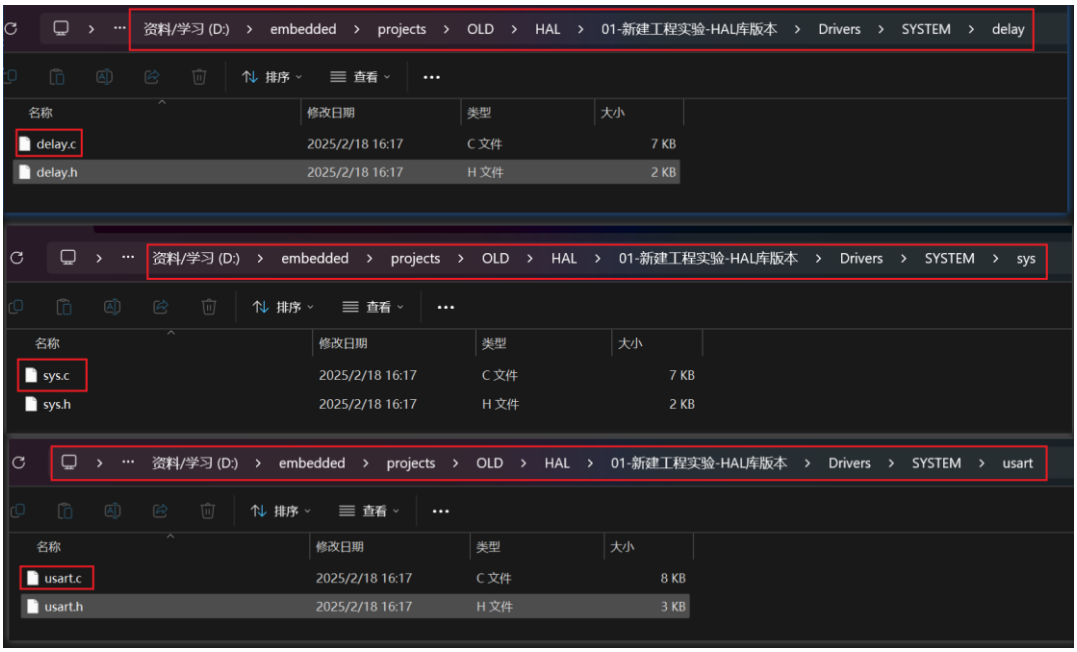


"stm32f4xx_it.c"文件的内容需要一下:





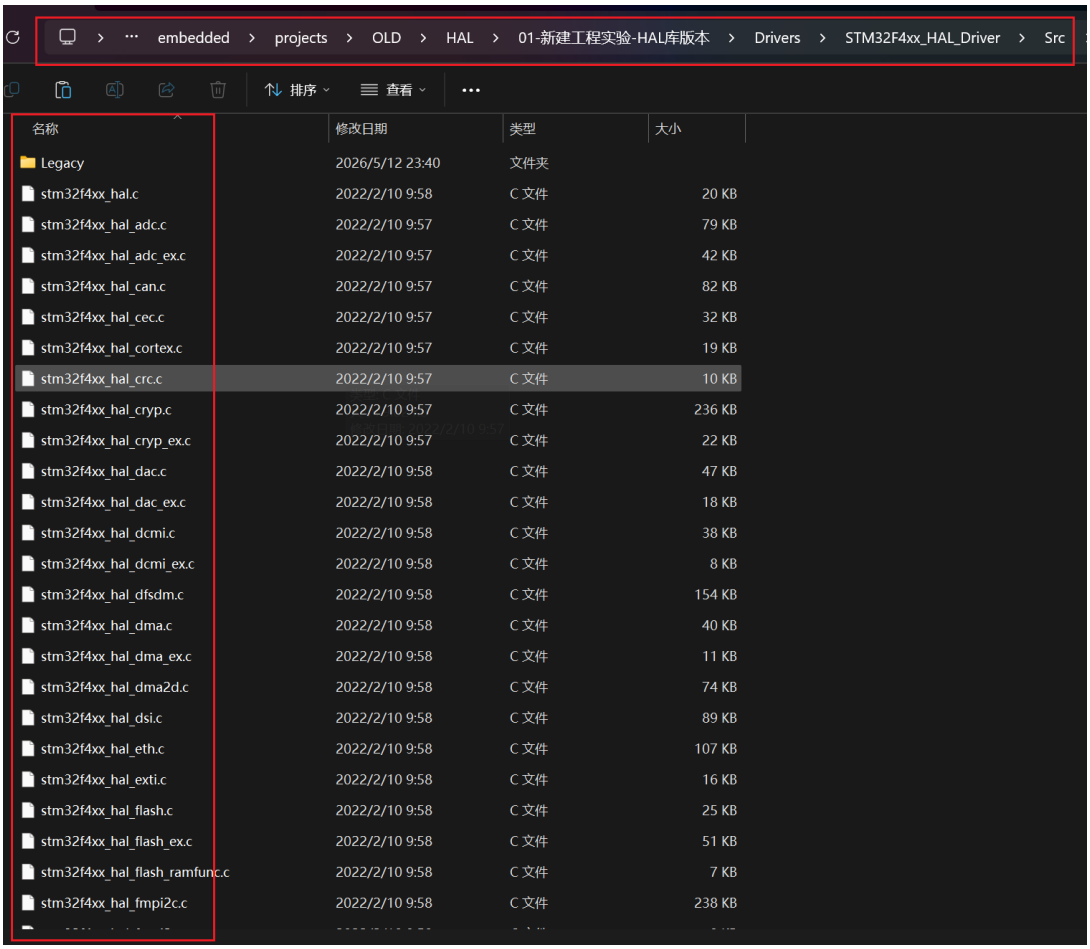
添加 SYSTEM 源码:
文件路径:



添加 STM32F4xx_HAL_Driver 源码：

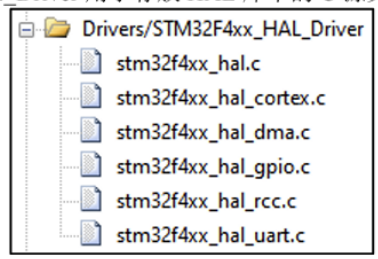
文件路径:

按需添加文件，看自己写的代码是否需要用到，用到那个加那个



④：Drivers/STM32F4xx_HAL_Driver 项目分组

Drivers/STM32F4xx_HAL_Driver 用于存放 HAL 库中的 C 源文件，如下图所示：



6.8.2.4 魔术棒设置

1，Target 选项卡　设置使用的编译器版本、外部晶振频率

2，Output 选项卡　设置Objects输出文件夹、生成.hex文件、输出浏览信息

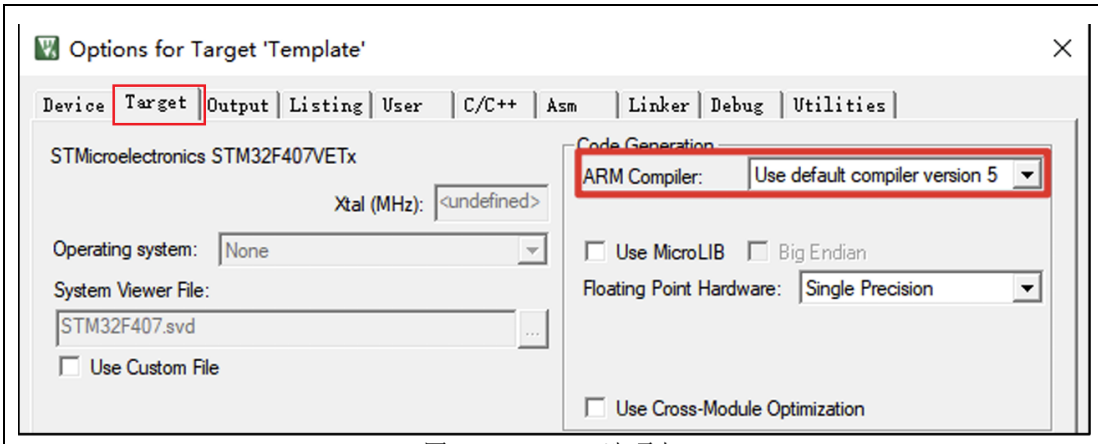
3，Listing 选项卡　设置Listing输出文件夹

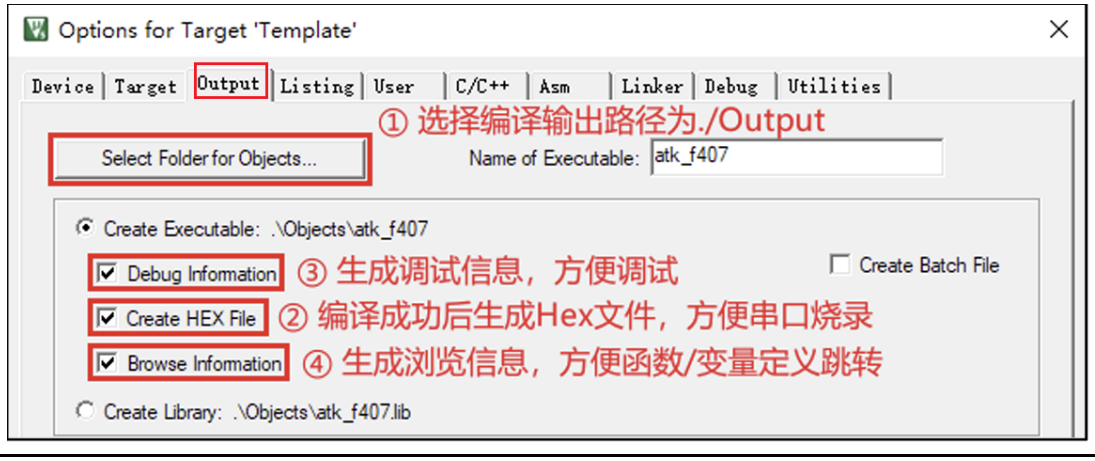
4，C/C++ 选项卡　设置全局宏定义、优化等级、C99 Mode、设置头文件包含路径

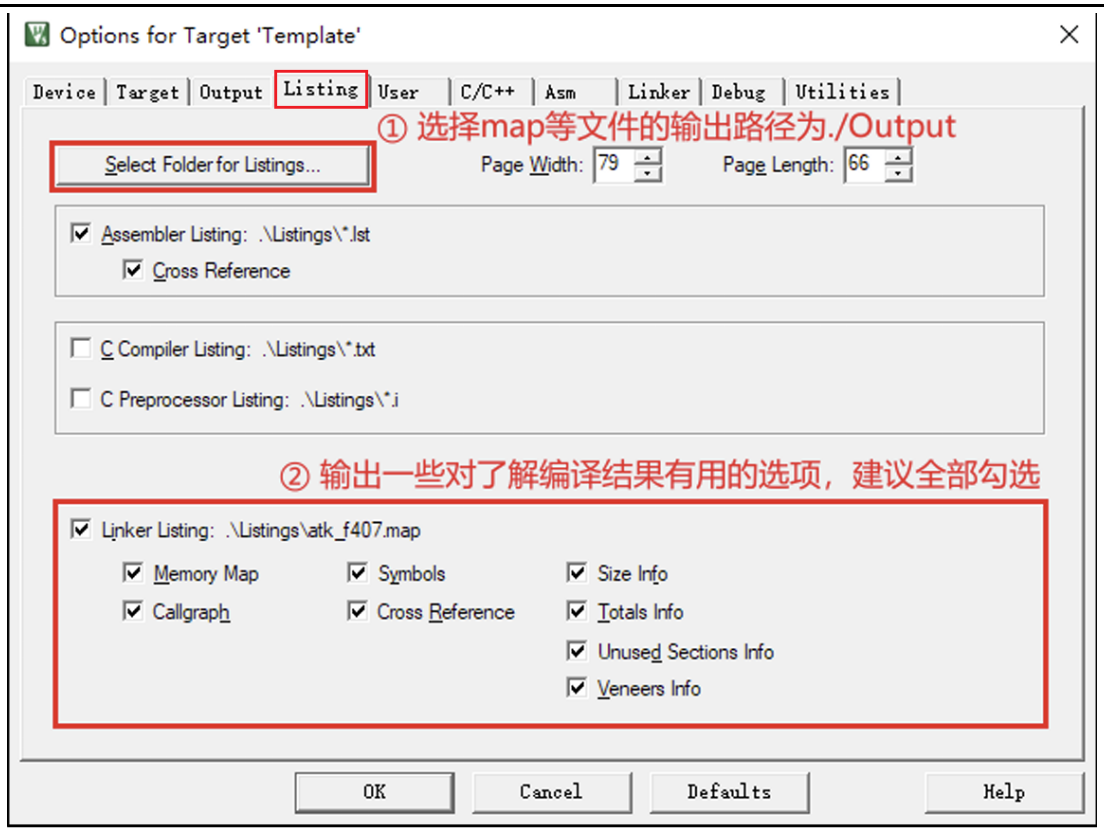
5，Debug 选项卡　设置使用的仿真器类型、选择的下载接口等

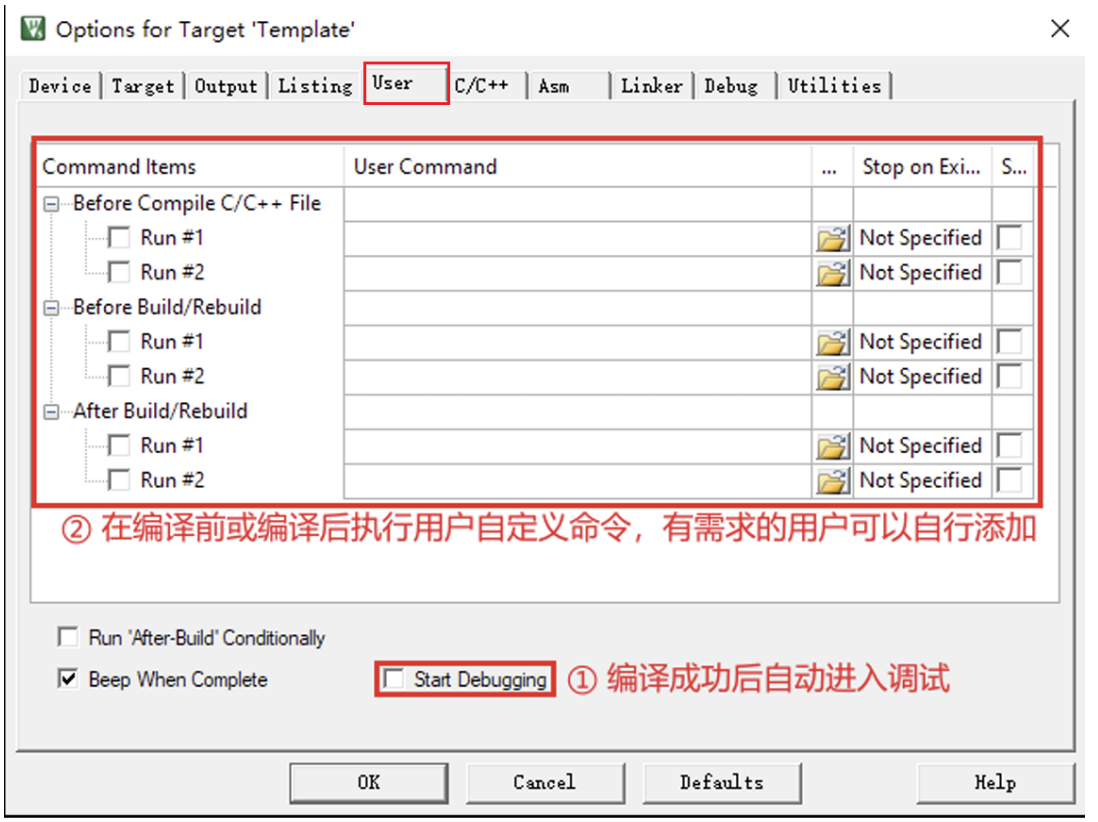
6，Utilities 选项卡　设置下载功能、下载算法、下载算法起始地址和大小等

7，Linker 选项卡(可选)　添加分散加载文件（北极星/MiniPro H750）









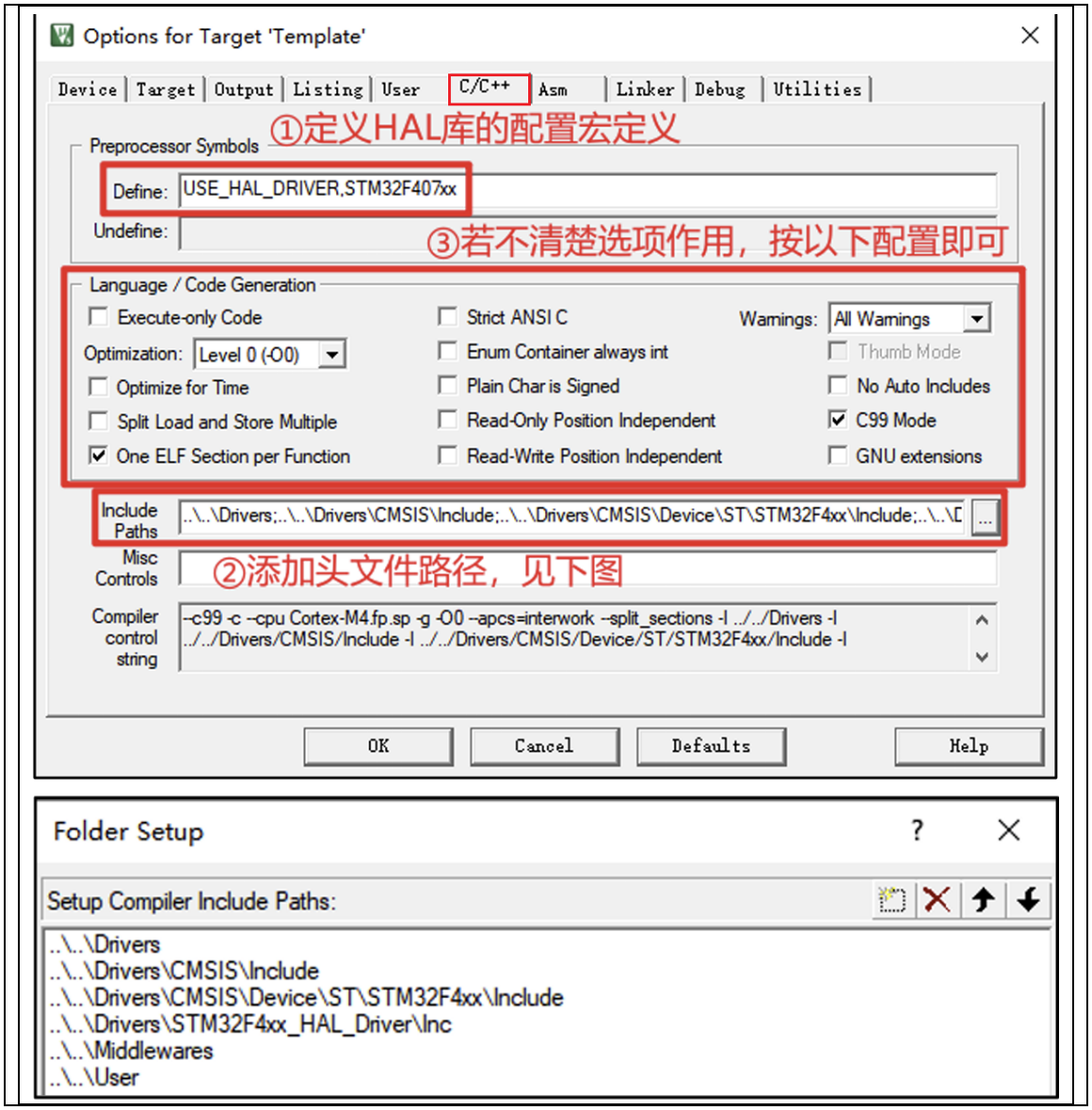
C/C++选项卡，为了正常使用 HAL 库在 Define 编辑框中添加“USE_HAL_DRIVER”和“STM32F407xx”的全局宏定义， 即“USE_HAL_DRIVER,STM32F407xx”，因为在“stm32f407xx.h”中有这两个定义，不声明宏定义则不会引入"stm32f4xx_hal.h"以及"stm32f407xx.h"文件，如下两图：

main.c | stm32f4xx.h

```
261     } while ((__STREXH(val, (__IO uint16_t *)&(REG))) != 0U); \
262   } while(0)
263
264 /* Atomic 16-bit register access macro to clear one or several bits
265 #define ATOMIC_CLEARH_BIT(REG, BIT)                              \
266   do {                                                           \
267     uint16_t val;                                                \
268     do {                                                         \
269       val = __LDREXH((__IO uint16_t *)&(REG)) & ~(BIT);          \
270     } while ((__STREXH(val, (__IO uint16_t *)&(REG))) != 0U); \
271   } while(0)
272
273 /* Atomic 16-bit register access macro to clear and set one or seve
274 #define ATOMIC_MODIFYH_REG(REG, CLEARMSK, SETMASK)
275   do {
276     uint16_t val;
277     do {
278       val = (__LDREXH((__IO uint16_t *)&(REG)) & ~(CLEARMSK)) | (SE
279     } while ((__STREXH(val, (__IO uint16_t *)&(REG))) != 0U);
280   } while(0)
281
282 /**
283   * @}
284   */
285
286 #if defined (USE_HAL_DRIVER)
287 #include "stm32f4xx_hal.h"
288 #endif /* USE_HAL_DRIVER */
289
290 #ifdef __cplusplus
291 }
```

main.c | stm32f4xx.h

```
122
123 /** @addtogroup Device_Included
124   * @{
125   */
126
127 #if defined(STM32F405xx)
128   #include "stm32f405xx.h"
129   #elif defined(STM32F415xx)
130   #include "stm32f415xx.h"
131   #elif defined(STM32F407xx)
132   #include "stm32f407xx.h"
133   #elif defined(STM32F417xx)
134   #include "stm32f417xx.h"
135   #elif defined(STM32F427xx)
136   #include "stm32f427xx.h"
137   #elif defined(STM32F437xx)
138   #include "stm32f437xx.h"
139   #elif defined(STM32F429xx)
140   #include "stm32f429xx.h"
141   #elif defined(STM32F439xx)
142   #include "stm32f439xx.h"
143   #elif defined(STM32F401xC)
144   #include "stm32f401xc.h"
```

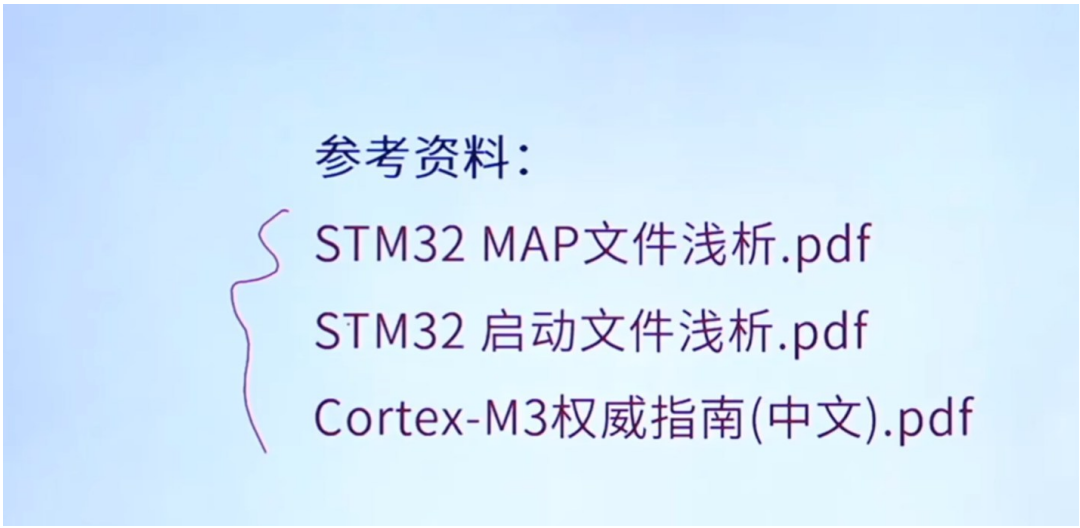


6.8.2.5 添加 main.c,并编写代码

```
#include "./SYSTEM/sys/sys.h"
#include "./SYSTEM/delay/delay.h"
#include "./SYSTEM/usart/usart.h"

int main(void)
{
    HAL_Init();                                 /* 初始化 HAL 库 */
    sys_stm32_clock_init(336, 8, 2, 7); /* 配置时钟，168MHz */
    delay_init(168);                            /* 初始化延时 */
    usart_init(115200);                         /* 初始化串口 */

    while (1)
    {
        printf("Hello, M144Z-M4 STM32F407!\r\n");
        delay_ms(1000);
    }
}
```

6.9 STM32 启动过程浅析

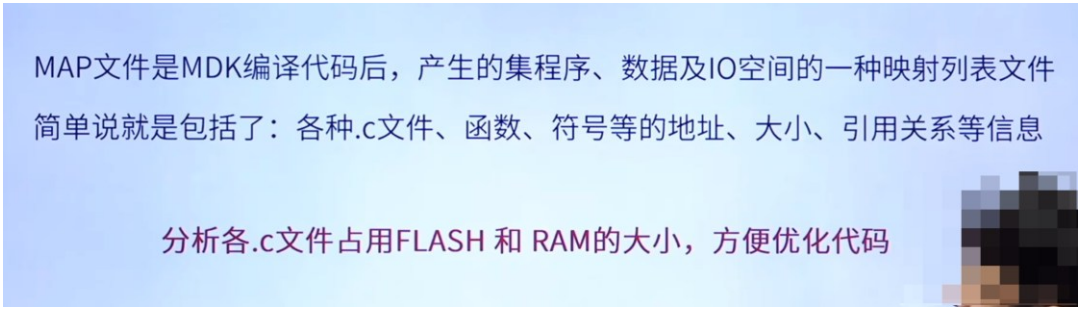
6.9.1 MAP 文件浅析



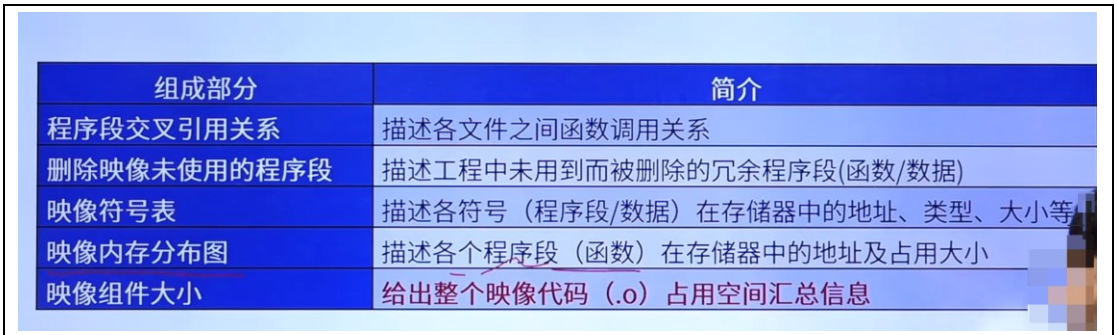
MDK编译过程文件（11种）

文件类型	简介
.o	可重定向对象文件，每个.c/.s文件都对应一个.o文件
.axf	可执行对象文件，由.o文件链接生成，仿真的时候需要用到此文件
.hex	INTEL Hex格式文件，用于下载到MCU运行，由.axf转换而来
.map	连接器生成的列表文件，对分析程序存储占用情况非常有用
其他	.crf、.d、.dep、.lnp、.lst、.htm、.build_log.htm等一般用不到

6.9.1.1 MAP 文件概念和作用



6.9.1.2 MAP 文件的组成



组成部分	简介
程序段交叉引用关系	描述各文件之间函数调用关系
删除映像未使用的程序段	描述工程中未用到而被删除的冗余程序段(函数/数据)
映像符号表	描述各符号（程序段/数据）在存储器中的地址、类型、大小等
映像内存分布图	描述各个程序段（函数）在存储器中的地址及占用大小
映像组件大小	给出整个映像代码（.o）占用空间汇总信息

MAP 文件中的对应位置：

① Section Cross References：程序段交叉引用关系

② Removing Unused input sections from the image.：删除映像未使用的程序段

③ Image Symbol Table：映像符号表，Global Symbols（全局函数会在此类下出现），Local Symbols（局部函数会在此类下出现）

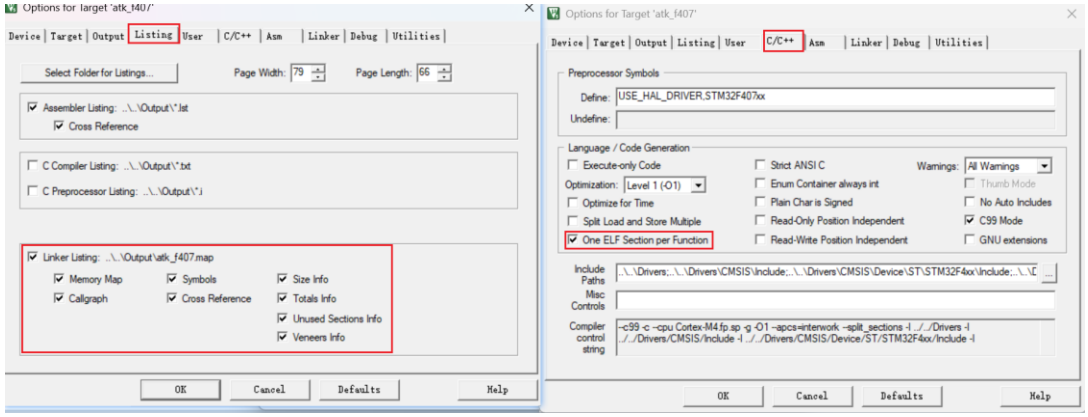
④ Memory Map of the image：映像内存分布图

⑤ Image component sizes：映像组件大小

6.9.1.3 MAP 文件实操

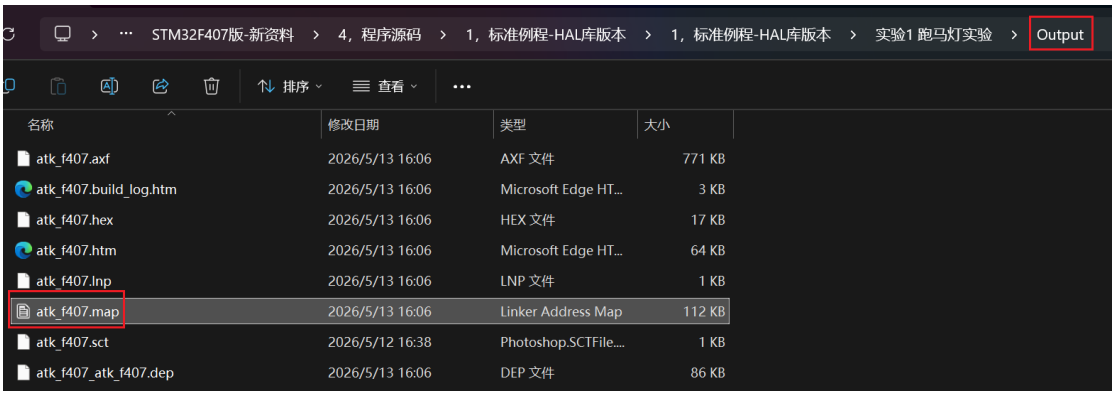
学会分析:哪个.c 占用 flash 和 ram 比较大，以便针对性的优化

魔术棒设置：

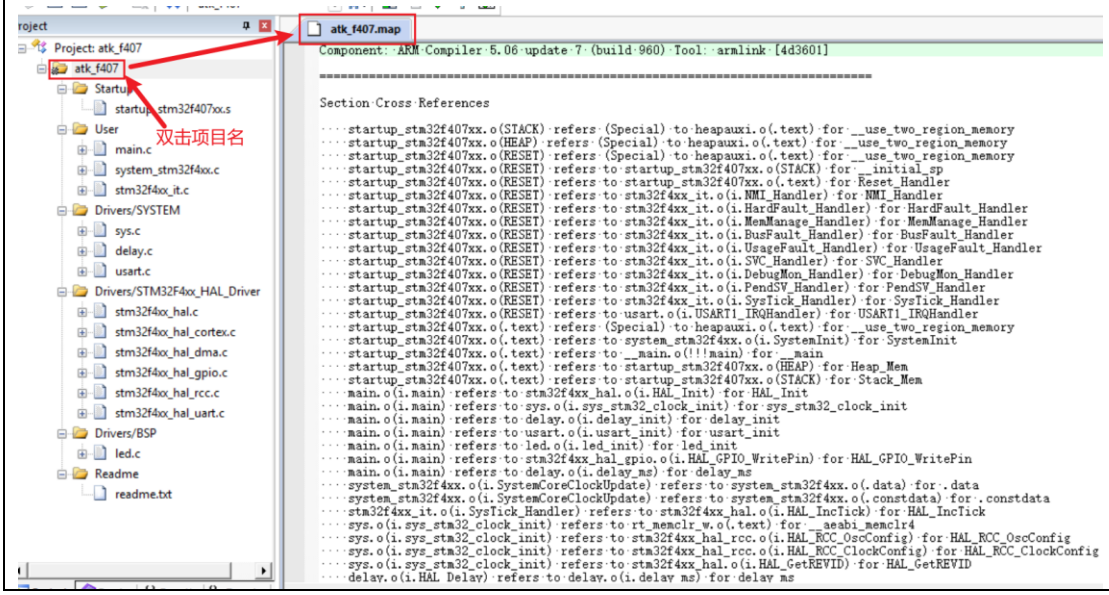


打开两种文件的方式：

⑥ ： 在 Output 文件里面直接打开



⑦ ： 直接在在 keil 软件里面双击项目即可打开



6.9.2 STM32 启动过程

6.9.2.1 STM32 启动模式（F1/F4/F7/H7）(自举模式)

2.1，STM32启动模式（F1/F4/F7/H7）（也称自举模式）

M3/M4/M7等内核复位后，做的第一件事：

1，从地址 0x0000 0000 处取出堆栈指针 MSP 的初始值，该值就是栈顶地址

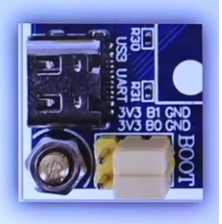
2，从地址 0x0000 0004 处取出程序计数器指针 PC 的初始值，该值是复位向量

芯片厂商可以会把0x0000 0000和0x0000 0004地址映射到其它的地址！！！

6.9.2.1.1 F1

STM32启动模式（F1）

在系统复位后，SYSCLK的第4个上升沿，BOOT引脚的值将被锁存



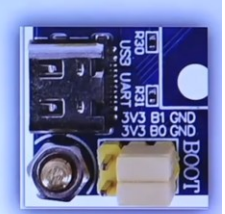
启动模式选择引脚		启动模式	0x00000000 映射地址	0x00000004 映射地址
BOOT1	BOOT0			
x	0	主闪存存储器	0x08000000	0x08000004
0	1	系统存储器	0x1FFFF000	0x1FFFF004
1	1	内置SRAM	0x20000000	0x20000004

无法下载程序解决方法：B0接3.3，按复位，然后再下载

6.9.2.1.2 F4

STM32启动模式（F4）

在系统复位后，SYSCLK的第4个上升沿，BOOT引脚的值将被锁存

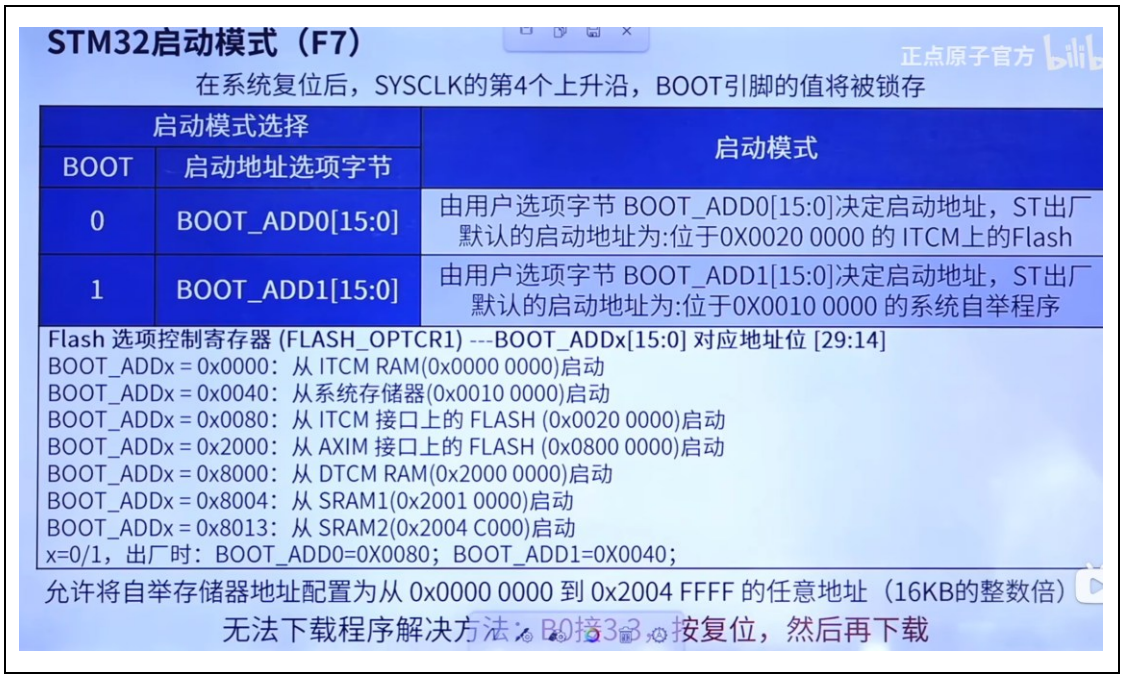


启动模式选择引脚		启动模式	0x00000000 映射地址	0x00000004 映射地址
BOOT1	BOOT0			
x	0	主闪存存储器	0x08000000	0x08000004
0	1	系统存储器	0x1FFF0000	0x1FFF0004
1	1	内置SRAM	0x20000000	0x20000004

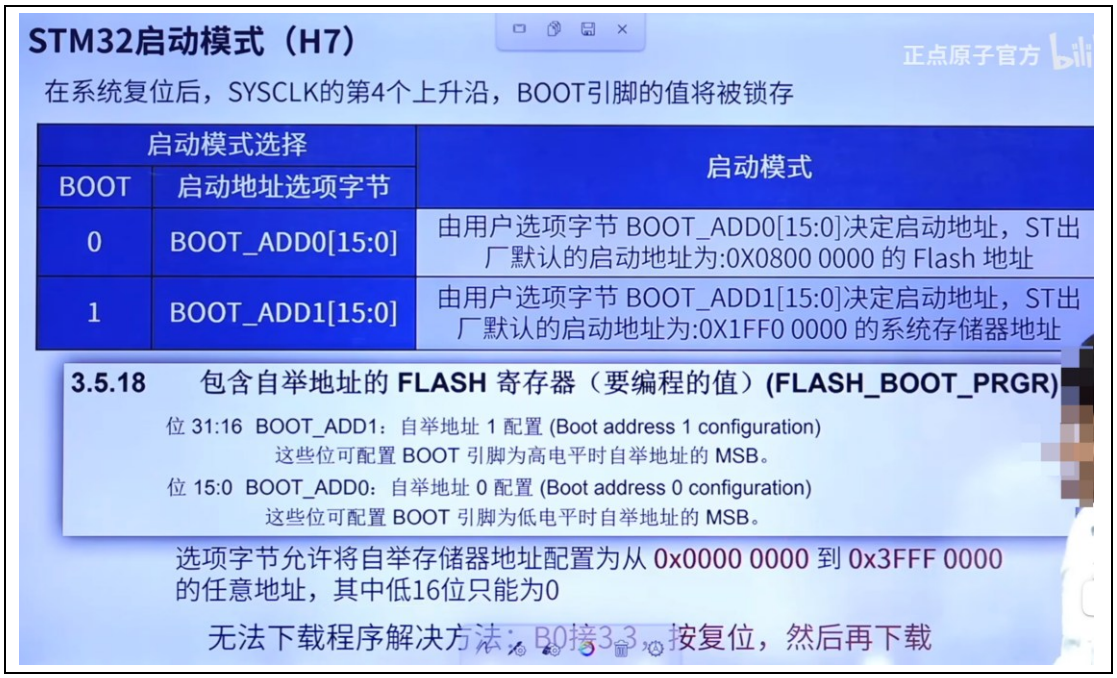
如果要使用FSMC重映射到外部的SRAM：可以通过配置SYSCFG_MEMRMP寄存器

无法下载程序解决方法：B0接3.3，按复位，然后再下载

6.9.2.1.3 F7

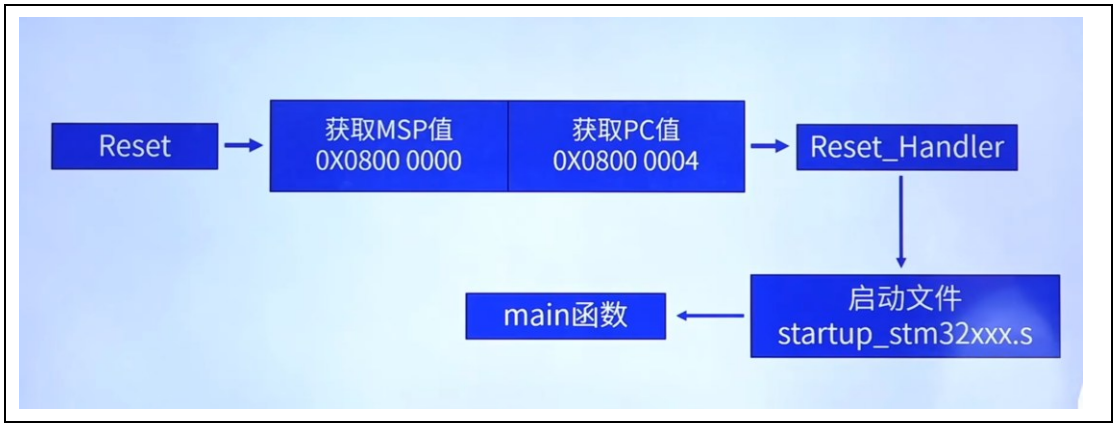


6.9.2.1.4 H7



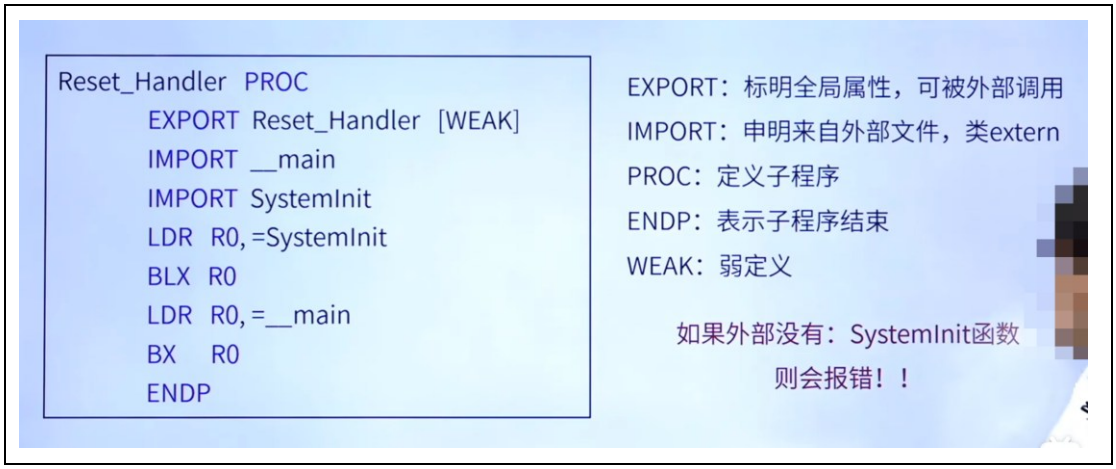
6.9.2.2 2 启动过程

6.9.2.2.1 内部 FLASH 启动为例

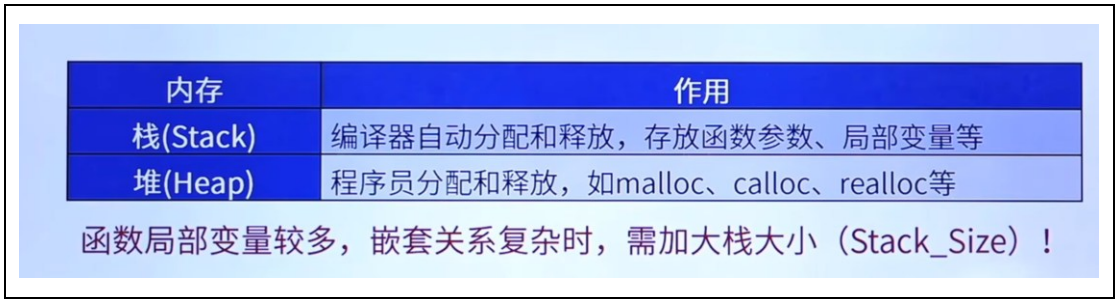


6.9.2.2.2 启动文件介绍

6.9.2.2.3 Reset_Hander 函数介绍

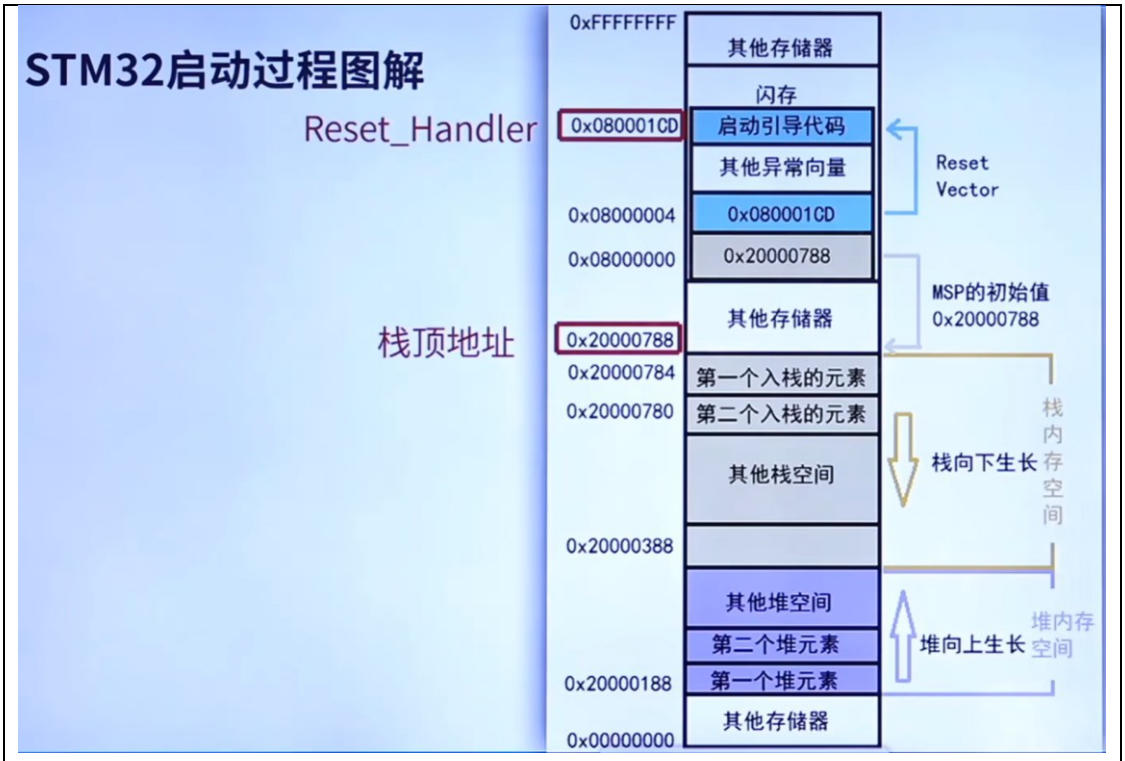


6.9.2.2.4 堆栈简介



6.9.2.2.5 STM32 启动过程图解

F1 为例：



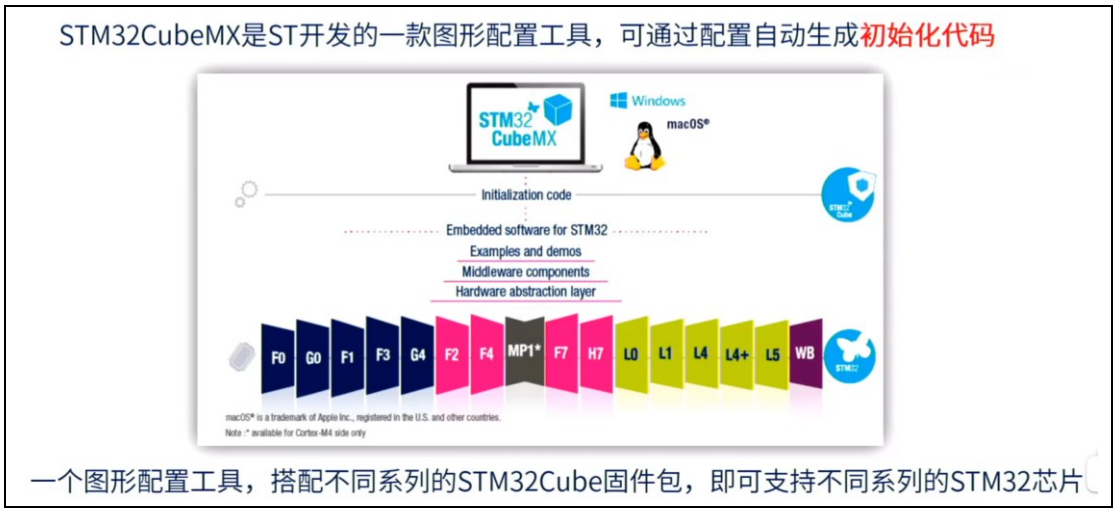
F4(startup_stm32f407xx.s):

Reset_Handler：0x08000229(复位函数地址)

__initial_sp：0x20000788(栈顶地址)

6.10 STM32CubeMX 简介和安装

6.10.1 STM32CubeMX 简介



6.10.2 STM32CubeMX 安装

6.10.2.1 STM32CubeMX 软件获取

1，获取Java软件

方式一：https://www.java.com

方式二：开发板A盘资料：A 盘→6，软件资料→1，软件→ 3，STM32CubeMX

2，获取STM32CubeMX软件

方式一：https://www.st.com

方式二：开发板A盘资料：A 盘→6，软件资料→1，软件→ 3，STM32CubeMX

JAVA 下载：
https://www.oracle.com/java/technologies/downloads/#java17-windows

STM32CubeMX 下载：
https://www.st.com.cn/zh/development-tools/stm32cubemx.html

6.10.2.2 搭建 java 运行环境

6.10.2.2.1 JDK17 安装包下载

一、JDK17 的安装包下载

JDK17的官网下载链接：

https://www.oracle .com/java /technologies/downloads/#java17

很多同学在其他的地方也能下载，但是我还是强烈推荐如果有条件使用官网的下载链接。下载如下图所示：

Java SE Development Kit 17.0.6 downloads

Thank you for downloading this release of the Java™ Platform, Standard Edition Development Kit (JDK™). The JDK is a development environment for building applications and components using the Java programming language.

The JDK includes tools for developing and testing programs written in the Java programming language and running on the Java platform.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	172.11 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.zip (sha256)
x64 Installer	153.22 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.exe (sha256)
x64 MSI Installer	152.01 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.msi (sha256)

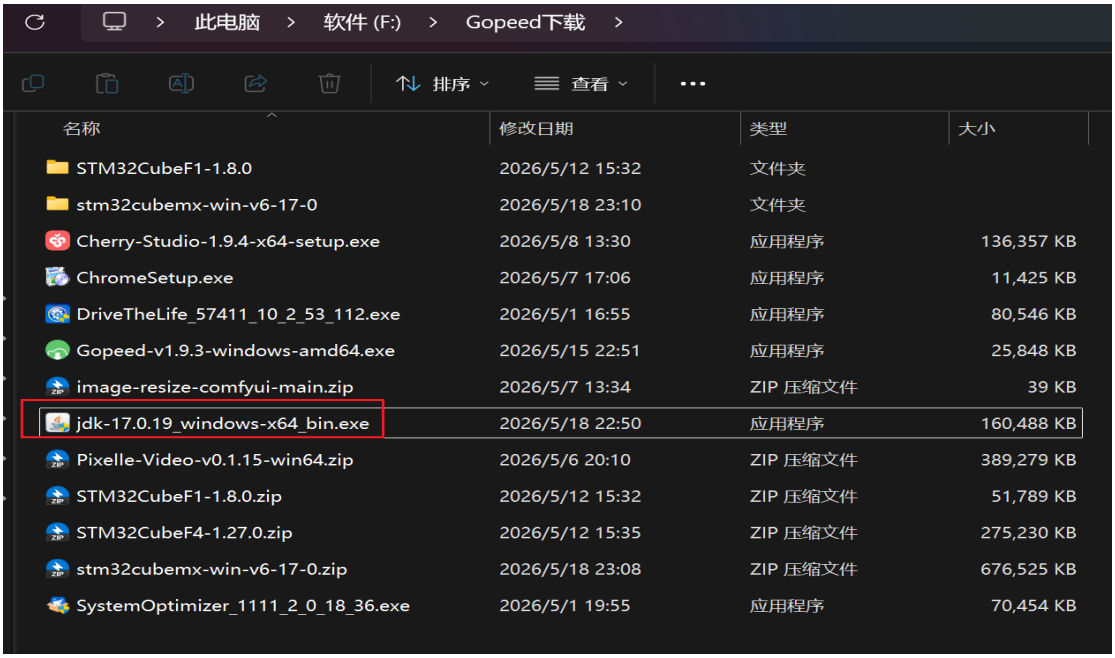
下载完成后如下图：

此电脑 › 新加卷 (D:) › soft › JDK ›

名称	修改日期	类型	大小
jdk-17_windows-x64_bin.exe	2023/3/7 16:29	应用程序	156,894 KB

6.10.2.2.2 安装 jdk17

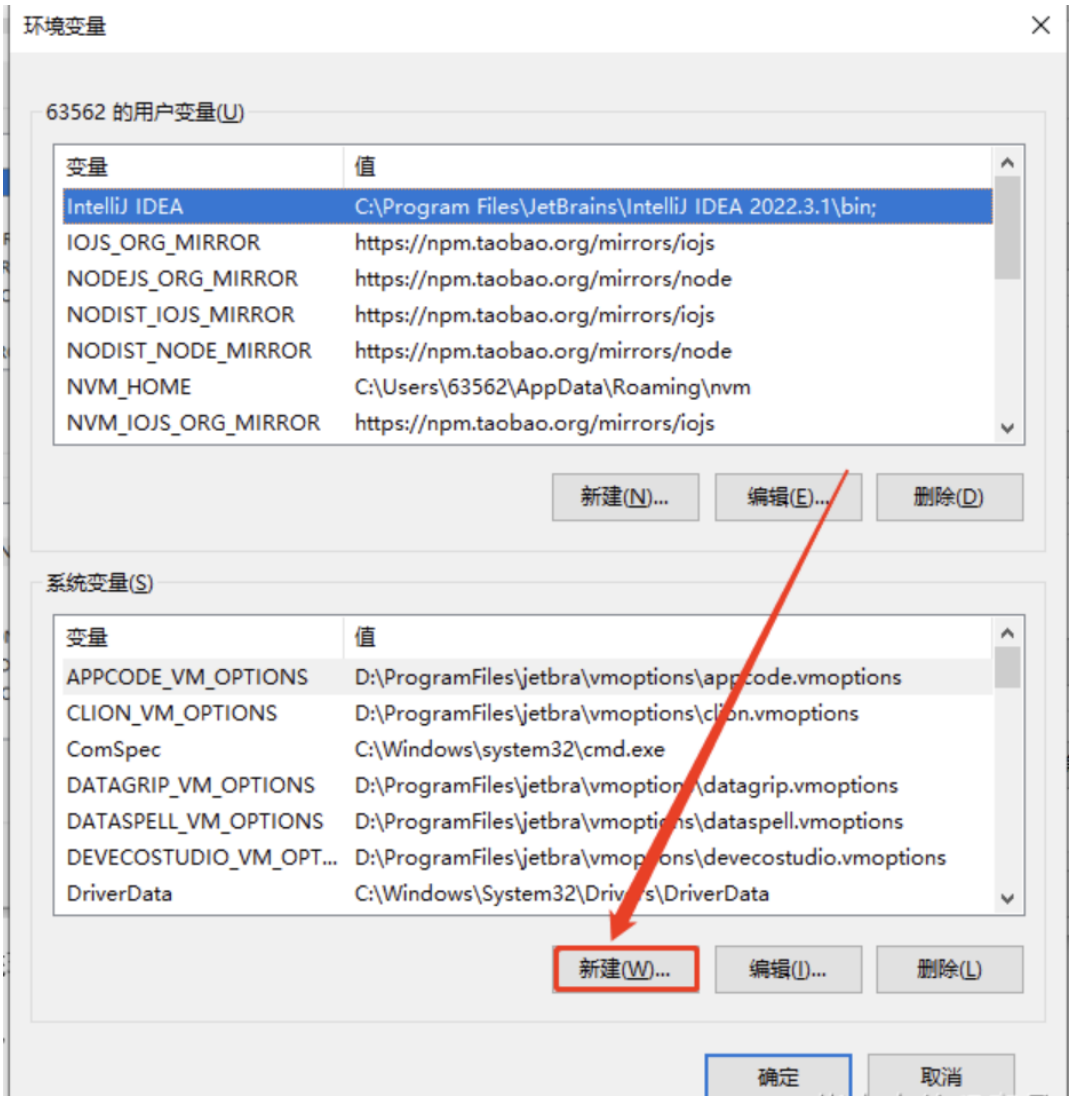
第一步：运行 JDK 的 exe 文件



无脑下一步即可

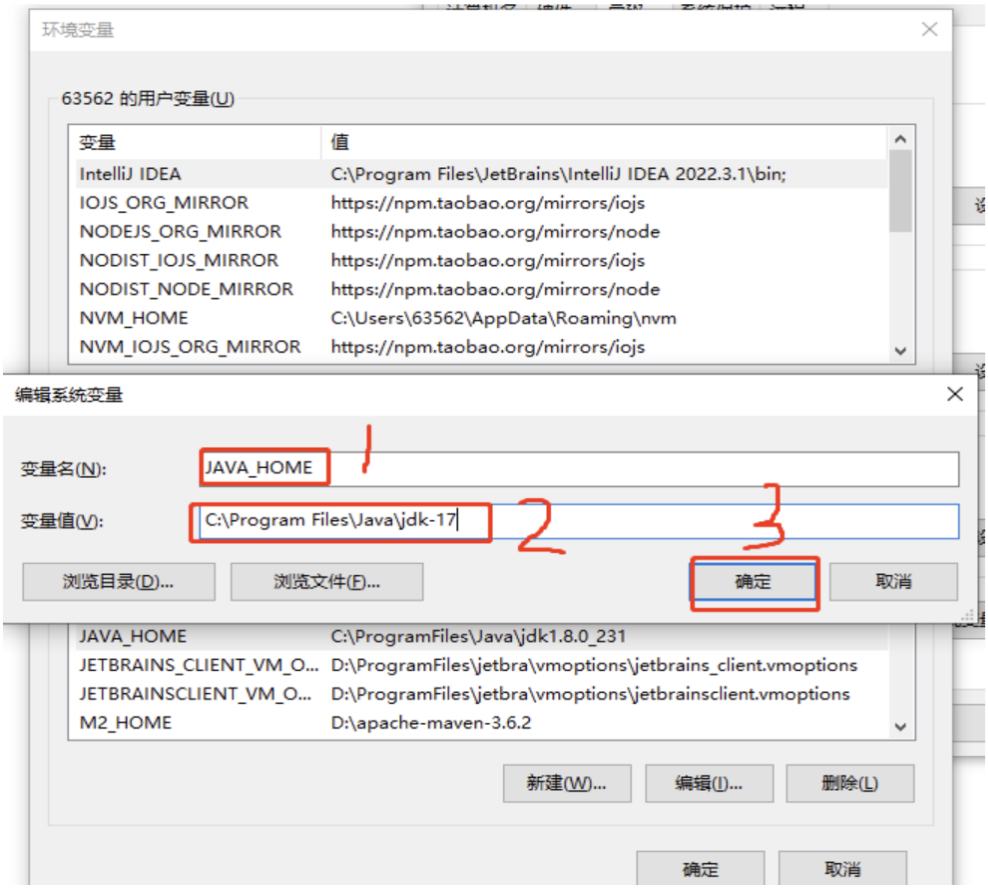
第二步：配置环境变量

① 新建系统环境变量

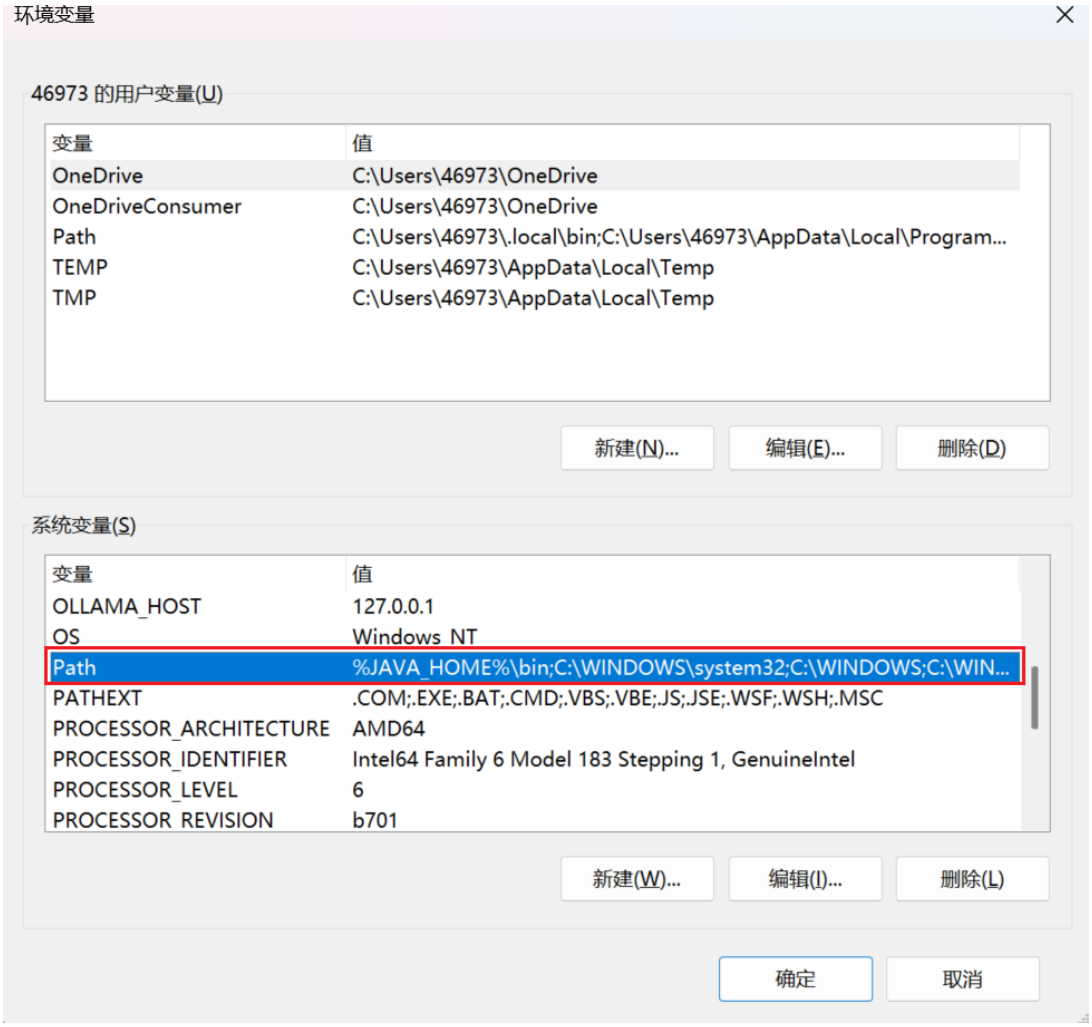


② 添加 JAVA_HOME 环境变量

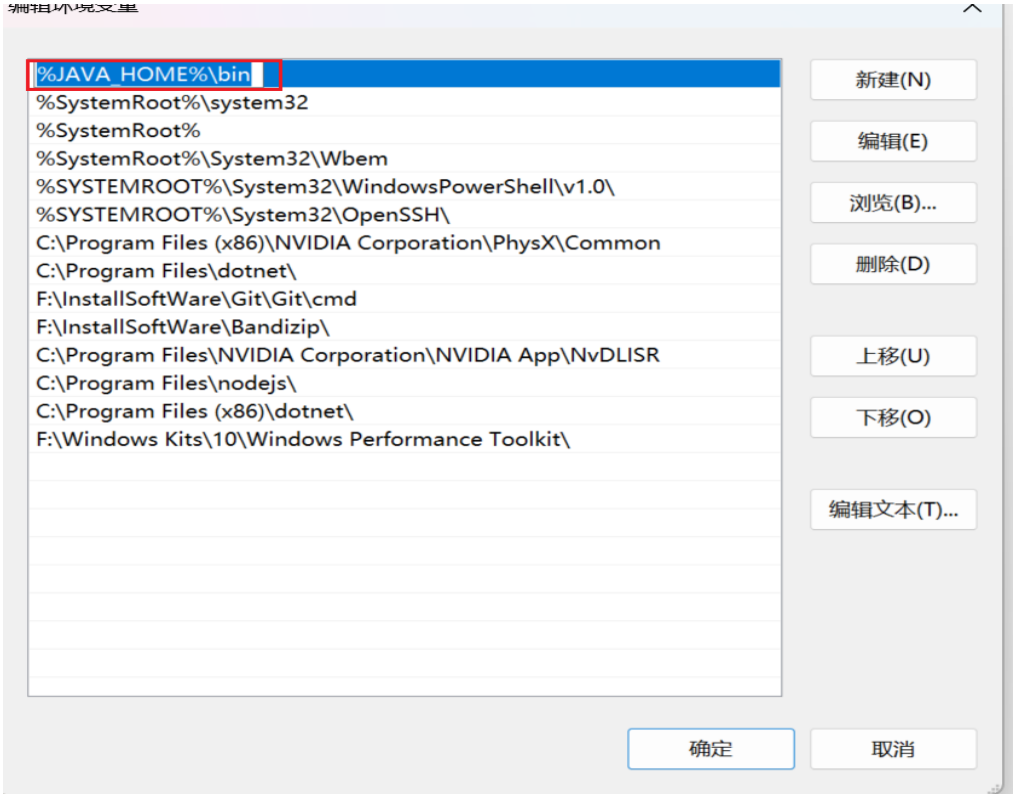
分别添加变量名：JAVA_HOME、变量值: C:Program FilesJavajdk-17,点击“确定”即可，如下图所示：



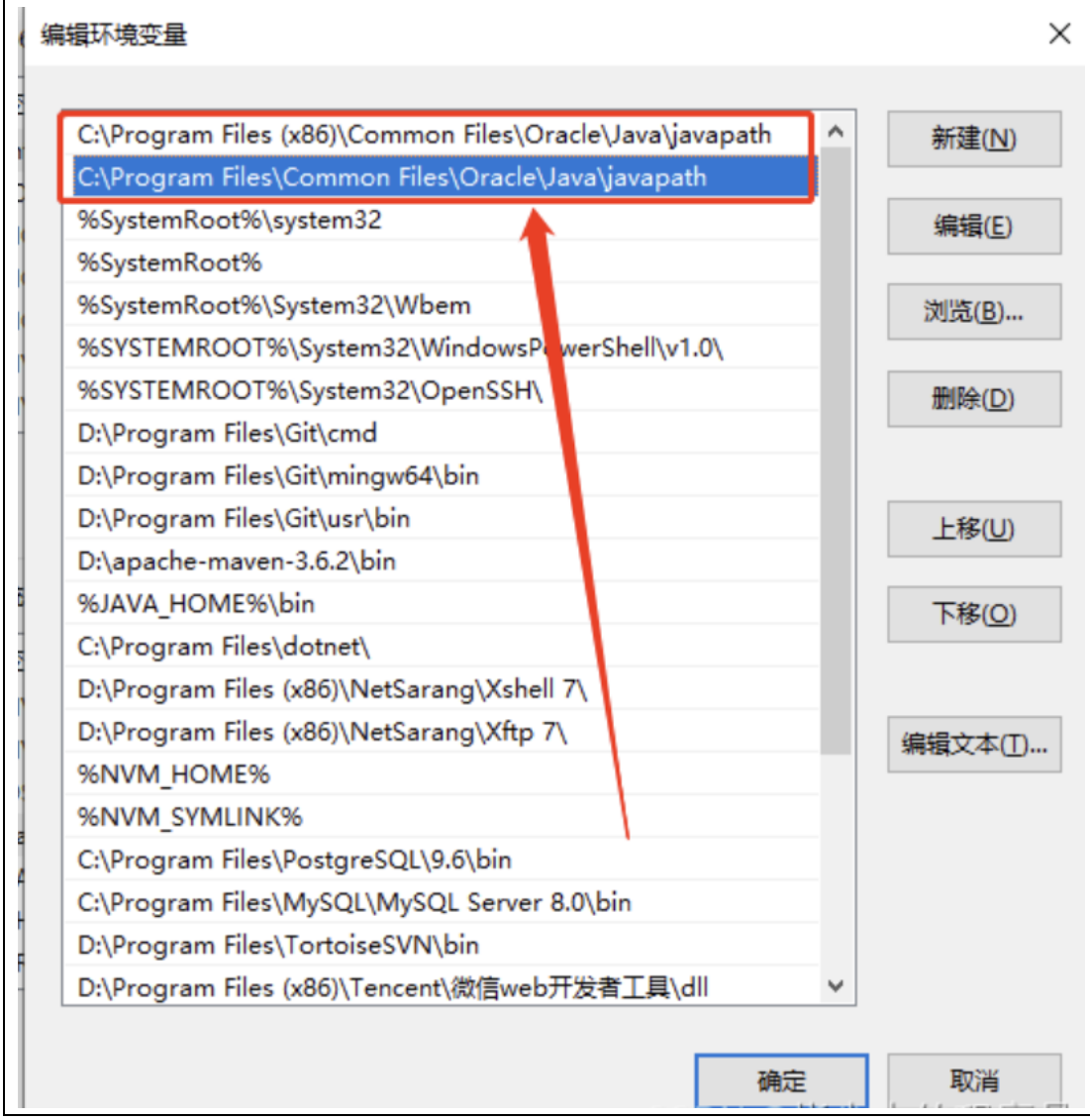
③ 添加 path



输入” %JAVA_HOME%\bin”:



注意:由于 JDK17 在我们安装的时候可能会自动进行环境变量配置，我们需要在环境变量配置 PATH 中删除如下信息:



④ 检测 JDK 是否安装成功

```
C:\Users\46973>java -version
java version "17.0.19" 2026-04-21 LTS
Java(TM) SE Runtime Environment (build 17.0.19+9-LTS-183)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.19+9-LTS-183, mixed mode, sharing)
```

6.10.2.3 安装 STM32CubeMX 软件

双击”SetupSTM32CubeMX-6.3.0-Win.exe”，然后按部就班进行安装

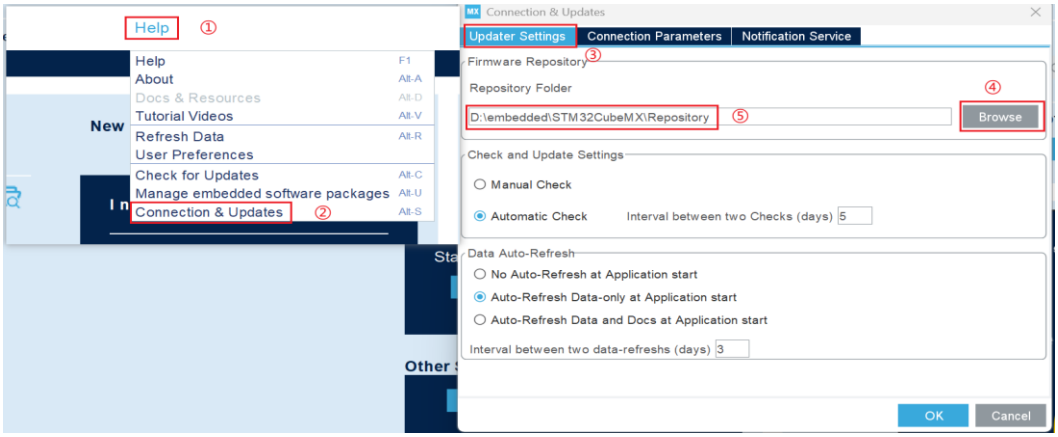
建议不要安装在C盘，占用系统盘内存

安装路径不能有中文字符

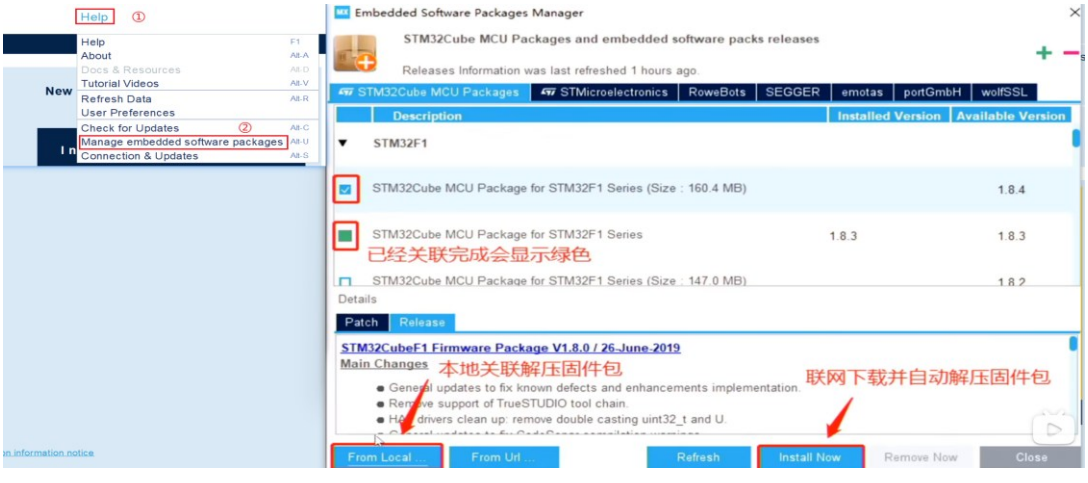
6.10.2.4 下载和关联 SEM32cube 固件包

建议在 D 盘新建一个仓库用于存放 STM32Cube 固件包，路径：D:\embedded\STM32CubeMX\Repository

打开软件进行关联

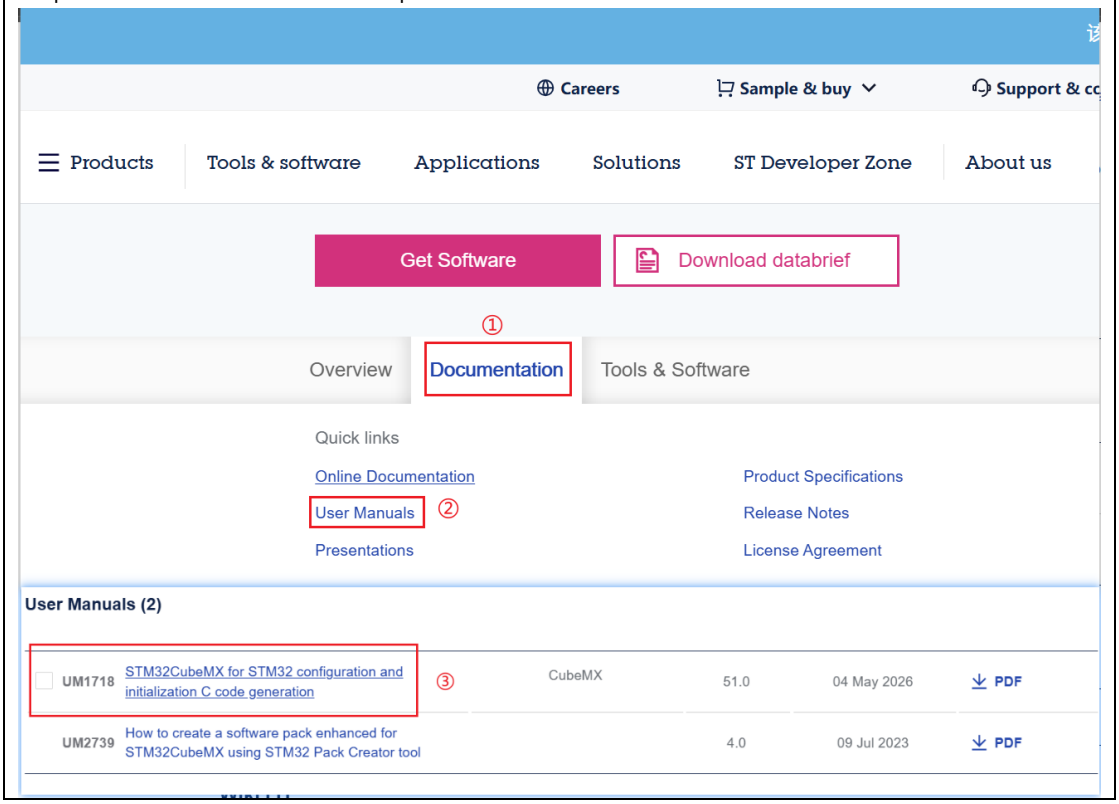


6.10.2.4.1 关联 STM32 固件包



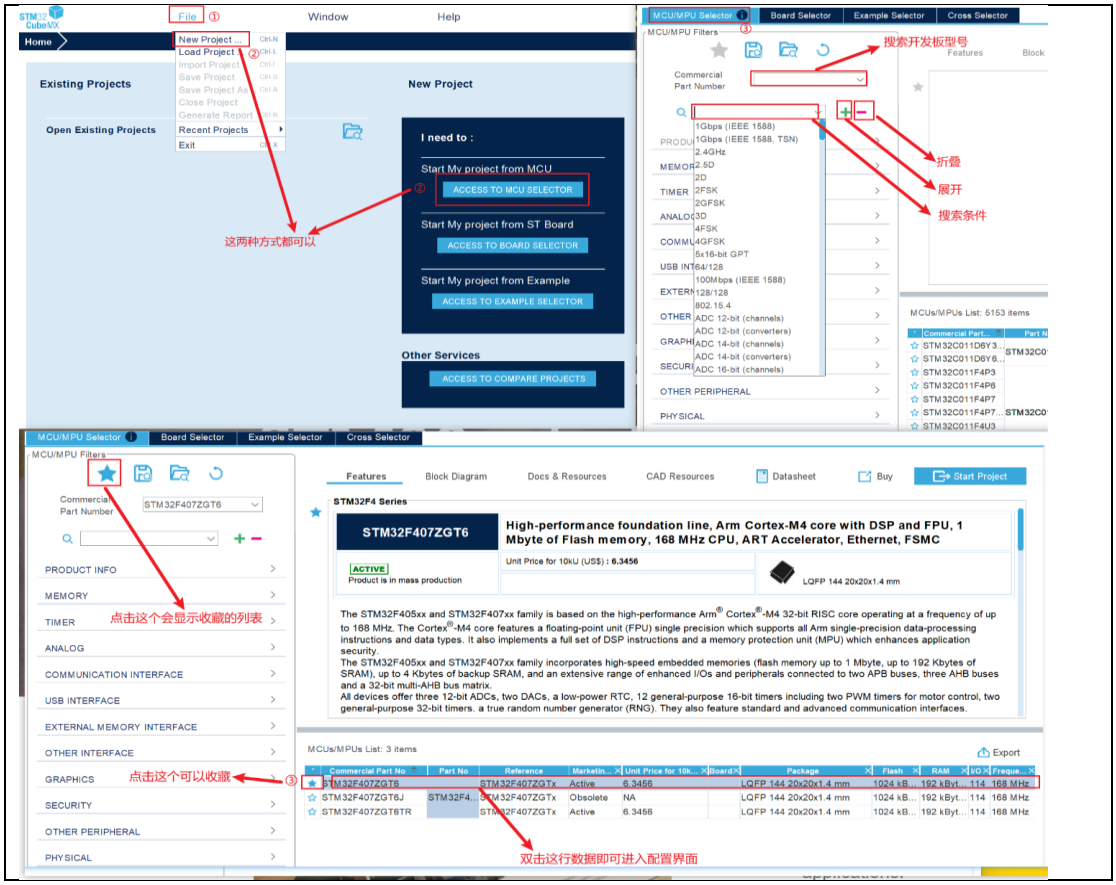
6.10.2.4.2 STM32CubeMX 使用手册

STM32CubeMX 使用手册下载：
https://www.st.com/en/development-tools/stm32cubemx.html#

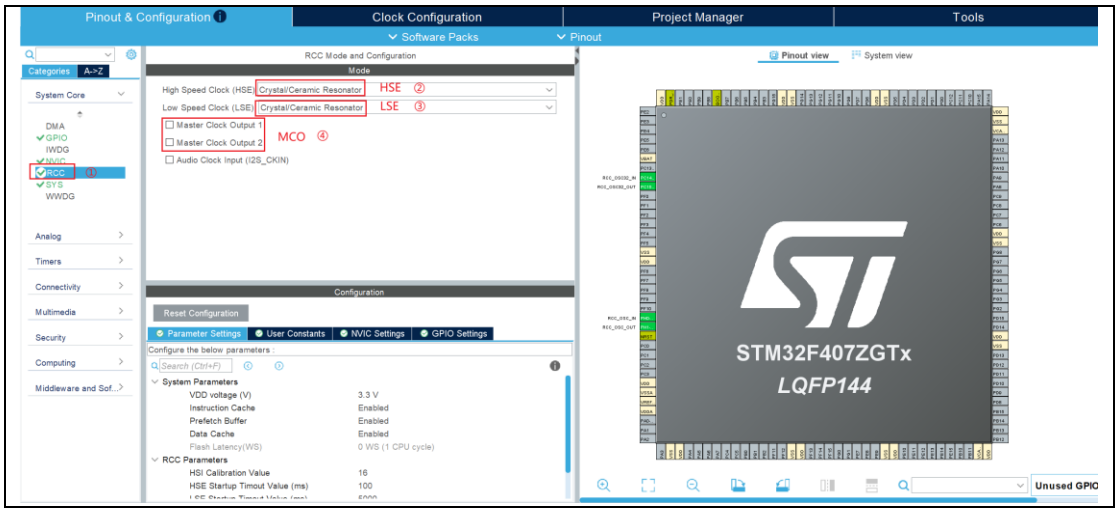


6.10.3 新建 STM32CubeMX 工程步骤

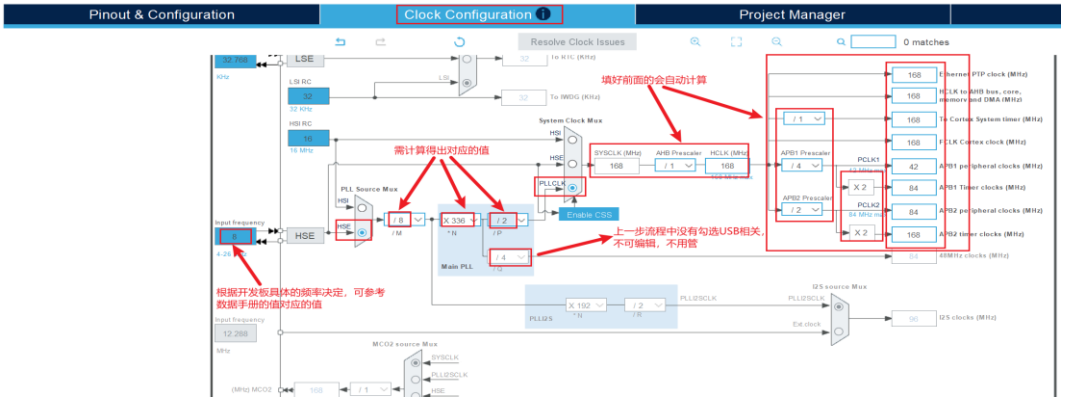
6.10.3.1 工程初步建立



6.10.3.2 时钟模块设置



6.10.3.3 时钟系统设置



以上数据不同开发板数值不一样，按需配置

① **参数说明：**

STM32F4 的时钟系统就像一个**精密的齿轮传动组**。外部晶振（HSE）是动力源，PLL（锁相环）是变速箱，最终输出给 CPU 和外设。

以下是各个参数的详细含义及计算逻辑：

参数	全称	含义	作用
PLL Source Mux	PLL 输入源选择	选择 PLL 的"原材料"来自哪里	通常选 **HSE**（外部晶振，更精准）或 **HSI**（内部 RC 振荡器，省钱但精度稍差）。
Input frequency	输入频率	进入 PLL 之前的原始频率	取决于你选的源。如果是 HSE，就是你板子上晶振的频率（如 8 MHz）。
/M	PLL 分频系数	**第一步：降压/分频**	将高频的输入信号降低到 **1~2 MHz**。这是为了给内部的 VCO（压控振荡器）提供一个标准的基准频率。
***N**	PLL 倍频系数	**第二步：升压/倍频**	将 /M 处理后的低频信号大幅放大。VCO 的输出频率必须在 **192~432 MHz** 之间。
/P	系统时钟分频	**第三步：输出给 CPU**	将 VCO 的高频信号分频，得到最终的 **SYSCLK**（系统主频）。F407 最高支持 168 MHz。
/Q	专用外设分频	**第四步：输出给 USB/SDIO**	从 VCO 单独引出一路，专门给 USB、SDIO、RNG 等需要严格 **48 MHz** 的外设供电。

① **物理限制与数学逻辑**

STM32 内部的 PLL 电路不是想怎么变就怎么变的，它有两个硬性物理限制：

1. **VCO 输入限制**：VCO 只能处理 **1 MHz 到 2 MHz** 的输入信号。
   - 如果你的晶振是 8 MHz，直接进 VCO 会烧坏或无法工作，所以必须用 **/M** 把它降到 1 MHz。
     8 MHz / M = 1 MHz ⇒ M = 8。
2. **VCO 输出限制**：VCO 产生的高频信号必须在 **192 MHz 到 432 MHz** 之间。
   - 我们要尽可能高以获得高性能，所以通常选接近上限的值，比如 336 MHz 或 432 MHz（但 F407 的 CPU 跑不到 432，所以选 336 比较稳妥）。
     1 MHz × N = 336 MHz ⇒ N = 336。

---

3. 实战计算演示（以你的 8 MHz 晶振为例）

我们的目标是：**CPU 跑 168 MHz，USB 跑 48 MHz。**

第一步：确定 M（满足 VCO 输入要求）

$$\text{VCO_Input} = \frac{\text{HSE}}{M}$$

要求 VCO_Input 在 1~2 MHz 之间。

取 M = 8，则 $\text{VCO_Input} = 8 / 8 = 1 \text{ MHz}$。（符合要求 ✅）

第二步：确定 N（满足 VCO 输出要求）

$$\text{VCO_Output} = \text{VCO_Input} \times N$$

要求 VCO_Output 在 192~432 MHz 之间。

为了得到 168 MHz 的主频，我们反推：如果 /P 最小是 2，那么 VCO 至少要是 $168 \times 2 = 336$ MHz。

取 N = 336，则 $\text{VCO_Output} = 1 \times 336 = 336 \text{ MHz}$。（符合要求 ✅）

第三步：确定 P（得到系统主频 SYSCLK）

$$\text{SYSCLK} = \frac{\text{VCO_Output}}{P}$$

我们要 168 MHz。

取 P = 2，则 SYSCLK = 336/2 = 168 MHz。（达到 F407 极限 ✅）

第四步：确定 Q（得到 USB 时钟）

$$\text{USB_CLK} = \frac{\text{VCO_Output}}{Q}$$

USB 协议强制要求 48 MHz。

取 Q = 7，则 $\text{USB_CLK} = 336 / 7 = 48 \text{ MHz}$。（完美匹配 ✅）

4. 总结公式

在 CubeMX 中，你只需要记住这个核心链条：

$$SYSCLK = \frac{HSE}{M} \times \frac{N}{P}$$

- **/M** 是为了让内部电路“吃得消”(降到 1-2 MHz)。
- ***N** 是为了让速度“提上去”(升到 192-432 MHz)。
- **/P** 是为了让 CPU“跑得稳”(分到 168 MHz)。
- **/Q** 是为了让 USB“不报错”(分到 48 MHz)。

5. 给你的建议

既然你有 Java 背景，可以把这个过程想象成 **Java 中的类型转换和数值计算**：

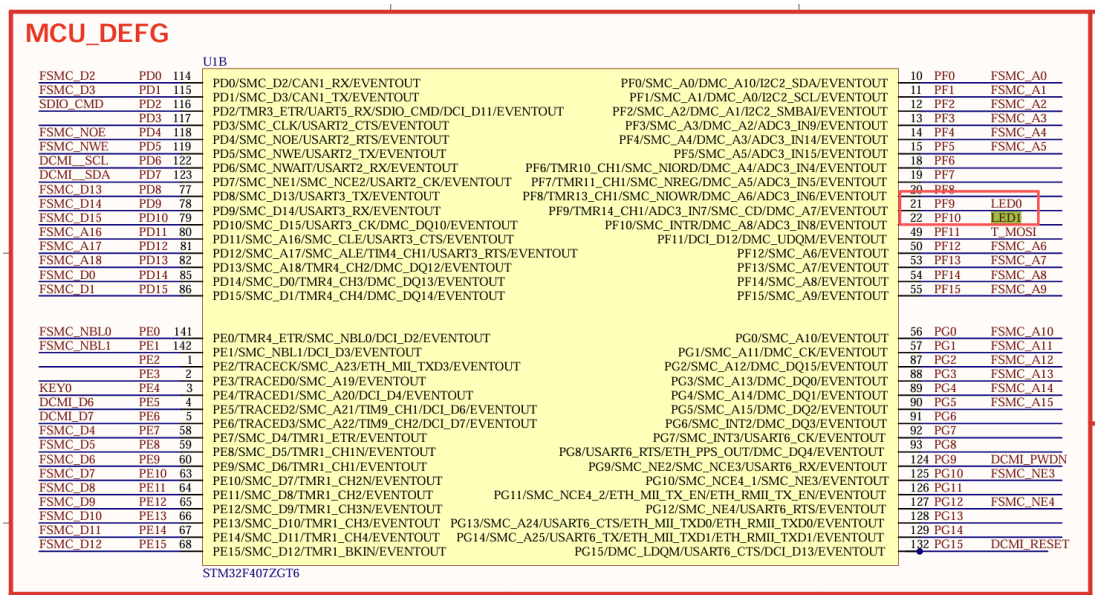
- `HSE` 是 `int input = 8;`
- `/M` 是 `double vco_in = input / 8.0; // 结果 1.0`
- `*N` 是 `double vco_out = vco_in * 336; // 结果 336.0`
- `/P` 是 `int sysclk = (int)(vco_out / 2); // 结果 168`

6.10.3.4 GPIO 引脚配置

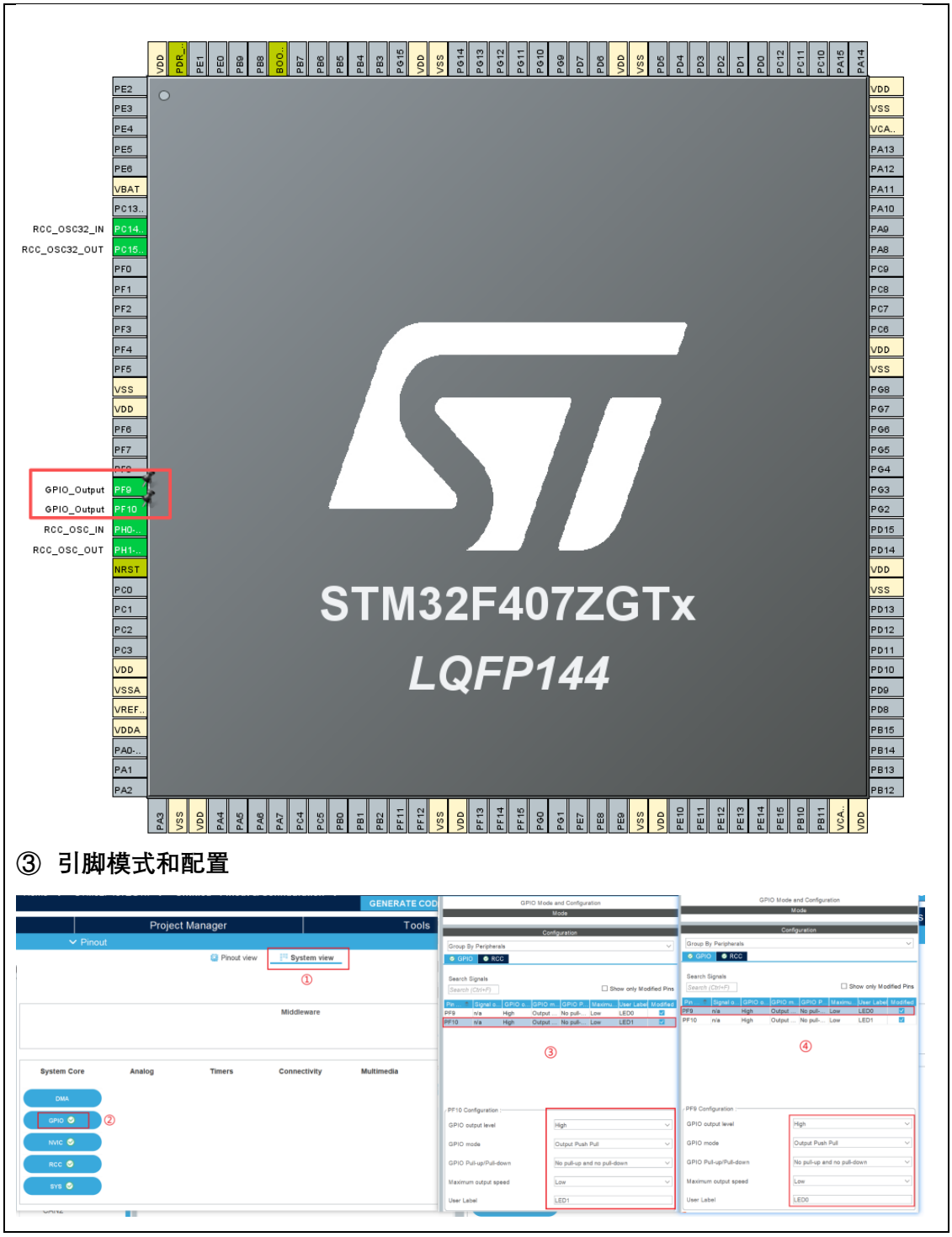
① 在原理图中找到 LED0 以及 LED1 对应的引脚号：

LED0:PF9

LED1:PF10



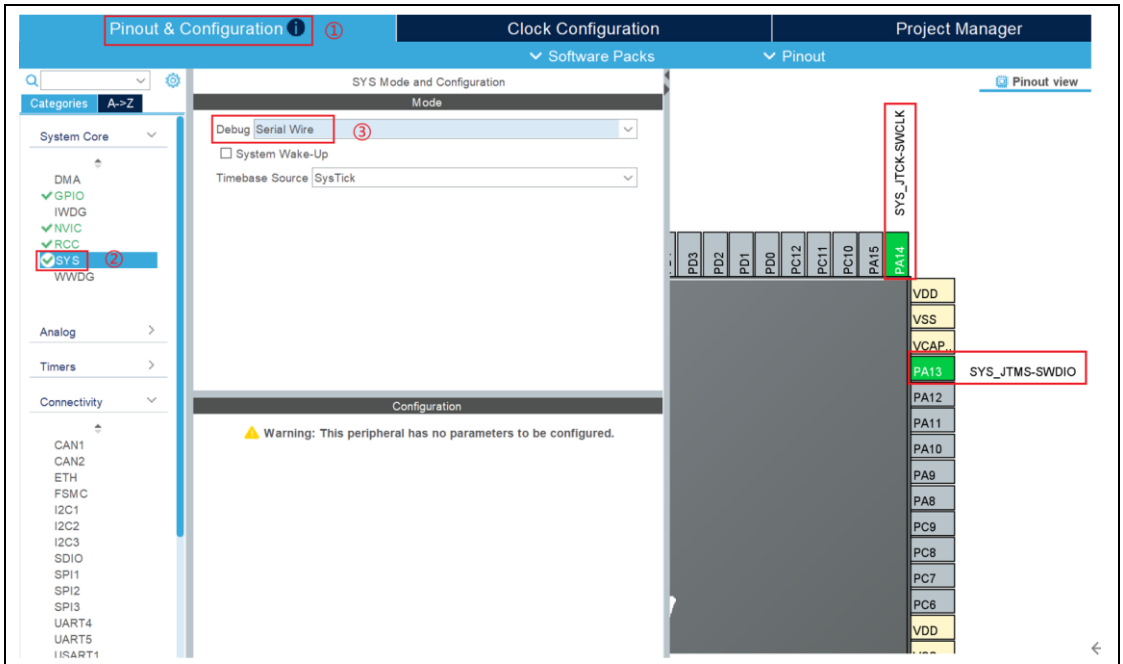
② 在 Pinout & Configuration 即“时钟模块设置”界面，在右下角搜索 PF9 和 PF10，搜到后会在芯片上闪烁黑点，点击然后设置为“GPIO_Output”



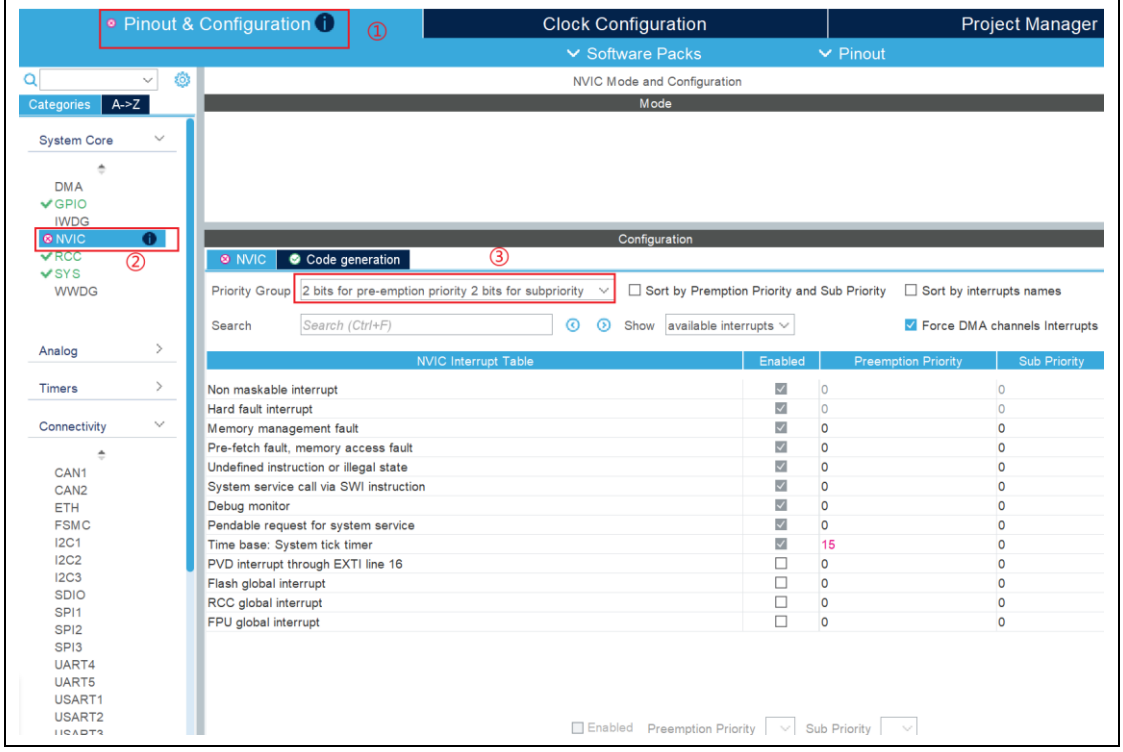
③ 引脚模式和配置

6.10.3.5 Cortex 内核配置

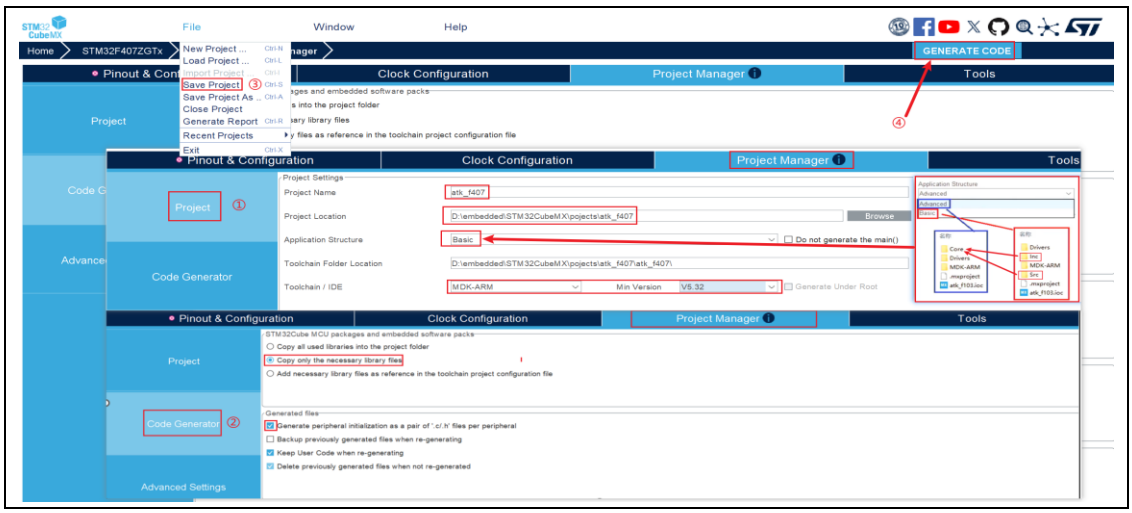
① SYS(DEBUG)配置



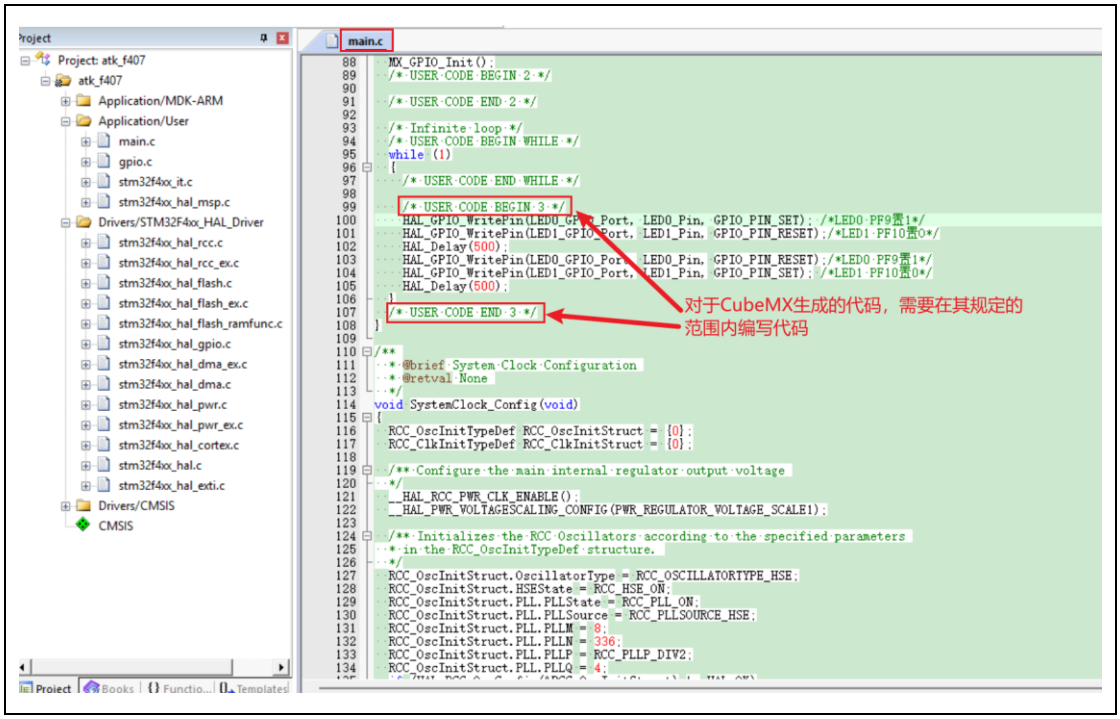
② NVIC(优先级分组)



6.10.3.6 生成工程源码

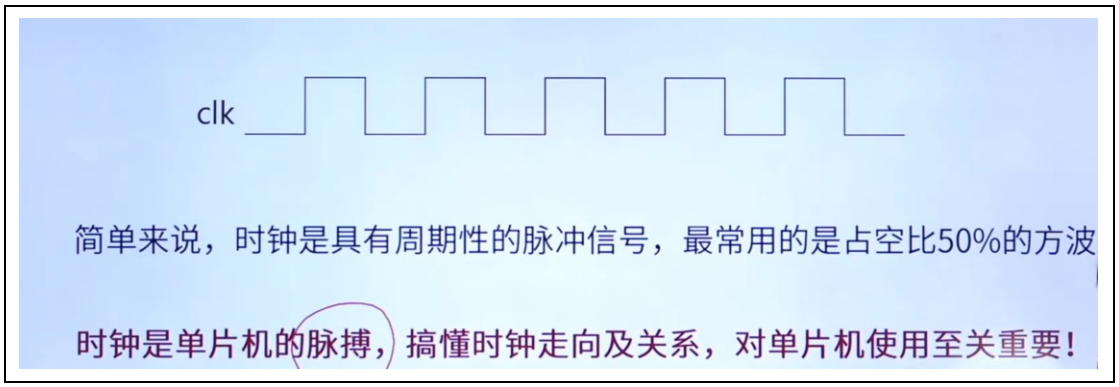


6.10.3.7　编写用户程序

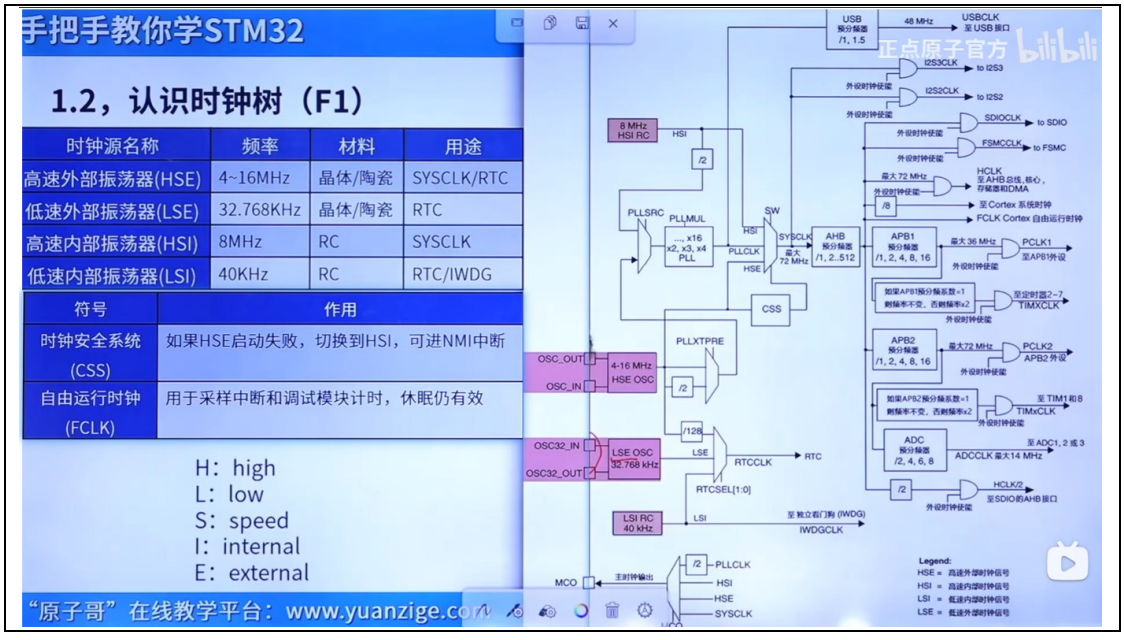


6.11 STM32 时钟系统

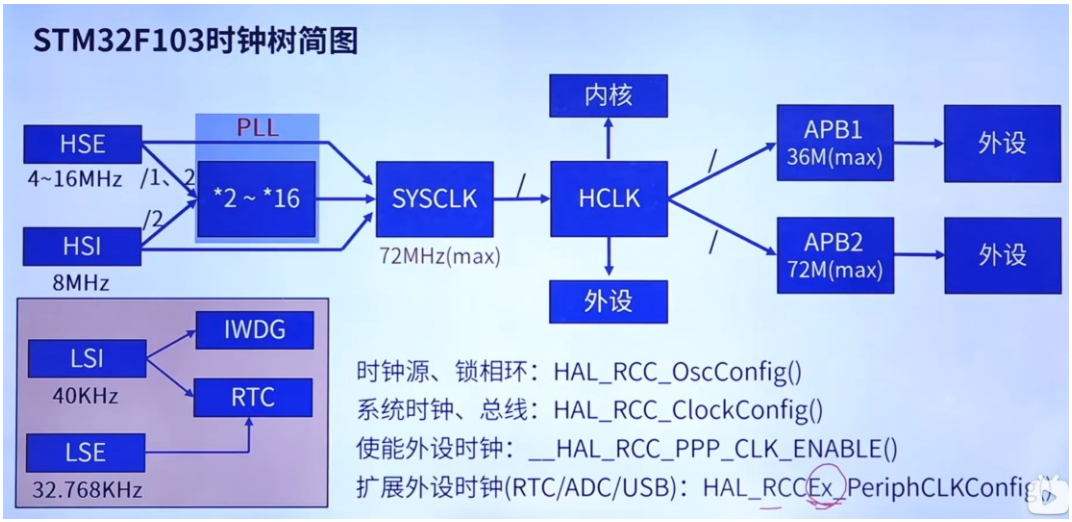
6.11.1 认识时钟树



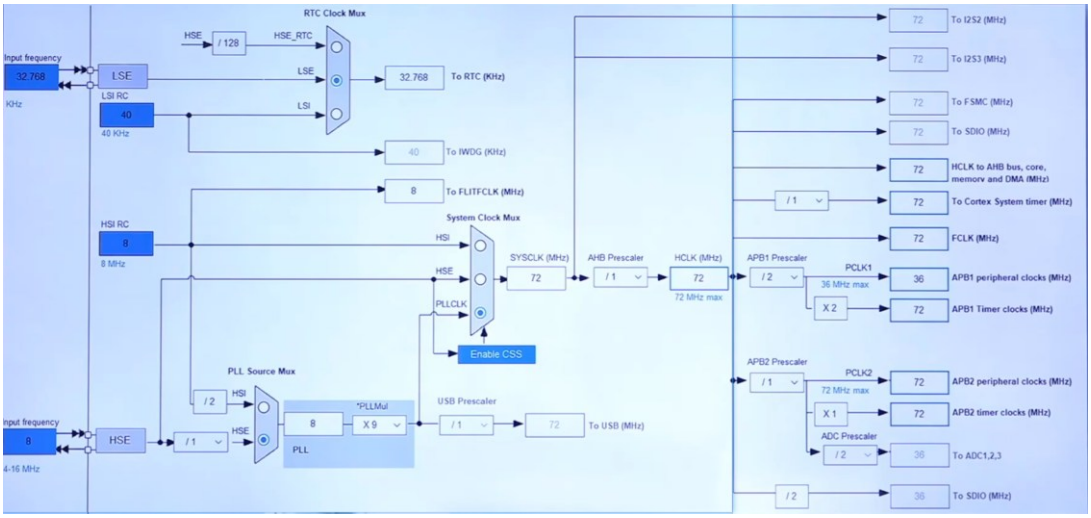
6.11.1.1　认识 F1 时钟树



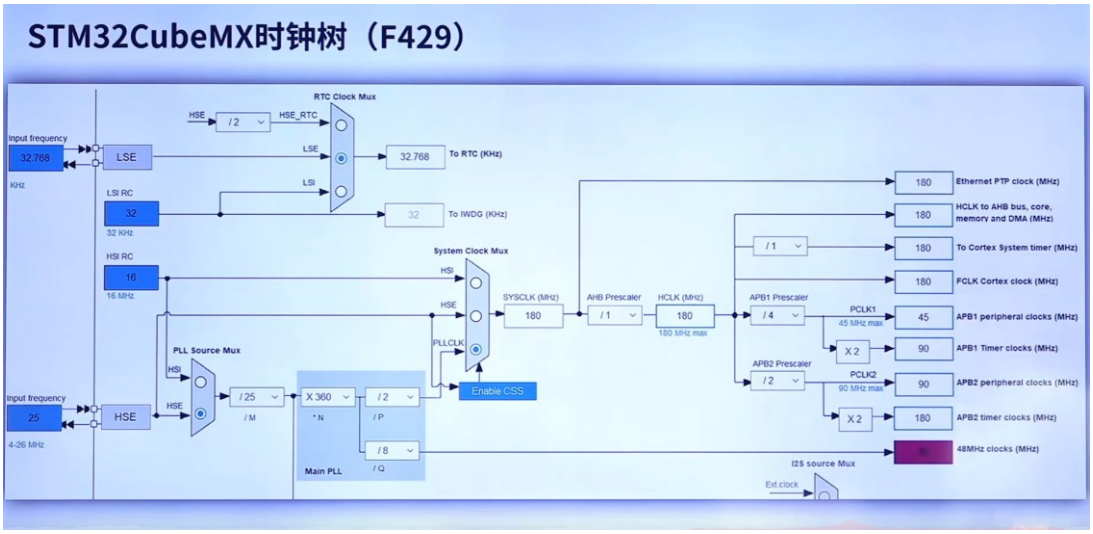
简图：



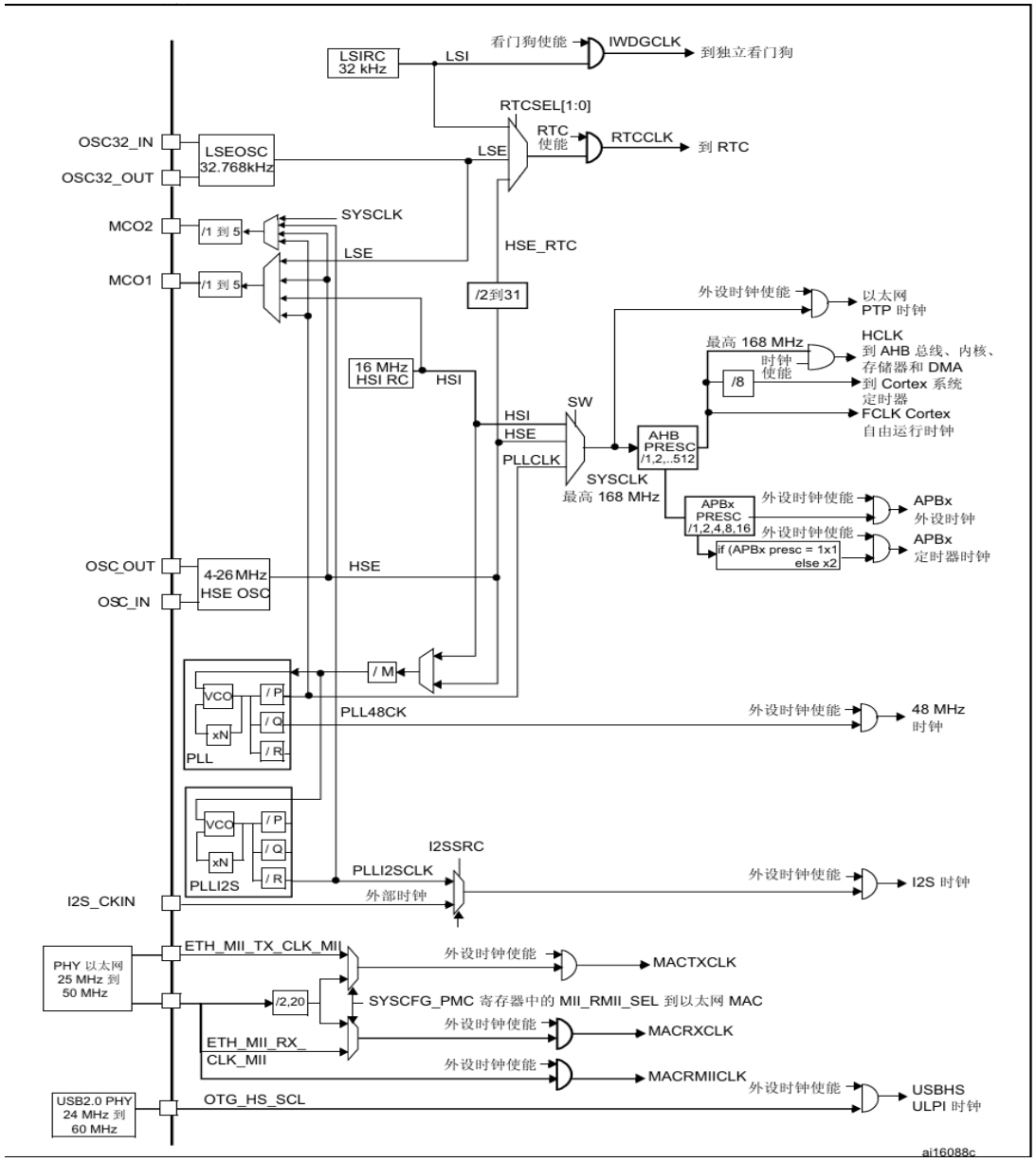
F407：

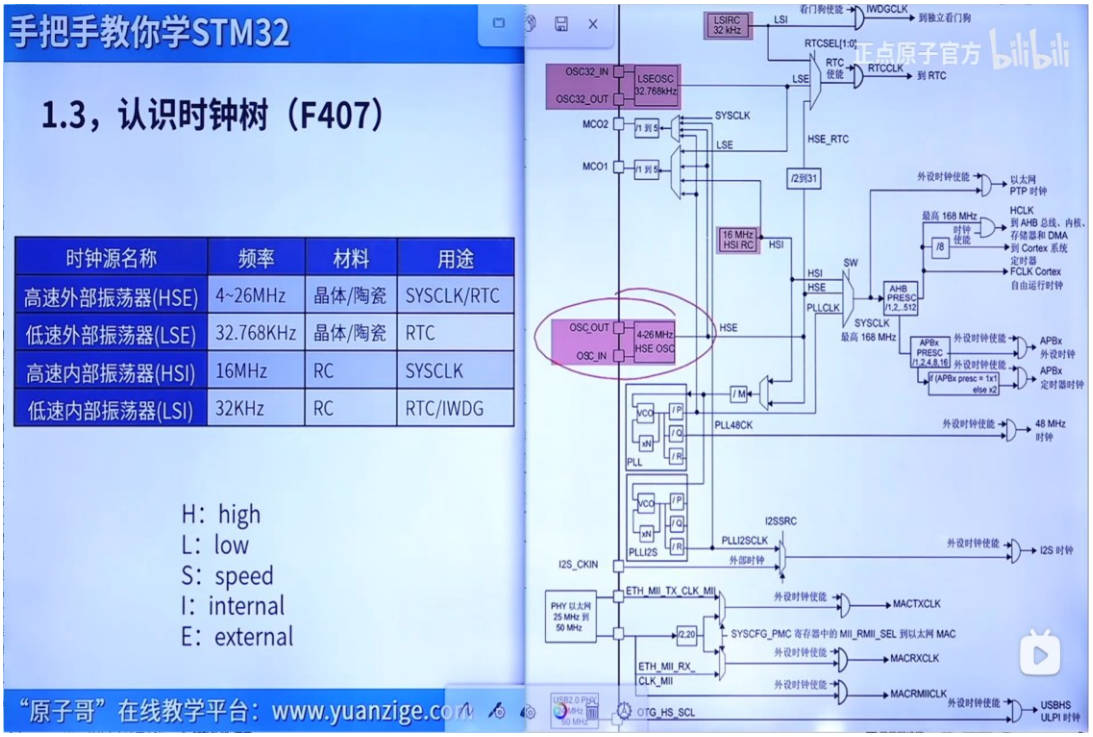


F429：

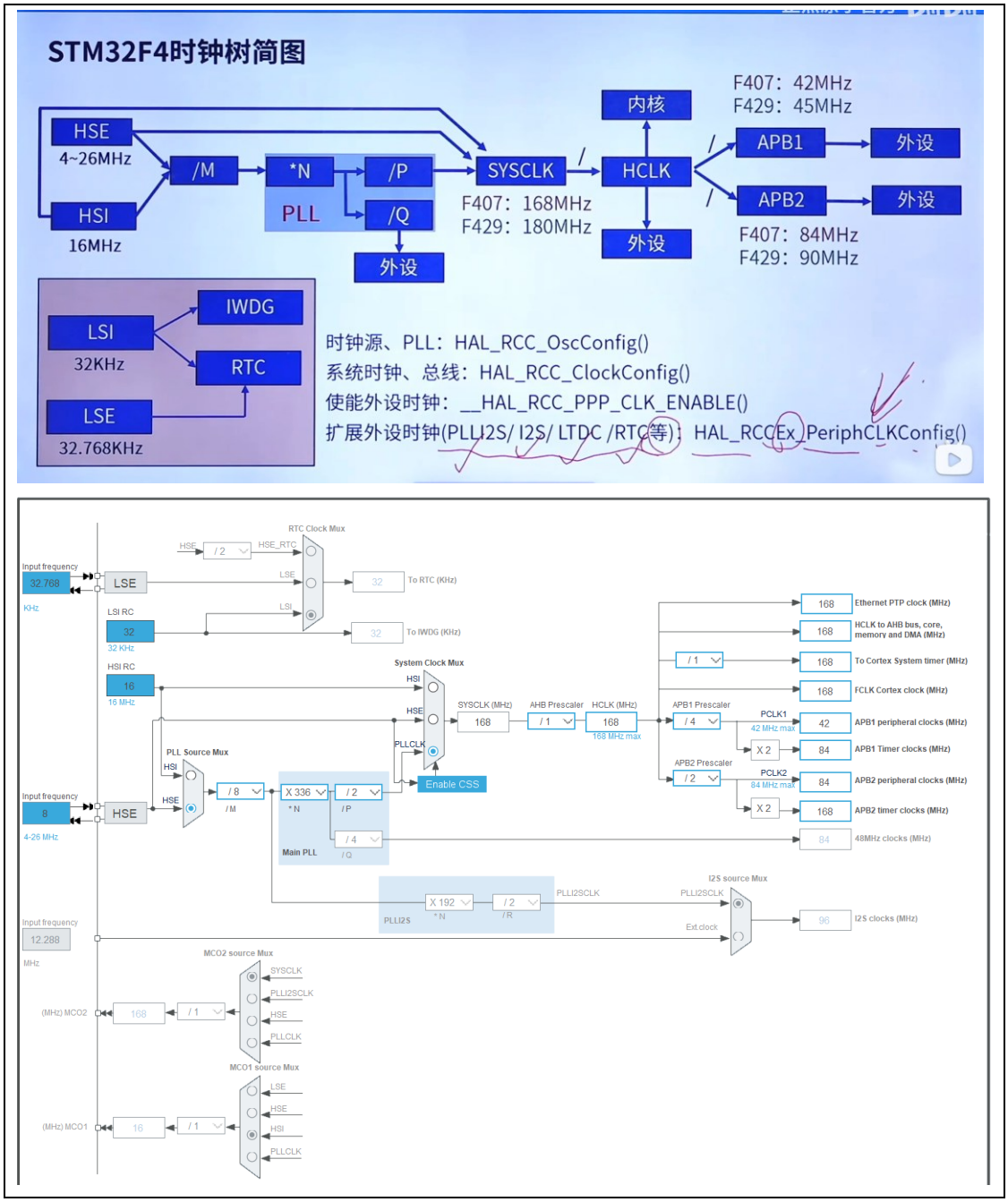


6.11.1.2　认识 F4 时钟树

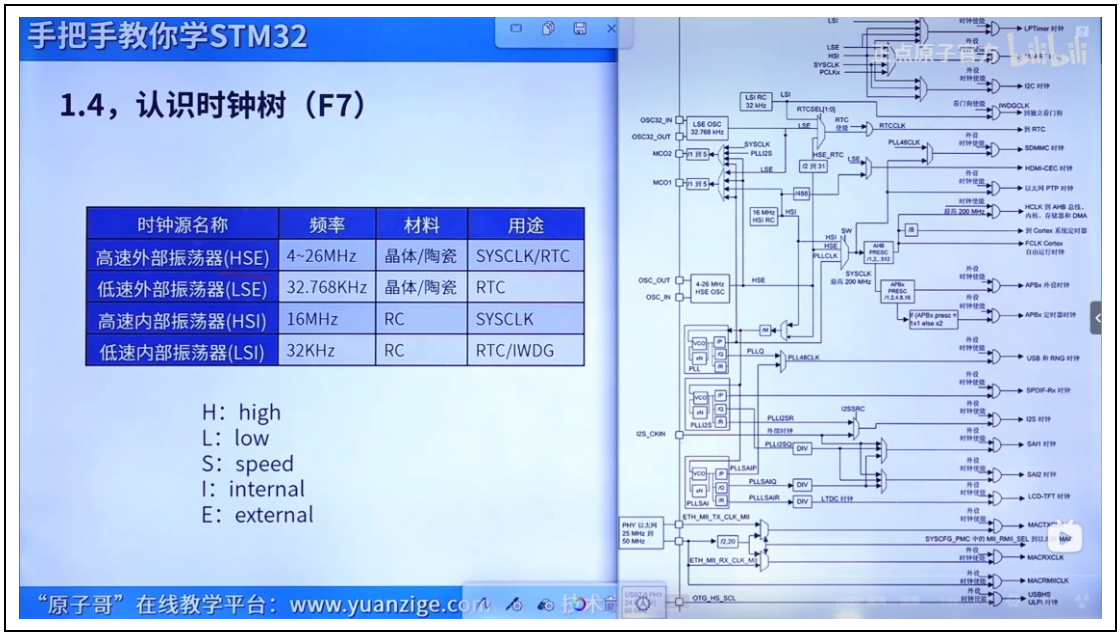

LSIRC 32 kHz
LSI
看门狗使能
IWDGCLK
到独立看门狗
RTCSEL[1:0]
RTC 使能
RTCCLK
到 RTC
OSC32_IN
OSC32_OUT
LSEOSC 32.768kHz
LSE
MCO2
MCO1
/1 到 5
SYSCLK
HSE_RTC
/2到31
外设时钟使能
以太网 PTP 时钟
最高 168 MHz
时钟使能
/8
HCLK 到 AHB 总线、内核、存储器和 DMA
到 Cortex 系统定时器
FCLK Cortex 自由运行时钟
16 MHz HSI RC
HSI
SW
HSE
PLLCLK
AHB PRESC /1,2,..512
SYSCLK 最高 168 MHz
APBx PRESC /1,2,4,8,16
APBx 外设时钟
APBx 定时器时钟
if (APBx presc = 1x1 else x2
OSC_OUT
OSC_IN
4-26 MHz HSE OSC
/ M
VCO
/ P
/ Q
/ R
xN
PLL
PLL48CK
48 MHz 时钟
PLLI2S
I2SSRC
PLLI2SCLK
外部时钟
I2S 时钟
I2S_CKIN
PHY 以太网 25 MHz 到 50 MHz
ETH_MII_TX_CLK_MII
MACTXCLK
/2,20
SYSCFG_PMC 寄存器中的 MII_RMII_SEL 到以太网 MAC
ETH_MII_RX_CLK_MII
MACRXCLK
MACRMIICLK
USB2.0 PHY 24 MHz 到 60 MHz
OTG_HS_SCL
USBHS ULPI 时钟
ai16088c


手把手教你学STM32
1.3，认识时钟树（F407）
时钟源名称 | 频率 | 材料 | 用途
高速外部振荡器(HSE) | 4~26MHz | 晶体/陶瓷 | SYSCLK/RTC
低速外部振荡器(LSE) | 32.768KHz | 晶体/陶瓷 | RTC
高速内部振荡器(HSI) | 16MHz | RC | SYSCLK
低速内部振荡器(LSI) | 32KHz | RC | RTC/IWDG
H: high
L: low
S: speed
I: internal
E: external
“原子哥”在线教学平台：www.yuanzige.com
正点原子官方

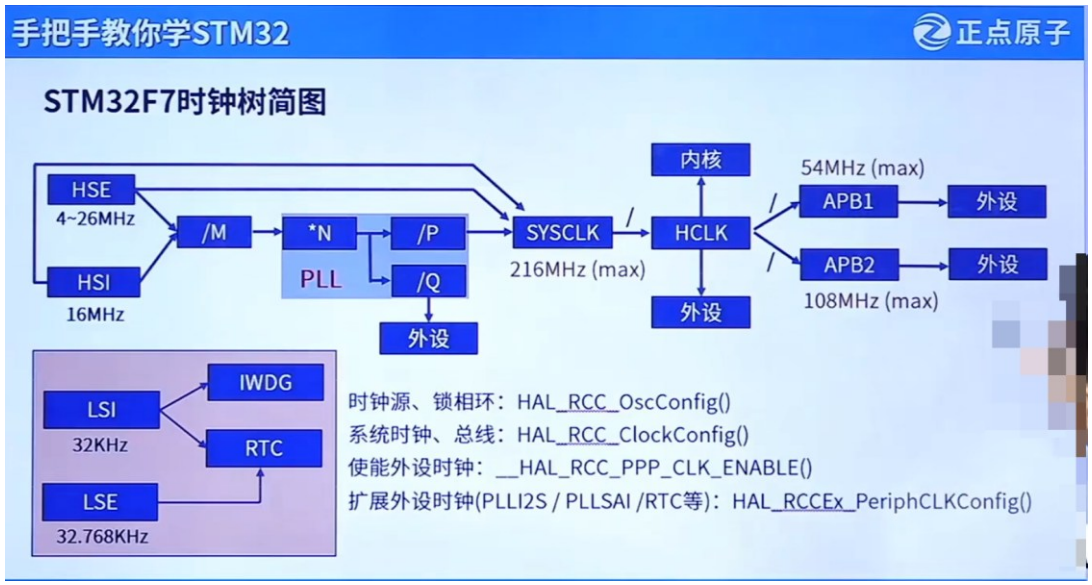
简图：

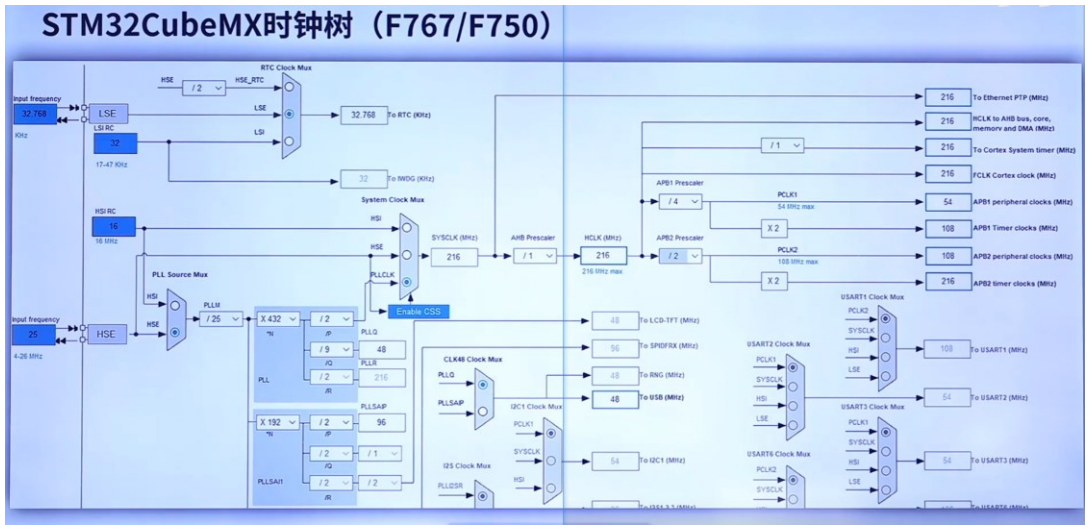


6.11.1.3 认识 F7 时钟树

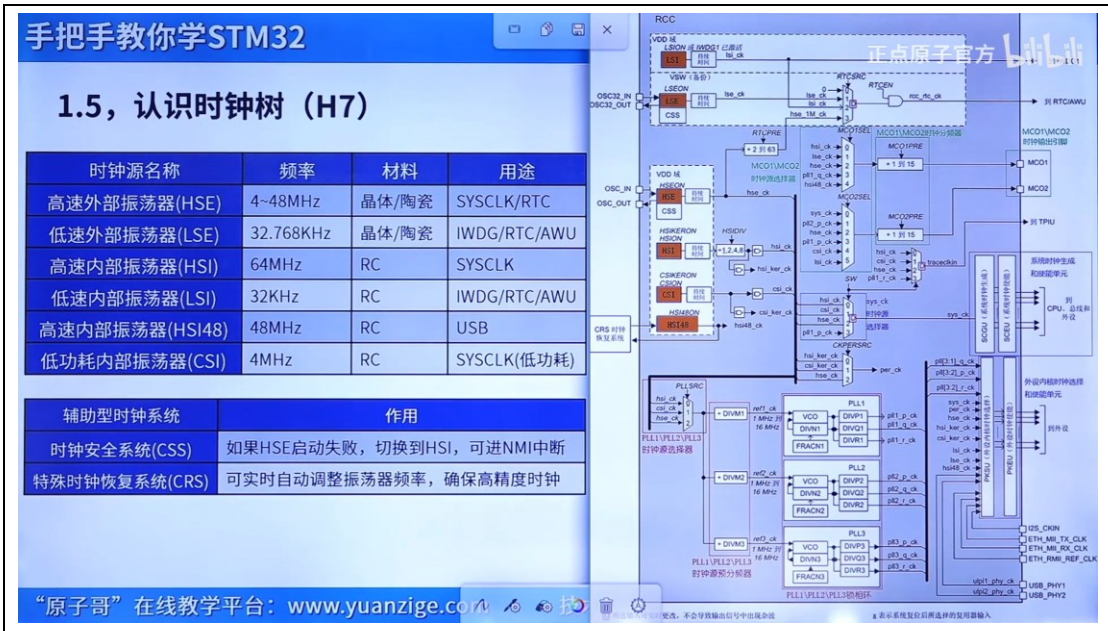


简图：

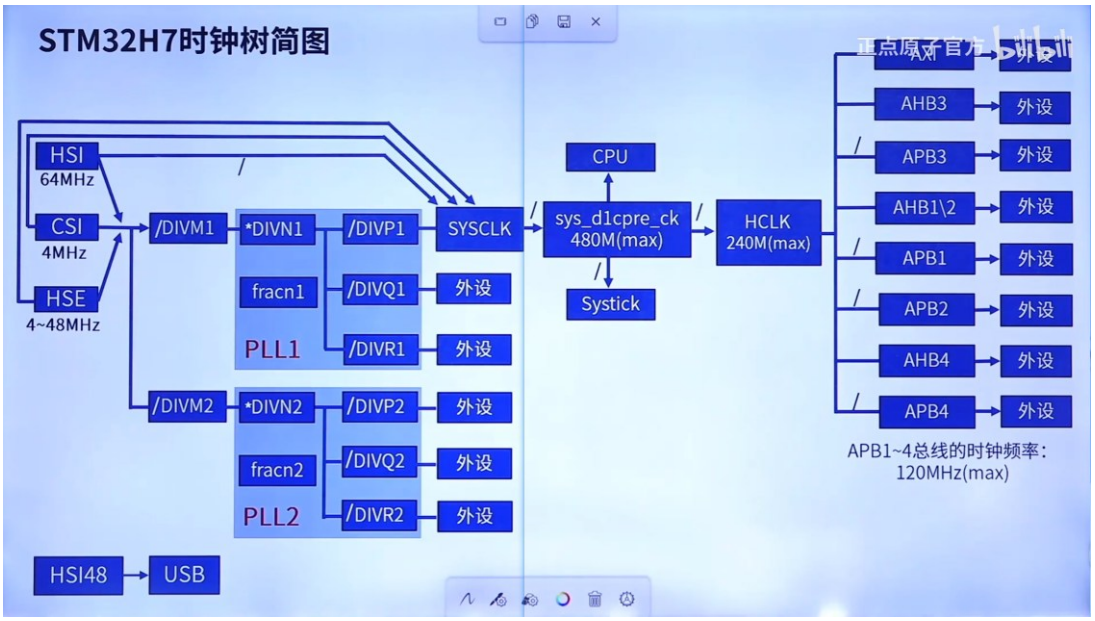




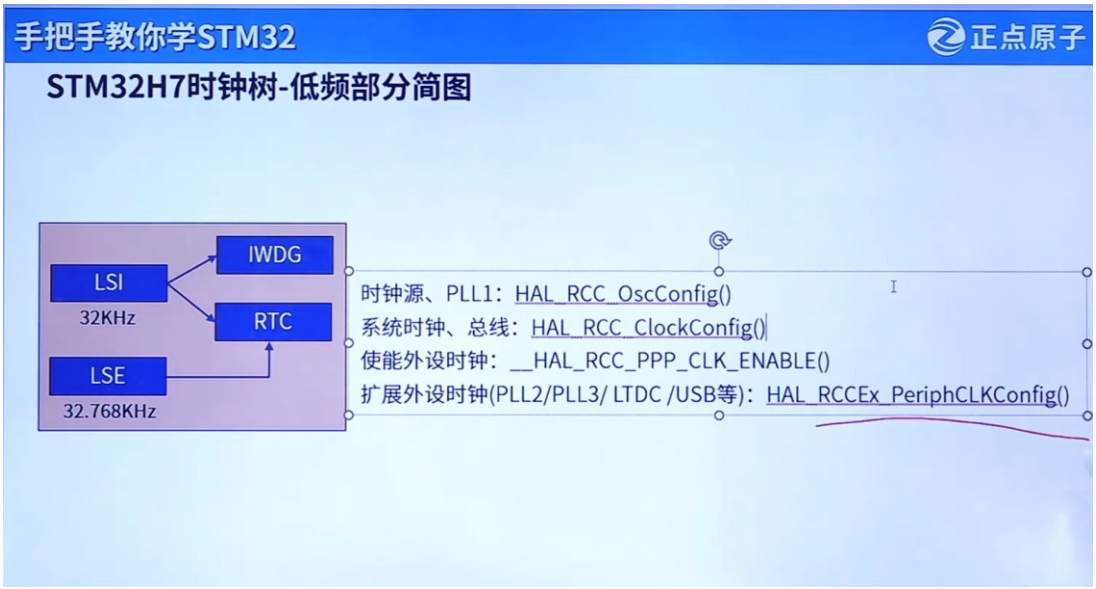
6.11.1.4 认识 H7 时钟树

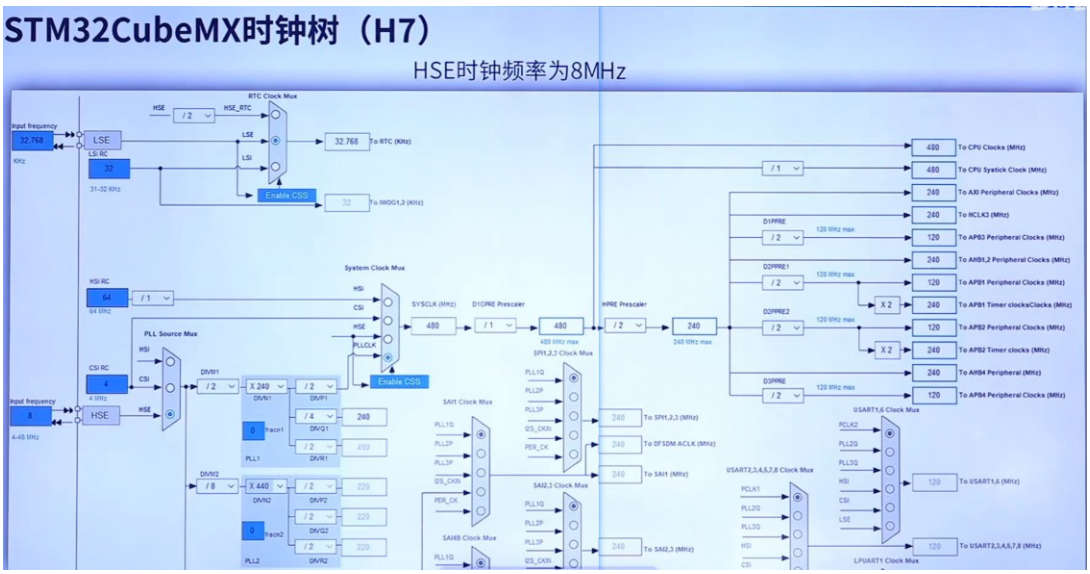


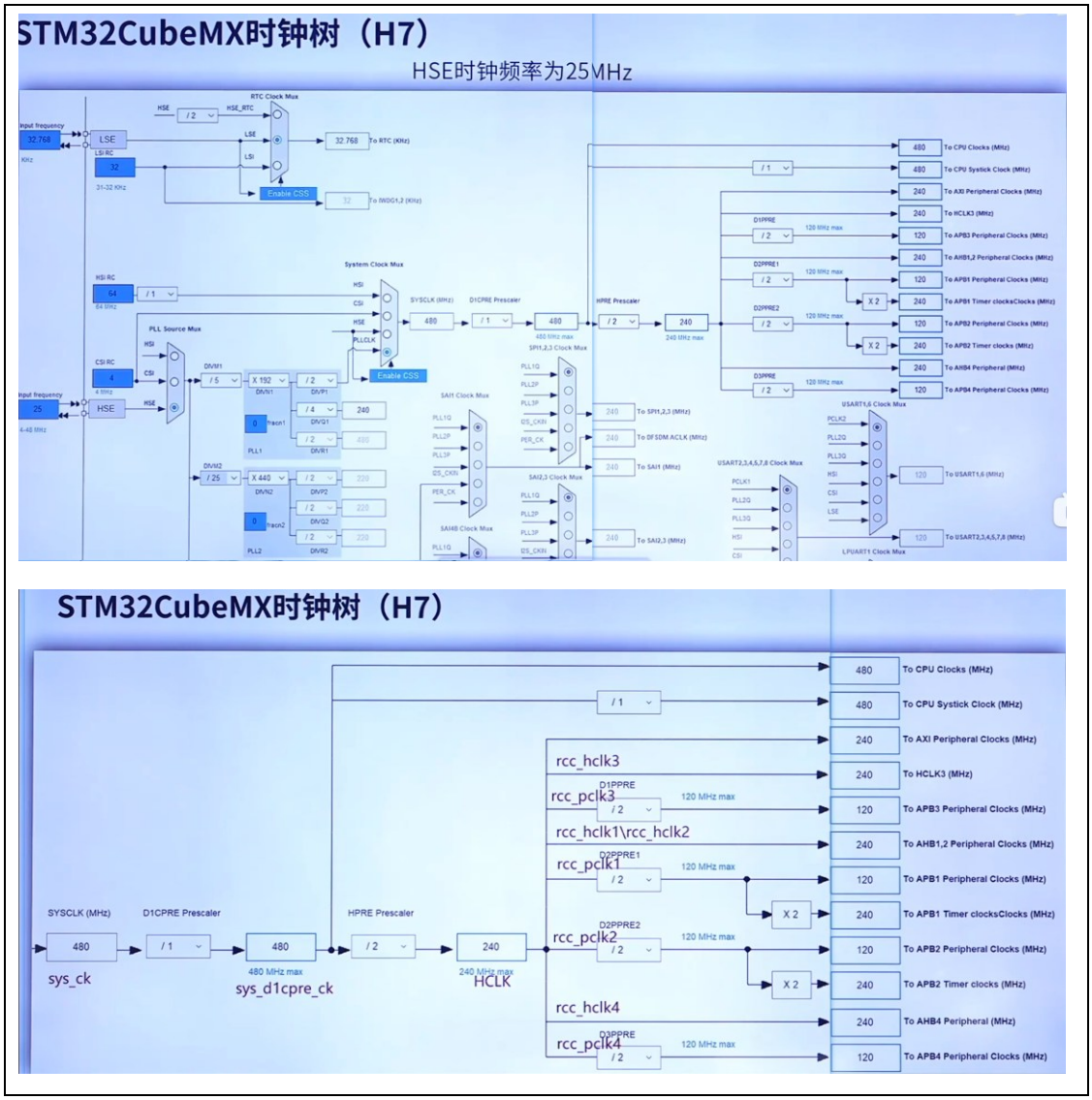
简图（高频）：


STM32H7时钟树简图
HSI
64MHz
CSI
4MHz
HSE
4~48MHz
/DIVM1
*DIVN1
fracn1
PLL1
/DIVP1
/DIVQ1
/DIVR1
外设
SYSCLK
CPU
sys_d1cpre_ck
480M(max)
Systick
HCLK
240M(max)
AHB3
APB3
AHB1\2
APB1
APB2
AHB4
APB4
APB1~4总线的时钟频率:
120MHz(max)
/DIVM2
*DIVN2
fracn2
PLL2
/DIVP2
/DIVQ2
/DIVR2
HSI48
USB

低频:

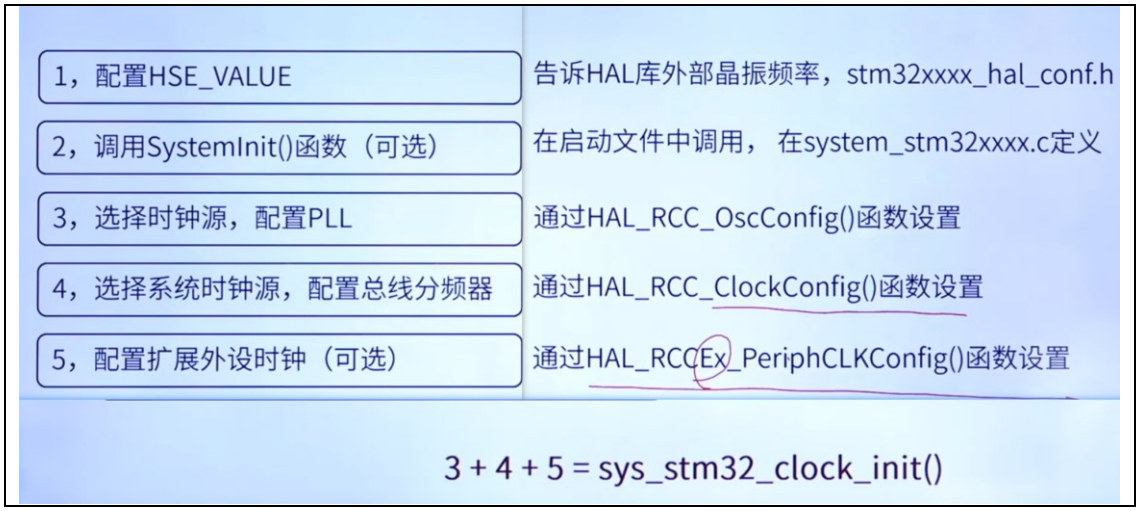

手把手教你学STM32
正点原子
STM32H7时钟树-低频部分简图
LSI
32KHz
LSE
32.768KHz
IWDG
RTC
时钟源、PLL1：HAL_RCC_OscConfig()
系统时钟、总线：HAL_RCC_ClockConfig()
使能外设时钟：__HAL_RCC_PPP_CLK_ENABLE()
扩展外设时钟(PLL2/PLL3/ LTDC /USB等)：HAL_RCCEx_PeriphCLKConfig()


STM32CubeMX时钟树（H7）
HSE时钟频率为8MHz



6.12 系统时钟配置

6.12.1 系统时钟配置步骤



6.12.2 外设时钟使能和失能

我们要使用某个外设，必需先使能该外设时钟！！！

HAL库使能某个外设时钟的方法，如：

```
__HAL_RCC_GPIOA_CLK_ENABLE();          /* 使能 GPIOA 时钟 */
```

HAL库禁止某个外设时钟的方法，如：

```
__HAL_RCC_GPIOA_CLK_DISABLE();         /* 禁止 GPIOA 时钟 */
```

6.12.3 Sys_stm32_clock_init 函数（F1）

1， HAL_RCC_OscConfig()函数

2， HAL_RCC_ClockConfig()函数

HAL_StatusTypeDef HAL_RCC_OscConfig(RCC_OscInitTypeDef *RCC_OscInitStruct)

```
typedef struct
{
  uint32_t OscillatorType;          /* 选择需要配置的振荡器 */
  uint32_t HSEState;                /* HSE 状态 */
  uint32_t HSEPredivValue;          /* HSE 预分频值 */
  uint32_t LSEState;                /* LSE 状态 */
  uint32_t HSIState;                /* HSI状态 */
  uint32_t HSICalibrationValue;     /* HSI 校准值 */
  uint32_t LSIState;                /* LSI 状态 */
  RCC_PLLInitTypeDef PLL;           /* PLL 结构体 */
}RCC_OscInitTypeDef;
```

```
typedef struct
{
  uint32_t PLLState;        /* PLL 状态 */
  uint32_t PLLSource;       /* PLL 时钟源 */
  uint32_t PLLMUL;          /* PLL 倍频系数 */
}RCC_PLLInitTypeDef;
```

HAL_StatusTypeDef HAL_RCC_ClockConfig(RCC_ClkInitTypeDef *RCC_ClkInitStruct, uint32_t FLatency)

```
typedef struct
{
  uint32_t ClockType;             /* 要配置的时钟（SYSCLK/HCLK/PCLK1/PCLK2） */
  uint32_t SYSCLKSource;          /* 系统时钟源 */
  uint32_t AHBCLKDivider;         /* AHB 时钟预分频系数 */
  uint32_t APB1CLKDivider;        /* APB1 时钟预分频系数 */
  uint32_t APB2CLKDivider;        /* APB2 时钟预分频系数 */
}RCC_ClkInitTypeDef;
```

F1 72M

```
uint32_t FLatency

#define FLASH_LATENCY_0  0x00000000U          /* FLASH 0个等待周期 */
#define FLASH_LATENCY_1  FLASH_ACR_LATENCY_0  /* FLASH 1个等待周期 */
#define FLASH_LATENCY_2  FLASH_ACR_LATENCY_1  /* FLASH 2个等待周期 */
```

24M

实际设置FLASH_ACR寄存器LATENCY位域，请参考《STM32F10xxx闪存编程参考手册.pdf》3.1小节

6.12.4 Sys_stm32_clock_init 函数（F4/F7）

1， HAL_RCC_OscConfig()函数

2， HAL_RCC_ClockConfig()函数

1， HAL_RCC_OscConfig()函数（F4/F7）

HAL_StatusTypeDef HAL_RCC_OscConfig(RCC_OscInitTypeDef *RCC_OscInitStruct)

```
typedef struct
{
  uint32_t OscillatorType;       /* 选择需要配置的振荡器 */
  uint32_t HSEState;             /* HSE 状态 */
  uint32_t LSEState;             /* LSE 状态 */
  uint32_t HSIState;             /* HSI 状态 */
  uint32_t HSICalibrationValue;  /* HSI 校准微调值，范围0x0~0x1F */
  uint32_t LSIState;             /* LSI 状态 */
  RCC_PLLInitTypeDef PLL;        /* PLL 结构体 */
}RCC_OscInitTypeDef;
```

```
typedef struct
{
  uint32_t PLLState;   /* PLL 状态 */
  uint32_t PLLSource;  /* PLL 时钟源 */
  uint32_t PLLM;       /* PLL 分频系数 M */
  uint32_t PLLN;       /* PLL 倍频系数 N */
  uint32_t PLLP;       /* PLL 分频系数 P */
  uint32_t PLLQ;       /* PLL 分频系数 Q */
}RCC_PLLInitTypeDef;
```

2， HAL_RCC_ClockConfig()函数（F4/F7）

HAL_StatusTypeDef HAL_RCC_ClockConfig(RCC_ClkInitTypeDef *RCC_ClkInitStruct, uint32_t FLatency)

```
typedef struct
{
  uint32_t ClockType;      /* 要配置的时钟（SYSCLK/HCLK/PCLK1/PCLK2） */
  uint32_t SYSCLKSource;   /* 系统时钟源 */
  uint32_t AHBCLKDivider;  /* AHB 时钟预分频系数 */
  uint32_t APB1CLKDivider; /* APB1 时钟预分频系数 */
  uint32_t APB2CLKDivider; /* APB2 时钟预分频系数 */
}RCC_ClkInitTypeDef;
```

```
uint32_t FLatency

#define FLASH_LATENCY_0  FLASH_ACR_LATENCY_0WS   /* FLASH 0个等待周期 */
#define FLASH_LATENCY_1  FLASH_ACR_LATENCY_1WS   /* FLASH 1个等待周期 */
#define FLASH_LATENCY_2  FLASH_ACR_LATENCY_2WS   /* FLASH 2个等待周期 */
...
#define FLASH_LATENCY_15  FLASH_ACR_LATENCY_15WS /* FLASH 15个等待周期 */
```

实际设置FLASH_ACR寄存器LATENCY位域

6.12.5 Sys_stm32_clock_init 函数（H7）

1，HAL_RCC_OscConfig()函数

2，HAL_RCC_ClockConfig()函数

3，HAL_RCCEx_PeriphCLKConfig()函数

1， HAL_RCC_OscConfig()函数（H7）

HAL_StatusTypeDef HAL_RCC_OscConfig(RCC_OscInitTypeDef *RCC_OscInitStruct)

```
typedef struct
{
      uint32_t OscillatorType;          /* 选择需要配置的振荡器 */
      uint32_t HSEState;                /* HSE 状态 */
      uint32_t LSEState;                /* LSE 状态 */
      uint32_t HSIState;                /* HSI 状态 */
      uint32_t HSICalibrationValue;     /* HSI 校准微调值 */
      uint32_t LSIState;                /* LSI 状态 */
      uint32_t HSI48State;              /* HSI48 状态 */
      uint32_t CSIState;                /* CSI 状态 */
      uint32_t CSICalibrationValue;     /* CSI 校准微调值 */
      RCC_PLLInitTypeDef PLL;           /* PLL 结构体 */
}RCC_OscInitTypeDef;
```

```
typedef struct
{
      uint32_t PLLState;          /* PLL1 状态 */
      uint32_t PLLSource;         /* PLL1 时钟源 */
      uint32_t PLLM;              /* PLL1 分频系数 M */
      uint32_t PLLN;              /* PLL1 倍频系数 N */
      uint32_t PLLP;              /* PLL1 分频系数 P */
      uint32_t PLLQ;              /* PLL1 分频系数 Q */
      uint32_t PLLR;              /* PLL1 分频系数 R */
      uint32_t PLLRGE;            /* PLL1 时钟输入范围 */
      uint32_t PLLVCOSEL;         /* PLL1 时钟输出范围 */
      uint32_t PLLFRACN;          /* PLL1 VCO 乘数因子的小数部分 */
}RCC_PLLInitTypeDef;
```

2，HAL_RCC_ClockConfig()函数（H7）

HAL_StatusTypeDef HAL_RCC_ClockConfig(RCC_ClkInitTypeDef *RCC_ClkInitStruct, uint32_t FLatency)

```
typedef struct
{
      uint32_t ClockType;             /* 要配置的时钟 */
      uint32_t SYSCLKSource;          /* 系统时钟源 */
      uint32_t SYSCLKDivider;         /* SYSCLK 分频系数 */
      uint32_t AHBCLKDivider;         /* AHB 时钟预分频系数 */
      uint32_t APB3CLKDivider;        /* APB3 时钟预分频系数 */
      uint32_t APB1CLKDivider;        /* APB1 时钟预分频系数 */
      uint32_t APB2CLKDivider;        /* APB2 时钟预分频系数 */
      uint32_t APB4CLKDivider;        /* APB4 时钟预分频系数 */
}RCC_ClkInitTypeDef;
```

```
uint32_t FLatency

#define FLASH_LATENCY_0  FLASH_ACR_LATENCY_0WS    /* FLASH 0个等待周期 */
#define FLASH_LATENCY_1  FLASH_ACR_LATENCY_1WS    /* FLASH 1个等待周期 */
#define FLASH_LATENCY_2  FLASH_ACR_LATENCY_2WS    /* FLASH 2个等待周期 */
...
#define FLASH_LATENCY_15  FLASH_ACR_LATENCY_15WS /* FLASH 15个等待周期 */
```

实际设置FLASH_ACR寄存器LATENCY位域

3， HAL_RCCEx_PeriphCLKConfig ()函数（H7）

HAL_StatusTypeDef HAL_RCCEx_PeriphCLKConfig(RCC_PeriphCLKInitTypeDef *PeriphClkInit)

```
typedef struct
{
        uint32_t PeriphClockSelection;          /* 要配置的扩展时钟 */
        RCC_PLL2InitTypeDef PLL2;               /* PLL2 时钟配置结构体 */
        RCC_PLL3InitTypeDef PLL3;               /* PLL3 时钟配置结构体 */
        uint32_t FmcClockSelection;             /* FMC 时钟源 */
        uint32_t QspiClockSelection;            /* QSPI 时钟源 */
        uint32_t Usart234578ClockSelection;     /* USART2/3/4/5/7/8 时钟源 */
        uint32_t Usart16ClockSelection;         /* USART1/6 时钟源 */
        uint32_t UsbClockSelection;             /* USB 时钟源 */
        ...省略...
}RCC_PeriphCLKInitTypeDef;
```

```
typedef struct
{
        uint32_t PLL2M;         /* PLL2 分频系数 M */
        uint32_t PLL2N;         /* PLL2 倍频系数 N */
        uint32_t PLL2P;         /* PLL2 倍频系数 P */
        uint32_t PLL2Q;         /* PLL2 倍频系数 Q */
        uint32_t PLL2R;         /* PLL2 倍频系数 R */
        uint32_t PLL2RGE;       /* PLL2 时钟输入范围 */
        uint32_t PLL2VCOSEL;    /* PLL2 时钟输出范围 */
        uint32_t PLL2FRACN;     /* PLL2 VCO 乘数因子的小数部分 */
}RCC_PLL2InitTypeDef;
```

6.13 SYSTEM 文件夹介绍

6.13.1 Sys 文件夹介绍

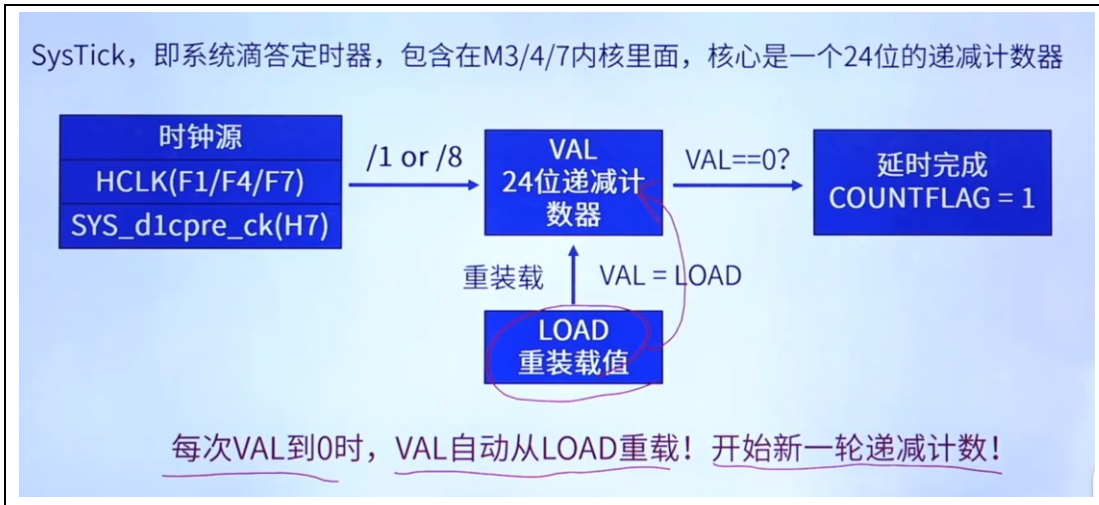
函数分类	相关函数	功能
中断类函数	sys_nvic_set_vector_table()	设置中断向量表地址
	sys_intx_enable()	开启所有中断
	sys_intx_disable()	关闭所有中断(但是不包括fault和NMI中断)
低功耗类函数	sys_wfi_set()	执行: WFI指令(执行完该指令进入低功耗状态)
	sys_standby()	进入待机模式
	sys_soft_reset()	系统软复位
设置栈顶地址函数	sys_msr_msp()	设置栈顶地址
系统时钟初始化函数	sys_stm32_clock_init()	设置系统时钟
Cache配置函数(F7/H7)	sys_cache_enable()	使能I-Cache和D-Cache, 开启D-Cache强制透写

6.13.2 Deley 文件夹介绍

6.13.2.1 Deley 文件夹函数简介

函数分类	相关函数	功能
使用OS	…	…
不使用OS	delay_init()	初始化系统滴答定时器
	delay_us()	用系统滴答定时器实现微秒延时
	delay_ms()	用微秒延时函数实现毫秒延时

6.13.2.2 SysTick 工作原理



SysTick控制及状态寄存器(CTRL)

位段	名称	类型	复位值	描述
16	COUNTFLAG	R	0	如果在上次读取本寄存器后，SysTick 已经数到了 0，则该位为 1。如果读取该位，该位将自动清零
2	CLKSOURCE	R/W	0	0=外部时钟源(STCLK) 8分频 1=内核时钟(FCLK) 1分频
1	TICKINT	R/W	0	1=SysTick 倒数到 0 时产生 SysTick 异常请求 0=数到 0 时无动作
0	ENABLE	R/W	0	SysTick 定时器的使能位

6.13.2.3 SysTick 寄存器介绍

SysTick重装载数值寄存器(LOAD)

位段	名称	类型	复位值	描述
23:0	RELOAD	R/W	0	当倒数至零时，将被重装载的值

SysTick当前数值寄存器(VAL)

位段	名称	类型	复位值	描述
23:0	CURRENT	R/Wc	0	读取时返回当前倒计数的值，写它则使之清零，同时还会清除在 SysTick 控制及状态寄存器中的 COUNTFLAG 标志

6.13.2.4 Delay_init()函数介绍

```
/**
 * @brief   初始化延迟函数
 * @param   sysclk: 系统时钟频率, 即CPU频率(rcc_c_ck), 168MHz
 * @retval  无
 */
void delay_init(uint16_t sysclk)
{
#if SYS_SUPPORT_OS                                  /* 如果需要支持OS */
    uint32_t reload;
#endif
    g_fac_us = sysclk;                              /* 由于在HAL_Init中已对systick做了配置，所以这里无需重新配置 */
#if SYS_SUPPORT_OS                                  /* 如果需要支持OS. */
    reload = sysclk;                                /* 每秒钟的计数次数 单位为M */
    reload *= 1000000 / delay_ostickspersec;        /* 根据delay_ostickspersec设定溢出时间,reload为24位
                                                     * 寄存器,最大值:16777216,在168M下,约合0.09986s左右
                                                     */
    g_fac_ms = 1000 / delay_ostickspersec;          /* 代表OS可以延时的最少单位 */
    SysTick->CTRL |= 1 << 1;                        /* 开启SYSTICK中断 */
    SysTick->LOAD = reload;                         /* 每1/delay_ostickspersec秒中断一次 */
    SysTick->CTRL |= 1 << 0;                        /* 开启SYSTICK */
#endif
}
```

6.13.2.5 Delay_us()函数

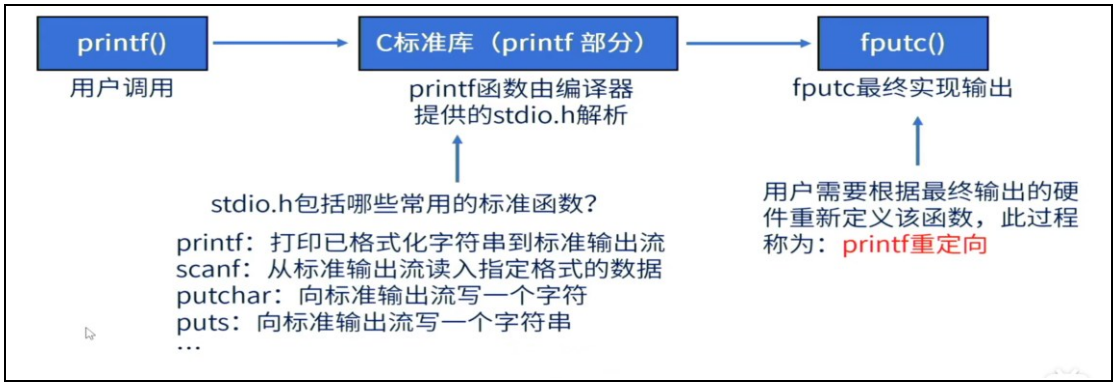
```c
/**
 * @brief     延时nus
 * @note      无论是否使用OS，都是用时钟摘取法来做us延时
 * @param     nus: 要延时的us数
 * @note      nus取值范围: 0 ~ (2^32 / fac_us) (fac_us一般等于系统主频，自行套入计算)
 * @retval    无
 */
void delay_us(uint32_t nus)
{
    uint32_t ticks;
    uint32_t told, tnow, tcnt = 0;
    uint32_t reload = SysTick->LOAD;                /* LOAD的值 */
    ticks = nus * g_fac_us;                         /* 需要的节拍数 */

#if SYS_SUPPORT_OS                                  /* 如果需要支持OS */
    delay_osschedlock();                            /* 锁定OS的任务调度器 */
#endif

    told = SysTick->VAL;                            /* 刚进入时的计数器值 */
    while (1)
    {
        tnow = SysTick->VAL;
        if (tnow != told)
        {
            if (tnow < told)
            {
                tcnt += told - tnow;                /* 这里注意一下SYSTICK是一个递减的计数器就可以了 */
            }
            else
            {
                tcnt += reload - tnow + told;
            }
            told = tnow;
            if (tcnt >= ticks)
            {
                break;                              /* 时间超过/等于要延迟的时间,则退出 */
            }
        }
    }

#if SYS_SUPPORT_OS                                  /* 如果需要支持OS */
    delay_osschedunlock();                          /* 恢复OS的任务调度器 */
#endif

}
```

6.13.2.6 Delay_ms()函数介绍

```c
/**
 * @brief     延时nms
 * @param     nms: 要延时的ms数 (0< nms <= (2^32 / fac_us / 1000))(fac_us一般等于系统主频，自行套入计算)
 * @retval    无
 */
void delay_ms(uint16_t nms)
{

#if SYS_SUPPORT_OS  /* 如果需要支持OS，则根据情况调用os延时以释放CPU */
    if (delay_osrunning && delay_osintnesting == 0)     /* 如果OS已经在跑了，并且不是在中断里面(中断里面不能任务调度) */
    {
        if (nms >= g_fac_ms)                            /* 延时的时间大于OS的最少时间周期 */
        {
            delay_ostimedly(nms / g_fac_ms);            /* OS延时 */
        }

        nms %= g_fac_ms;                                /* OS已经无法提供这么小的延时了,采用普通方式延时 */
    }
#endif

    delay_us((uint32_t)(nms * 1000));                   /* 普通方式延时 */
}
```

6.13.3 Ustart 文件夹介绍

6.13.3.1 Printf 函数输出流程



6.13.3.2 Printf 的使用

1，printf("字符串\r\n");

```
例：printf("Hello World!\r\n");
```

2，printf("输出控制符"，输出参数);

```
例：uint32_t temp = 10;
    printf("%d\r\n", temp);      /* %d是输出控制符，temp是输出参数 */
```

常用输出控制符表

控制符	说明
%d	按十进制整型数据的实际长度输出
%ld	输出长整型数据
%md	m为指定的输出字段的宽度。如果数据的位数小于m，则左端补以空格，若大于m，则按实际位数输出
%c	用来输出一个字符
%s	用来输出字符串
%u	输出无符号整型（unsigned）
%f	输出实数，包括单精度和双精度，以小数形式输出
%x	以十六进制形式输出整数

常用转义字符表

控制符	说明
\r	回车，将当前位置移到本行开头
\n	换行，将当前位置移到下一行
\t	水平制表(HT)（跳到下一个TAB位置）
\\	代表一个反斜线字符
\'	代表一个单引号字符
\"	代表一个双引号字符
\?	代表一个问号
\0	空字符(NUL)

3，printf("输出控制符1输出控制符2…"，输出参数1，输出参数2，…);

```
例：  uint32_t temp1 = 5;
      uint32_t temp2 = 10;
      printf("%d%x \r \n", temp1,temp2);
```

4，printf("非输出控制符 输出控制符 非输出控制符"，输出参数);

```
例：uint32_t temp = 10;
    printf("temp=  %d  收到over\r\n", temp);
```

5，如何输出%、\和双引号

```
例：printf("%% \r\n");
    printf("\\\r\n");
    printf("\"\"\r\n");
```

6.13.3.3 Printf 函数的支持

1，避免使用半主机模式　两种方法：微库法、代码法

2，实现fputc函数　实现单个字符输出

半主机模式简介

用于 ARM 目标的一种机制，可将来自应用程序代码的输入/输出请求传送至运行调试器的主机

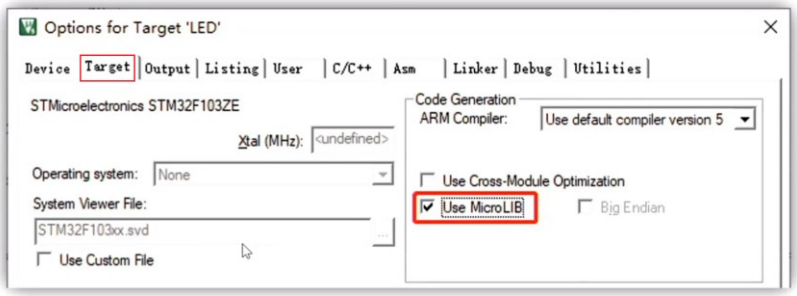
简单说：就是通过仿真器实现开发板在电脑上的输入和输出

一般我们：不使用半主机模式！！！

参考：https://blog.csdn.net/weixin_40093087/article/details/77979108

方法一：微库法

在魔术棒→Target选项卡，勾选：Use Micro LIB，即可避免半主机模式



方法二：代码法

1个预处理、 2个定义、3个函数

1，#pragma import(__use_no_semihosting)，确保不从C库中使用半主机函数

2，定义：__FILE结构体，避免HAL库某些情况下报错

3，定义： FILE __stdout，避免编译报错

4，实现：_ttywrch、_sys_exit和_sys_command_string等三个函数

AC5和AC6不使用半主机模式稍有差异，详见源码

微库法 VS 代码法

方法	优点	缺点
微库法	简单	某些标准C库函数运行慢、兼容性差
代码法	标准C库函数运行快、兼容性好	稍微复杂

代码法：

```
/*****************************************-代码法-start-
***********************************************/
/* 加入以下代码，支持 printf 函数，而不需要选择 use MicroLIB */

#if 1
#if (__ARMCC_VERSION >= 6010050)                    /* 使用 AC6 编译器时
*/
__asm(".global __use_no_semihosting\n\t");          /* 声明不使用半主机模式
*/
__asm(".global __ARM_use_no_argv \n\t");            /* AC6 下需要声明 main 函
数为无参数格式，否则部分例程可能出现半主机模式 */

#else
/* 使用 AC5 编译器时，要在这里定义__FILE 和 不使用半主机模式 */
#pragma import(__use_no_semihosting)

struct __FILE
{
    int handle;
    /* Whatever you require here. If the only file you are using is */
    /* standard output using printf() for debugging, no file handling */
    /* is required. */
};

#endif

/* 不使用半主机模式，至少需要重定义_ttywrch\_sys_exit\_sys_command_string
函数,以同时兼容 AC6 和 AC5 模式 */
int _ttywrch(int ch)
{
    ch = ch;
    return ch;
}

/* 定义_sys_exit()以避免使用半主机模式 */
void _sys_exit(int x)
{
    x = x;
}

char *_sys_command_string(char *cmd, int len)
```

```
{
    return NULL;
}

/* FILE 在 stdio.h 里面定义. */
FILE __stdout;

/*
重定义 fputc 函数, printf 函数最终会通过调用 fputc 输出字符串到串口
微库法，代码法这个函数都不能注释掉
*/
int fputc(int ch, FILE *f)
{
    while ((USART1->SR & 0X40) == 0);                    /* 等待上一个字符发送
完成 */

    USART1->DR = (uint8_t)ch;                            /* 将要发送的字符 ch
写入到 DR 寄存器 */
    return ch;
}
#endif
/*********************************************-代码法-END-
*******************************************/
```

实现fputc函数

```
#define USART_UX                USART1
/* 重定义 fputc 函数, printf 函数最终会通过调用 fputc 输出字符串到串口 */
int fputc(int ch, FILE *f)
{
   while ((USART_UX->SR & 0X40) == 0); /* 等待上一个字符发送完成 */
   USART_UX->DR = (uint8_t)ch;         /* 将要发送的字符 ch 写入到 DR 寄存器 */
   return ch;
}
```

屏蔽这个，是啥结果?